Dispensa di Ingegneria del Software

Fondamenti, progettazione, qualità ed evoluzione

Indice

1 Fondamenti di ingegneria del software e qualità **1**
1.1 Ingegneria del software e prodotto software . . . 1
1.2 Correttezza, requisiti e attributi di qualità . . . 1
1.2.1 Dependability e compromessi fra manutenibilità ed efficienza . . . 2
1.3 Responsabilità, coesione e accoppiamento . . . 2
1.3.1 KISS, DRY, YAGNI e Law of Demeter . . . 3
1.3.2 Tell, Don't Ask e separazione command–query . . . 3
1.4 Verifica, validazione e cambiamento controllato . . . 4
1.4.1 Strumenti di IA e responsabilità del progettista . . . 4
1.5 Dai bisogni a codice e test: procedimento operativo . . . 4

2 Java e programmazione orientata agli oggetti **6**
2.1 Java: compilazione ed esecuzione . . . 6
2.1.1 JDK, JRE, JVM, bytecode e JIT . . . 6
2.1.2 Compilazione, avvio e metodo main . . . 6
2.2 Classi, oggetti, campi e accesso allo stato . . . 7
2.2.1 Un programma completo: membri statici e membri di istanza . . . 8
2.3 Package, collection, tipi generici e iterazione . . . 8
2.3.1 Generics, boxing e tipi primitivi . . . 9
2.3.2 Enhanced for e Iterator . . . 9
2.4 Interfacce, classi astratte e contratti . . . 9
2.4.1 Programmare per le interfacce . . . 9
2.4.2 Classi astratte e differenze rispetto alle interfacce . . . 10
2.4.3 Contratto e sostituzione: IAccount e AccountV2 . . . 10
2.4.4 Stato comune e operazione astratta: Libro . . . 11
2.5 Ereditarietà, specializzazione e riuso . . . 11
2.5.1 Esempio Persona, Studente ed Esame . . . 11
2.6 Compatibilità dei tipi, cast e uguaglianza . . . 13
2.6.1 Identità degli oggetti e confronto del contenuto . . . 13
2.7 Polimorfismo, binding e dispatch . . . 13
2.7.1 Overload e override . . . 14
2.7.2 Account e SavingAccount: chiamata dalla superclasse alla sottoclasse . . . 15
2.8 Record e immutabilità dei dati . . . 15
2.9 Inferenza del tipo delle variabili locali . . . 16
2.10 Caso progressivo: leggere e aggregare importi . . . 16
2.10.1 Dalla lettura monolitica alla separazione dei compiti . . . 16
2.10.2 Pagamenti con comandi di calcolo e query di lettura . . . 17
2.10.3 Pagamenti con risultati calcolati su richiesta . . . 18
2.10.4 Una raccolta comune o una per file? . . . 19
2.11 Progettazione a oggetti: procedimento operativo . . . 20

3 Requisiti, UML e analisi orientata agli oggetti **21**
3.1 Requisiti, specifiche e feature . . . 21
3.2 Dai requisiti alla struttura del sistema . . . 22
3.3 Casi di uso: attori, confine e obiettivi . . . 22
3.4 Diagrammi delle attività: decisioni, parallelismo e corsie . . . 23
3.5 Diagrammi degli stati: eventi, guardie e azioni . . . 26
3.6 Diagramma delle classi e relazioni strutturali . . . 28
3.6.1 Dalle relazioni ai campi di Ordine . . . 28
3.7 Interazioni: collaborazione e sequenza . . . 29
3.7.1 Seguire una sequenza completa: MainEsami . . . 30
3.8 Individuare classi, attributi e operazioni . . . 32

4 Testing, JUnit e verifica della qualità **33**
4.1 Casi di test, oracle e debugging . . . 33
4.2 Verifica, validazione e livelli di test . . . 33
4.3 Progettazione black-box e white-box . . . 34
4.3.1 Intervallo ammesso, confini e stress . . . 35
4.4 Osservabilità, contratti e indipendenza dei test . . . 35
4.5 TDD, BDD e struttura del caso di test . . . 36
4.5.1 Dallo scheletro compilabile al test Red . . . 36
4.6 Copertura, percorsi e criteri di arresto . . . 36
4.7 JUnit: annotazioni, asserzioni e fixture . . . 37
4.7.1 Catalogo: test di somma e massimo . . . 38
4.7.2 Variante con due prodotti e preparazione indipendente . . . 39
4.7.3 Test eseguibili di Pagamenti e lettura da file . . . 40
4.7.4 Un metodo con input intero: scheletro, test e soluzione . . . 41
4.8 Maven: progetto, dipendenze e ciclo di build . . . 41
4.8.1 Creare il progetto e fissare la configurazione . . . 42
4.9 Procedimento per progettare ed eseguire un test . . . 43

5 Refactoring e miglioramento della struttura del codice **44**
5.1 Obiettivo e limite del refactoring . . . 44
5.1.1 Invarianza e sequenza di lavoro . . . 45
5.2 Code smell e scelta della trasformazione . . . 45
5.3 Estrai Metodo . . . 46
5.3.1 Esempio: separare stampa, calcolo e dettagli . . . 46
5.3.2 Applicare l'estrazione a Pagamenti . . . 47
5.4 Sostituisci Temp con Query . . . 48
5.5 Dividi Variabile Temp . . . 48
5.5.1 Due fasi del moto e due accelerazioni . . . 49
5.6 Sposta Metodo . . . 50
5.7 Estrai Classe . . . 50
5.8 Oggetto Parametro e sostituzione dei condizionali . . . 50

6 Java funzionale, lambda, collection e stream **52**
6.1 Dallo stile imperativo allo stile dichiarativo . . . 52
6.2 Lambda, interfacce funzionali e funzioni di ordine superiore . . . 52
6.2.1 Lo stesso comportamento in tre forme . . . 54
6.2.2 Riferimenti a metodi e a costruttori . . . 54
6.3 Collection: strutture dati, costi e origine dello stream . . . 55
6.4 Pipeline: sorgente, passaggi lazy e operazione terminale . . . 56

6.5 Filter e map: selezione e trasformazione . . . 56
6.5.1 Dati completi per trasformazioni e ricerche . . . 57
6.6 Collect e toList: materializzare il risultato . . . 57
6.7 Reduce: accumulazione, identità e associatività . . . 58
6.7.1 Esempi di riduzione . . . 58
6.7.2 Esempio: somma delle età . . . 58
6.8 Comparator, max e sorted: confronto e ordine . . . 59
6.8.1 Esempio: selezione e ordinamento delle persone . . . 60
6.9 Distinct, findFirst, findAny e limit . . . 60
6.9.1 Esempio: ruoli distinti . . . 60
6.10 Operazioni stateless e stateful . . . 60
6.11 Generare sequenze con iterate e generate . . . 61
6.12 Debug della pipeline con peek . . . 61
6.13 Stream primitivi e conversioni . . . 62
6.13.1 Esempio: verificare l'ordinamento di una lista . . . 62
6.13.2 Esempio: terne contigue che formano triangoli . . . 62
6.14 Esercizio svolto: buste paga e raggruppamento per ruolo . . . 63
6.15 Stream paralleli: condizioni di correttezza e prestazioni . . . 64
6.15.1 Leggere un confronto sperimentale . . . 64

7 Design pattern: principi e pattern creazionali 67
7.1 Design pattern: problemi ricorrenti, ruoli e conseguenze . . . 67
7.2 Quale aspetto della costruzione deve variare? . . . 68
7.3 Singleton: unicità e accesso controllato . . . 68
7.3.1 Logs lazy, Fib eager e variante Multiton . . . 69
7.4 Factory Method: delegare la scelta del prodotto . . . 71
7.4.1 Creator con contatore e sospensione degli studenti . . . 73
7.4.2 Posti del teatro: client, creatore e prodotto . . . 74
7.5 Object Pool: acquisizione, rilascio e riuso . . . 75
7.6 Dependency Injection: costruzione e wiring . . . 76
7.7 Abstract Factory: prodotti coerenti di una famiglia . . . 77
7.7.1 Esempio: famiglie di icone e testi . . . 78
7.8 Builder: costruire passo per passo un oggetto complesso . . . 80
7.8.1 Tre modi di configurare Biglietto . . . 80
7.9 Prototype: creazione mediante copia . . . 81
7.9.1 Esempio: agenda di impegni e quaderno di note . . . 83

8 Design pattern strutturali 84
8.1 Comporre strutture e localizzare le dipendenze . . . 84
8.2 Adapter: tradurre un contratto incompatibile . . . 84
8.2.1 Delega nella sequenza delle chiamate . . . 87
8.2.2 Esempio Label e LabelC . . . 87
8.3 Facade: accesso di alto livello a un sottosistema . . . 88
8.3.1 Esempi Translator e Registro . . . 89
8.4 Composite: elementi semplici e composizioni ricorsive . . . 90
8.4.1 Esempi: file, cartelle e figure . . . 92
8.5 Decorator: responsabilità aggiunte a un oggetto . . . 93
8.5.1 Esempio: bordo e catena di decorazioni . . . 94
8.6 Bridge: due gerarchie che variano indipendentemente . . . 95
8.6.1 Quattro lati espressi mediante una primitiva comune . . . 98

9 Design pattern comportamentali e MVC **100**
9.1 Algoritmi, comunicazione e distribuzione delle responsabilità . . . 100
9.2 Template Method: scheletro fisso e passi specializzati . . . 101
9.2.1 Esempio: apertura di un documento . . . 102
9.3 State: comportamento dipendente dallo stato corrente . . . 102
9.3.1 Transizioni e concatenamento delle chiamate . . . 104
9.3.2 Esempio: modalità di scrittura di un libro . . . 104
9.4 Strategy: algoritmi alternativi intercambiabili . . . 105
9.5 Observer: notificare i dipendenti senza conoscerne i tipi . . . 106
9.5.1 Protocollo di aggiornamento coerente . . . 108
9.5.2 Rubrica osservabile e salvataggio su file . . . 108
9.5.3 Observable storico e Reactive Streams . . . 109
9.6 MVC: dati, presentazione e gestione degli input . . . 111
9.7 Chain of Responsibility: trovare un gestore lungo la catena . . . 112
9.7.1 Esempio: aiuto contestuale . . . 113
9.8 Command: una richiesta diventa un oggetto . . . 114
9.8.1 Protocollo undo e redo a più livelli . . . 115
9.8.2 Esempio: menu e calendario . . . 115
9.9 Mediator: incapsulare il protocollo cooperativo . . . 115
9.9.1 Coordinare lettura, numeri e nomi . . . 117
9.10 Iterator: attraversare senza esporre la rappresentazione . . . 117
9.11 Null Object: rappresentare un comportamento neutro . . . 117
9.12 Isolare la causa del cambiamento . . . 117

10 Processi di sviluppo, Agile, XP e Scrum **119**
10.1 Attività, prodotti e coerenza del processo . . . 119
10.1.1 Dalle specifiche all'implementazione . . . 119
10.1.2 Verifica, rilascio e cambiamento . . . 119
10.2 Cascata: stabilità e documentazione delle fasi . . . 120
10.3 Sviluppo evolutivo, incrementale e per riuso . . . 120
10.4 Spirale: obiettivi e riduzione dei rischi . . . 121
10.5 Approccio Agile: valori, feedback e cambiamento . . . 122
10.6 Extreme Programming: dodici pratiche che si sostengono . . . 123
10.6.1 Gioco di pianificazione, story card e piccole release . . . 123
10.6.2 Pair programming, metafora, design semplice e CRC . . . 124
10.6.3 Testing, integrazione, proprietà collettiva e ritmo sostenibile . . . 125
10.6.4 Strumenti di collaborazione e condivisione della conoscenza . . . 126
10.7 Scrum: empirismo, lean e responsabilità del team . . . 126
10.7.1 Trasparenza, ispezione e adattamento . . . 126
10.7.2 Product Goal, Sprint Goal, backlog e Definition of Done . . . 127
10.7.3 Eventi e cadenze . . . 127
10.8 Esempio: gestione degli ordini con XP e Scrum . . . 127
10.9 Dominio, specifiche e strumenti di IA . . . 128

11 Evoluzione, manutenzione, metriche e difetti **129**
11.1 Il cambiamento dopo la consegna . . . 129
11.2 Manutenzione ed evoluzione . . . 129
11.2.1 Tipi e costo delle modifiche . . . 129
11.2.2 Costo e modelli di manutenzione . . . 130
11.2.3 Ristrutturazione e ricostruzione . . . 130
11.3 Dinamiche e leggi di Lehman . . . 130

11.4 Metriche del software . . . 131
11.4.1 Scopo e limiti della misurazione . . . 131
11.4.2 Dimensione e commenti . . . 131
11.5 Metriche CK per sistemi a oggetti . . . 132
11.5.1 Metodi ed ereditarietà . . . 132
11.5.2 Accoppiamento, risposta e coesione . . . 132
11.6 Difetti, metriche e interventi . . . 133
11.7 Esempio integrato . . . 133

Glossario dei termini tecnici **134**

Elenco delle tabelle

1.1 Attributi di qualità e domande di controllo. 2

6.1 Interfacce funzionali usate nelle pipeline. 53
6.2 Complessità delle collection: inserimento in coda; * indica posizione già individuata. 55

9.1 Dipendenze da contenere e pattern adatti a isolarle. 118

10.1 Le dodici pratiche di Extreme Programming. 123

Elenco delle figure

2.1 Codice e diagrammi del dispatch fra `Account` e `SavingAccount`. 15

3.1 Dal caso di uso generale Inserimento Ordine agli obiettivi del cliente e al comportamento opzionale. 23
3.2 Ricezione, convalida, registrazione o rifiuto di un ordine e registrazione del log. 24
3.3 Pagamento con alternative e attività concorrenti. 25
3.4 Corsie Cliente, Vendite e Magazzino e scambio di documenti. 26
3.5 Esempio di transizioni con eventi, guardie e azioni. 27
3.6 Stato composto sequenziale e concorrente. 27
3.7 Associazioni fra ordini, clienti, prodotti, fornitori e classi di raccolta. 29
3.8 Messaggi numerati fra ordine, cliente, fornitore, stato e prodotto. 30
3.9 Creazione dello Studente e creazione di un Esame attraverso il metodo nuovoEsame. 31
3.10 Attivazioni, chiamate interne e cicli nella stampa degli esami e nel calcolo della media. 31

6.1 Gerarchia Collection/List/Set e confronto dei costi delle operazioni nelle condizioni specificate. 56
6.2 Ricerca su cinque milioni di elementi e riempimento: quattro core, Java 11.0.2. 65
6.3 Ricerca su cinque milioni di elementi e riempimento: otto core, Java 18.0.1; tempi in millisecondi. 65

7.1 Costruttore privato, riferimento e accesso statici; operazioni sullo stato dell'unica istanza. 69
7.2 Ruoli del Factory Method nel diagramma delle classi. 72
7.3 Creazione delegata al ConcreteCreator e uso del prodotto. 72
7.4 Nel teatro il Main configura il creatore e usa Posto; Biglietto conserva il posto assegnato. 74
7.5 `StCreator` costruisce `StudenteAttivo` e gli fornisce `PianoStudi`. 76
7.6 Due tipi di prodotto e due famiglie nell'Abstract Factory. 78
7.7 Il client chiede la copia tramite l'interfaccia Prototype. 82

8.1 Object Adapter: Target è separato dalla libreria Adaptee. 85
8.2 `Label` crea `LabelServer` solo quando necessario; il frame `opt` esprime il controllo opzionale. 86
8.3 Il client invoca request sul Target; Adapter traduce la chiamata in specificRequest sull'Adaptee. 87
8.4 Prima della facciata ciascun client conosce più classi del sottosistema. 89
8.5 La facciata concentra l'accesso al sottosistema. 89
8.6 Component, Leaf e Composite e molteplicità dei figli. 91
8.7 Il decoratore è un Component e contiene un Component. 94
8.8 La struttura iniziale fonde forma e piattaforma in una classe per ogni combinazione. 96

8.9 Abstraction mantiene un Implementor; le due gerarchie possono essere estese separatamente. . . . 97
8.10 Le gerarchie Forma e Drawing sono collegate; V1Drawing e V2Drawing adattano le piattaforme P1 e P2. . . . 98
8.11 Vertici, primitive di linea e punto e due implementazioni grafiche indipendenti dalle forme. . . . 99

9.1 AbstractClass definisce il template e le primitive; ConcreteClass implementa i passi. . . . 101
9.2 Application e Document mantengono la struttura comune; MyApplication e MyDocument specializzano i passi. . . . 102
9.3 Implementazione del template che invoca primitiveOperation1 e primitiveOperation2 ridefinite nella sottoclasse. . . . 102
9.4 Context delega allo State corrente e i ConcreteState forniscono i comportamenti. 103
9.5 Le dipendenze dirette da ciascun osservatore vengono sostituite dal contratto Observer. . . . 107
9.6 Il Subject conserva gli Observer e il ConcreteObserver legge lo stato di interesse. 107
9.7 Model come Subject e View e Controller come osservatori. . . . 111
9.8 Il riferimento al successore collega i gestori senza una lista globale nel singolo handler. . . . 113
9.9 Client configura comando e ricevente; Invoker conosce soltanto Command. . . . 114
9.10 La finestra di ricerca e le dipendenze dirette fra i controlli prima del mediatore. 116
9.11 La finestra di ricerca centralizza le interazioni dei controlli in un mediatore. . . . 116

10.1 Cicli guidati dal rischio: prototipi, verifiche, piani e sviluppo progressivo. . . . 122

Capitolo 1

Fondamenti di ingegneria del software e qualità

1.1 Ingegneria del software e prodotto software

L'ingegneria del software applica metodi, strumenti e pratiche organizzate alla produzione, all'esercizio e all'evoluzione del software. Il suo obiettivo non è soltanto ottenere un programma che produca un risultato oggi, ma un prodotto che continui a soddisfare bisogni espliciti anche quando cambiano requisiti, persone, piattaforme e dati. Il software è infatti immateriale e modificabile, ma questa flessibilità non rende gratuiti i cambiamenti: una modifica può introdurre difetti, contraddire una decisione precedente o rendere il codice meno comprensibile.

Un *prodotto software* comprende il programma eseguibile, il codice sorgente, i dati e gli artefatti necessari per comprenderlo, usarlo, verificarlo e modificarlo. Un *artefatto* è il risultato di un'attività: sono artefatti una specifica dei requisiti, un diagramma UML, il codice, una configurazione di build, un piano di test e una guida utente. La qualità dipende anche dalla coerenza fra tali elementi: un test non può verificare correttamente un requisito ambiguo, e un'implementazione non è facilmente manutenibile se il suo modello è ignoto o superato.

Nel ciclo di vita si riconoscono attività ricorrenti, non necessariamente svolte una sola volta né in un unico ordine:

1. comprendere bisogni, vincoli e criteri di accettazione;
2. definire e convalidare requisiti verificabili;
3. progettare architettura, responsabilità, interfacce, dati e algoritmi;
4. implementare, integrare e controllare il prodotto;
5. consegnare, usare, correggere ed evolvere il software.

Un processo di sviluppo decide come coordinare queste attività, chi produce gli artefatti e quando si raccolgono feedback. Requisiti, progetto, codice e test restano collegati durante l'intero ciclo di vita: cambiare un requisito può richiedere di modificare sia il programma sia i controlli usati per verificarlo.

1.2 Correttezza, requisiti e attributi di qualità

Un *requisito* descrive una proprietà o un servizio richiesto al sistema. I requisiti funzionali indicano ciò che il sistema deve fare; quelli non funzionali impongono vincoli o livelli di qualità, quali prestazioni, affidabilità, sicurezza e manutenibilità. Una formulazione come "il sistema deve essere veloce" non è verificabile finché non precisa contesto, carico e misura; per esempio,

un requisito può indicare il tempo massimo di risposta per una certa operazione e per un certo numero di utenti.

La *correttezza* riguarda la conformità del comportamento a una specifica: per gli input ammessi, il sistema deve produrre risultati ed effetti previsti. La correttezza è necessaria ma non esaurisce la qualità. Un programma può calcolare correttamente un risultato e tuttavia essere difficile da usare, troppo lento, vulnerabile o quasi impossibile da modificare. La *qualità del software* è quindi l'insieme delle proprietà che rendono il prodotto idoneo al suo uso e alla sua evoluzione; va valutata rispetto ai requisiti, al contesto e ai compromessi del progetto.

Tabella 1.1: Attributi di qualità e domande di controllo.

Attributo	Domanda di controllo
Correttezza	Il comportamento osservato soddisfa la specifica?
Affidabilità	Il servizio resta disponibile e produce risultati corretti nelle condizioni previste?
Efficienza	Tempi, memoria e altre risorse rispettano i vincoli dichiarati?
Usabilità	Gli utenti riescono a comprendere e usare il sistema per i propri obiettivi?
Manutenibilità	È possibile comprendere, correggere, testare e modificare il sistema con costo controllato?
Portabilità	Il sistema può essere trasferito o adattato all'ambiente richiesto?

Gli attributi non sono indipendenti: aggiungere controlli può aumentare affidabilità e costo di esecuzione; una generalizzazione può facilitare estensioni future ma rendere il progetto iniziale più complesso. Perciò non esiste una qualità assoluta separata dallo scopo del sistema. I requisiti non funzionali rendono espliciti i compromessi che devono essere accettati.

1.2.1 Dependability e compromessi fra manutenibilità ed efficienza

La *dependability* è la capacità di offrire un servizio su cui si possa riporre una fiducia giustificata. Comprende aspetti distinti: *security*, protezione dei dati e degli accessi; *safety*, assenza di pericoli inaccettabili per persone e cose; e *reliability*, probabilità di funzionare senza guasti per un intervallo nelle condizioni previste. Un sistema può proteggere bene gli accessi ma avere guasti frequenti, oppure funzionare regolarmente senza essere sicuro rispetto agli attacchi: i termini non sono intercambiabili.

La manutenibilità ha un ruolo centrale nel ciclo di vita: un prodotto di successo può sopravvivere all'hardware iniziale e ricevere sempre più richieste di estensione. L'efficienza riguarda soprattutto tempo di CPU, memoria e altre risorse. Un'ottimizzazione che rende più difficile leggere e modificare il codice deve essere motivata da un vincolo concreto; nell'High Performance Computing (HPC), per esempio, il peso delle prestazioni può giustificare compromessi differenti.

1.3 Responsabilità, coesione e accoppiamento

La struttura interna influenza direttamente la manutenibilità. Una *responsabilità* è un compito coerente assegnato a un elemento del sistema. Nella progettazione a oggetti una *classe* descrive i dati e le operazioni dei propri oggetti: attribuirle responsabilità significa decidere quali informazioni conserva e quali compiti svolge. Il Principio di Singola Responsabilità (SRP) suggerisce che un elemento abbia un motivo principale per cambiare: se una classe calcola importi, gestisce la persistenza e produce schermate, modifiche indipendenti tendono a concentrarsi nello stesso punto.

La *coesione* esprime quanto dati e operazioni di un componente siano rivolti allo stesso scopo; una coesione elevata rende più semplice attribuire e comprendere la responsabilità. L'*accoppiamento* esprime quanto un componente dipenda da altri componenti. Un accoppiamento non è sempre un errore—gli oggetti devono collaborare—ma dipendenze inutili propagano i cambiamenti. L'*incapsulamento* limita la conoscenza della rappresentazione interna, mentre un'*interfaccia* espone le operazioni con cui collaborare. Il contratto di tali operazioni stabilisce che cosa un oggetto offre e a quali condizioni: chi lo usa può dipendere dal servizio senza conoscere ogni dettaglio della sua realizzazione.

Lo *spaghetti code* è una descrizione informale di codice con flusso e dipendenze difficili da seguire, spesso ricco di duplicazioni, condizioni annidate e responsabilità mescolate. Non è una metrica né una diagnosi: è un segnale per esaminare struttura, test e requisiti. Il *debito tecnico* indica il costo futuro generato da una soluzione rapida ma poco curata. Non coincide con ogni scelta pragmatica: diventa debito quando la scorciatoia non viene resa esplicita e ostacola sistematicamente le modifiche successive.

1.3.1 KISS, DRY, YAGNI e Law of Demeter

KISS: Keep It Simple, Stupid. Scegliere una soluzione semplice da comprendere e gestire. La complessità non è una prova di qualità; un'astrazione è utile quando semplifica un problema reale.

DRY: Don't Repeat Yourself. Evitare di duplicare la stessa conoscenza. Se una regola è copiata in più metodi, una correzione deve essere ripetuta ovunque e rischia di lasciare versioni incoerenti. Estrarre un metodo può localizzarla.

YAGNI: You Ain't Gonna Need It. Implementare ciò che serve ai requisiti esistenti. Preparare varianti speculative aumenta codice, test e dipendenze senza un beneficio già richiesto.

LoD: Law of Demeter. Limitare la conoscenza dei dettagli degli oggetti lontani: comunicare soprattutto con i collaboratori diretti. Una catena che attraversa molti oggetti per raggiungere e modificare un dato espone la struttura interna e favorisce cambiamenti a cascata.

Un ordine di grandezza di tre–cinque dipendenze può essere usato come euristica, cioè come soglia orientativa per riesaminare l'accoppiamento, ma la LoD non stabilisce un limite numerico universale. Occorre distinguere una classe che conosce troppe altre classi da un'astrazione stabile usata da molti client, cioè da molti utilizzatori. Anche incroci e cicli in un diagramma sono segnali da interpretare in base alle responsabilità effettive.

1.3.2 Tell, Don't Ask e separazione command–query

Tell, Don't Ask suggerisce di assegnare all'oggetto che possiede i dati l'operazione coerente con quei dati, evitando che il client estragga tutto lo stato e ne ricostruisca all'esterno la logica. La *Command–Query Separation* (CQS) distingue invece due tipi di operazione:

- una *query* restituisce informazioni e non modifica lo stato osservabile;
- un *command* modifica lo stato e, nella forma didattica rigorosa, non restituisce un risultato applicativo.

Per esempio, `getSomma()` osserva un risultato e `inserisci(...)` aggiorna un catalogo. La separazione rende espliciti gli effetti e facilita test e riuso. Una richiesta può anche essere rappresentata mediante un oggetto, secondo il pattern Command. Questa scelta aggiunge

un'altra idea: rappresentare una richiesta mediante un oggetto; non coincide con la sola distinzione fra metodi command e query.

I sintomi di cattiva programmazione comprendono metodi lunghi che usano stato globale, classi con nomi di procedure, poche collaborazioni significative e duplicazioni. Un modello a oggetti esprime responsabilità e relazioni, anziché usare le classi come semplici contenitori di funzioni. Ciò non obbliga ogni classe a essere mutabile: oggetti immutabili, servizi senza stato e funzioni pure possono essere appropriati. La valutazione riguarda responsabilità e contratti, non la presenza meccanica di campi modificabili.

1.4 Verifica, validazione e cambiamento controllato

La *verifica* controlla se gli artefatti sono costruiti correttamente rispetto alle loro specifiche e decisioni di progetto; la *validazione* controlla se il prodotto risponde al bisogno dell'utente. Le due attività si sostengono a vicenda: requisiti chiari permettono test significativi, mentre risultati dei test e feedback degli utenti possono rivelare requisiti incompleti o non realistici.

I test eseguono il software e confrontano un risultato osservato con un *oracle*, cioè un risultato o una proprietà attesa. Le verifiche statiche, quali revisione, compilazione e analisi del codice, operano senza esecuzione. Nessuna singola tecnica dimostra ordinariamente l'assenza di tutti i difetti: la fiducia deriva dall'uso coordinato di specifiche, revisione, test a livelli diversi e controllo delle modifiche.

Il miglioramento strutturale deve essere disciplinato. Se un comportamento è corretto ma il codice è difficile da modificare, il *refactoring* interviene sulla struttura senza alterare il comportamento osservabile. I *test di regressione* rieseguono controlli già superati per individuare effetti indesiderati delle modifiche. Se cambia il requisito, invece, occorre prima aggiornare la specifica e i criteri di verifica: non basta conservare il comportamento precedente.

1.4.1 Strumenti di IA e responsabilità del progettista

L'intelligenza artificiale (IA) generativa può produrre codice, ma il risultato richiede progettazione e verifica: può contenere difetti e vulnerabilità, e può rendere necessarie diagnosi e correzioni mediante debugging. Riutilizzare soluzioni note e progettare una soluzione per un problema nuovo richiedono valutazioni diverse. Comprendere requisiti, architettura e contratti aiuta a formulare richieste precise e a valutare l'implementazione ottenuta. Le capacità degli strumenti possono evolvere; la disponibilità di codice generato non dimostra, da sola, la correttezza del sistema complessivo.

1.5 Dai bisogni a codice e test: procedimento operativo

Per affrontare una funzionalità in modo coerente con i principi del capitolo si può seguire questa sequenza:

1. formulare il bisogno e trasformarlo in requisiti funzionali e non funzionali verificabili;
2. identificare gli artefatti coinvolti e le responsabilità principali;
3. progettare contratti e dipendenze, mantenendo coese le responsabilità e limitando l'accoppiamento non necessario;
4. implementare in passi piccoli e controllabili;
5. definire casi di test con input, stato iniziale, oracle e risultato atteso;
6. integrare e raccogliere feedback; se il requisito cambia, aggiornare in modo coerente specifica, modello, codice e test;

7. misurare e riesaminare le parti che mostrano complessità o dipendenze problematiche, quindi applicare manutenzione o refactoring quando necessario.

In un processo iterativo questa sequenza può essere ripetuta per ogni incremento, mantenendo esplicito il rapporto fra bisogno, decisioni di progetto e verifica.

Capitolo 2

Java e programmazione orientata agli oggetti

2.1 Java: compilazione ed esecuzione

La *programmazione orientata agli oggetti*, o *OOP* (*Object-Oriented Programming*), organizza il programma in oggetti che collaborano. Un *oggetto* è un'entità software con uno stato, cioè dati memorizzati, e operazioni che agiscono su tale stato. Una *classe* ne definisce struttura e comportamento: ogni oggetto è un'*istanza* di una classe, mentre una classe può avere zero o più istanze durante l'esecuzione. Java offre un ambiente di compilazione ed esecuzione e costrutti con cui esprimere classi, interfacce, incapsulamento, ereditarietà e polimorfismo.

2.1.1 JDK, JRE, JVM, bytecode e JIT

Il *JDK* (*Java Development Kit*) è l'insieme completo degli strumenti necessari per sviluppare in Java. Include il compilatore `javac`, strumenti di supporto e la *JRE*. La *JRE* (*Java Runtime Environment*) è l'ambiente di esecuzione: comprende le librerie standard e la *JVM*. La *JVM* (*Java Virtual Machine*) è la macchina virtuale che esegue il codice intermedio prodotto dalla compilazione.

Il codice sorgente Java, contenuto in file `.java`, viene compilato da `javac` in *bytecode*, contenuto normalmente in file `.class`. Il bytecode è un formato intermedio: non è codice macchina specifico per una CPU, ma non è nemmeno codice sorgente Java; è pensato per una macchina astratta, la JVM. Durante l'esecuzione, la JVM interpreta o traduce il bytecode nelle istruzioni native dell'architettura. Il *JIT* (*Just In Time*) è il compilatore dinamico interno alla JVM che può trasformare parti di bytecode in codice macchina durante l'esecuzione.

Il flusso essenziale è:

$$\text{sorgente } .java \xrightarrow{\texttt{javac}} \text{bytecode } .class \xrightarrow{\text{JVM/JIT}} \text{istruzioni native} \xrightarrow{\text{CPU}} \text{esecuzione.} \tag{2.1}$$

Nella formula, `javac` indica il compilatore statico, JVM/JIT indica l'ambiente che esegue o compila dinamicamente il bytecode, e CPU indica il processore fisico che esegue le istruzioni native. Le operazioni di input/output (I/O) comprendono interazione con l'utente, file, connessioni di rete e socket, e usano librerie e risorse esterne durante l'esecuzione.

Ogni opcode del bytecode è codificato in un byte; un'istruzione può comprendere anche operandi. L'indipendenza dalla CPU riguarda il bytecode: la JVM deve essere disponibile per la piattaforma concreta.

2.1.2 Compilazione, avvio e metodo main

La compilazione di una classe avviene con il comando:

```
javac NomeClasse.java
```

che genera il file `NomeClasse.class`. L'esecuzione avviene con:

```
java NomeClasse
```

a condizione che la classe contenga il metodo di avvio:

```
public static void main(String[] args)
```

La parola chiave `static` indica che un campo o un metodo appartiene alla classe e non alla singola istanza. Poiché `main` è statico, al suo interno si possono chiamare direttamente altri metodi statici; per chiamare un metodo non statico occorre invece disporre di un'istanza.

Per convenzione, i nomi di classe iniziano con lettera maiuscola; i nomi di variabile e metodo usano il *camelCase* con iniziale minuscola; i metodi hanno nomi verbali, mentre le classi sono descritte preferibilmente da sostantivi. Questa distinzione è coerente con l'OOP: le classi modellano entità del dominio, mentre i metodi rappresentano azioni o servizi.

Da JDK 11 si può anche avviare direttamente un sorgente con `java NomeClasse.java`: il launcher gestisce la compilazione, senza richiedere di produrre manualmente il file `.class`. Le due modalità hanno argomenti diversi: `java NomeClasse` indica il nome della classe compilata, mentre `java NomeClasse.java` indica un sorgente. L'estensione `.class` non va aggiunta al nome della classe da avviare. JDK, JRE e JVM descrivono ruoli distinti; la loro distribuzione concreta dipende dal JDK installato.

2.2 Classi, oggetti, campi e accesso allo stato

Il codice di una classe ne definisce struttura, comportamento e relazioni prima che siano create le istanze. Gli oggetti, invece, vengono creati durante l'esecuzione e interagiscono per soddisfare i requisiti del problema. La classe di un oggetto, una volta creato, è fissata: il suo tipo a runtime non cambia.

Un *campo* o *attributo* è una variabile appartenente a una classe o a un oggetto. Un *metodo* è un'operazione invocabile su una classe o su un oggetto. Una variabile di istanza non inizializzata assume valori di default: `0` per i tipi numerici, `false` per `boolean`, `null` per i riferimenti. Una variabile locale, cioè dichiarata dentro un metodo, deve invece essere inizializzata prima dell'uso.

L'*incapsulamento* consiste nel nascondere i dettagli interni di un oggetto e nel permettere l'accesso allo stato attraverso metodi appropriati. La visibilità controlla quali parti del programma possono accedere a campi e metodi:

`private` visibile solo all'interno della stessa classe;

`public` visibile a tutti i client che possono accedere alla classe;

`protected` visibile alla classe, alle classi dello stesso package e, con le regole di accesso previste da Java, alle sottoclassi.

Un oggetto ben progettato espone operazioni coerenti con la propria responsabilità e protegge lo stato da modifiche incompatibili con il contratto. Il *costruttore* inizializza una nuova istanza; l'espressione `new` richiede la creazione dell'oggetto e ne restituisce un riferimento. Un riferimento consente di raggiungere un oggetto e va distinto dai dati che l'oggetto contiene.

Senza modificatore si ha accesso *package-private*, limitato alle classi dello stesso package. `protected` comprende sia l'accesso nel package sia quello previsto per le sottoclassi; non significa soltanto "visibile alle sottoclassi".

2.2.1 Un programma completo: membri statici e membri di istanza

`String` rappresenta testo; `LocalDate`, importata da `java.time`, una data senza ora. `LocalDate.now()` legge la data corrente; `System.out` è il flusso di uscita standard e `println` stampa aggiungendo un fine riga.

```
import java.time.LocalDate;

public class HelloWorld {
    private static final String msg = "Lezione di Java";
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println(msg);
        System.out.println(LocalDate.now());
    }
}
```

Salvando il testo in `HelloWorld.java`, `javac HelloWorld.java` lo compila e `java HelloWorld` lo avvia. Le tre righe sono `Hello World`, `Lezione di Java` e la data dell'esecuzione: per esempio `2025-03-03` se eseguito quel giorno. Il metodo statico usa direttamente il campo statico `msg`; per conservare una data distinta per ciascun oggetto si può invece scrivere:

```
import java.time.LocalDate;

public class HelloWorld {
    private final LocalDate d = LocalDate.now();
    public void printDate() { System.out.println(d); }
    public static void main(String[] args) {
        HelloWorld world = new HelloWorld();
        System.out.println("Hello World");
        System.out.println("Lezione di Java");
        world.printDate();
    }
}
```

Le due classi sono alternative da compilare separatamente. Nella seconda il diagramma di classe contiene il campo privato `d: LocalDate`, l'operazione pubblica `printDate(): void` e `main` statico. `main` crea l'istanza `world`; `printDate` legge la data conservata nell'istanza, che non viene ricalcolata a ogni chiamata.

Le parole chiave precisano il contratto: `class` dichiara una classe; `new` crea un oggetto e ne invoca il costruttore; `import` permette di usare un nome di tipo senza scriverne ogni volta il package; `public` rende un membro accessibile ai client, mentre `private` ne limita l'accesso alla classe. `static` associa il membro alla classe; `void` indica che il metodo non restituisce un valore. `return espressione;` restituisce un valore e conclude il metodo; `return;` conclude anticipatamente un metodo `void`.

`final` ha tre usi da distinguere. Una variabile o un campo finale può essere assegnato una sola volta; una classe finale non può essere estesa; un metodo finale non può essere ridefinito. Un riferimento finale non rende immutabile l'oggetto: una `final List<String>` può ancora ricevere elementi, ma la variabile non può essere riassegnata a un'altra lista.

2.3 Package, collection, tipi generici e iterazione

Un *package* è uno spazio di nomi che organizza le classi Java. Le classi del package `java.lang`, come `String`, `System` e `Math`, sono importate automaticamente. Per usare classi di altri

package occorre un'istruzione `import`; ad esempio `import java.time.*` rende disponibili le classi del package indicato. L'import rende un nome disponibile al compilatore e non copia nel bytecode tutte le classi del package; inoltre, l'asterisco non importa i sottopackage.

Le *collection* sono interfacce e classi della JRE, nel package `java.util`, per gestire gruppi di oggetti. Le principali interfacce sono:

`List` sequenza ordinata, ammette duplicati e accesso per indice;

`Set` insieme senza duplicati; l'ordine non è garantito in generale;

`Map` struttura a coppie chiave-valore, con chiavi uniche.

`List` e `Set` estendono `Collection`; `Map` non estende `Collection`, perché raggruppa coppie chiave-valore anziché singoli elementi. `Collection` estende `Iterable`; un oggetto `Iterable` sa produrre un *iteratore*, cioè un oggetto che consente di scorrere una sequenza tramite `hasNext()` e `next()`.

Esempi di implementazioni alternative sono `ArrayList`/`LinkedList`, `HashSet`/`TreeSet` e `HashMap`/`TreeMap`. La scelta dell'interfaccia stabilisce le operazioni disponibili, mentre quella della concreta determina rappresentazione, costi e garanzie ulteriori: per esempio TreeSet e TreeMap mantengono un ordinamento, a differenza di HashSet e HashMap che non garantiscono un ordine di iterazione.

Il ciclo *enhanced for* scorre tutti gli elementi senza gestire esplicitamente gli indici:

```
for (float v: valori) totale += v;
```

L'enhanced for è una forma di *zucchero sintattico*: rende più concisa l'iterazione. Per le istanze di `Iterable`, il compilatore usa `iterator()`, `hasNext()` e `next()`; il ciclo è disponibile anche sugli array, che non devono implementare `Iterable`.

2.3.1 Generics, boxing e tipi primitivi

I *generics* parametrizzano un tipo con altri tipi: `List<T>` descrive una lista di elementi `T`, e `List<Float>` una lista di riferimenti a `Float`. Il compilatore può così controllare il tipo degli elementi. I parametri devono essere tipi di riferimento: `List<Integer>` è valido, `List<int>` non lo è. `int` rappresenta un valore primitivo; `Integer` è la classe che lo incapsula in un oggetto. L'*autoboxing* converte il valore primitivo nel corrispondente oggetto, mentre l'*unboxing* effettua la conversione inversa. Il valore numerico e l'oggetto che lo rappresenta restano concetti distinti.

2.3.2 Enhanced for e Iterator

Implementare `Iterable` permette anche a una collezione personalizzata di essere percorsa con l'enhanced for. Il client usa il protocollo dell'iteratore senza conoscere se gli elementi siano conservati in un array, in una lista o in un'altra struttura.

2.4 Interfacce, classi astratte e contratti

2.4.1 Programmare per le interfacce

Un'*interfaccia* Java è un tipo che definisce un contratto: stabilisce che cosa un oggetto sa fare, senza specificare come lo fa. Dichiara i metodi specificando nome, parametri e tipo di ritorno. La *firma* Java identifica il metodo attraverso nome e tipi dei parametri; il tipo di ritorno appartiene alla dichiarazione, ma non distingue da solo due metodi. Non può avere costruttori né attributi di istanza non inizializzati. Se in un'interfaccia si dichiara un attributo, il compilatore lo tratta come costante `public static final`. Sono disponibili anche i metodi

`default`, dotati di corpo: da Java 8 consentono di aggiungere un'operazione a un'interfaccia esistente senza obbligare immediatamente tutte le implementazioni precedenti a definirla. Un'interfaccia con un unico metodo *astratto* è detta funzionale; eventuali metodi default non aumentano il numero di metodi astratti da implementare.

Una *implementazione* è una classe che fornisce il corpo dei metodi richiesti dall'interfaccia. Per esempio, `List` definisce operazioni come `add()`, `get()`, `size()`, `remove()` e `contains()`; `ArrayList` e `LinkedList` sono implementazioni diverse.

Il principio di progettazione è: *programmare per le interfacce, non per le implementazioni*. Ciò significa dichiarare variabili, parametri e tipi di ritorno con il tipo più astratto utile. Per esempio:

```
List<Float> valori = new ArrayList<>();
```

La variabile `valori` è dichiarata come `List<Float>`, mentre l'oggetto creato è un `ArrayList`. Se in futuro si scegliesse `LinkedList`, il codice client che dipende solo da `List` potrebbe rimanere invariato. Questo riduce la propagazione dei cambiamenti e facilita i test, perché un'implementazione reale può essere sostituita da una fittizia quando si vuole isolare una classe.

2.4.2 Classi astratte e differenze rispetto alle interfacce

Una *classe astratta* non può essere istanziata direttamente. Può contenere metodi concreti, cioè con corpo, e metodi `abstract`, cioè senza implementazione. Un metodo astratto è utile perché altri metodi già implementati possono invocarlo, i client possono aspettarsi che l'operazione esista, e le sottoclassi concrete sono obbligate a implementarlo. Se una sottoclasse non implementa tutti i metodi astratti ereditati, deve essere a sua volta dichiarata astratta.

La scelta dipende dalla relazione da esprimere: un'interfaccia definisce capacità condivisibili anche da classi non correlate; una classe astratta si usa quando esiste una relazione autentica fra classi, del tipo "è un", e quando alcune parti del comportamento sono comuni. Inoltre una classe può implementare più interfacce, ma in Java può estendere una sola classe, anche astratta. L'ereditarietà multipla di implementazione può creare il *diamond problem*: se una classe ereditasse da due superclassi con lo stesso metodo, sarebbe ambiguo quale implementazione usare.

2.4.3 Contratto e sostituzione: IAccount e AccountV2

```
interface IAccount {
    void setBalance(float value);
}
class AccountV2 implements IAccount {
    private float balance;
    @Override public void setBalance(float value) { balance = value; }
    public float getBalance() { return balance; }
}
```

Un client che riceve `IAccount acc` può invocare `acc.setBalance(1234f)` senza conoscere la rappresentazione del saldo. Il punto di configurazione può fornire `new AccountV2()` o un'altra implementazione compatibile. Il getter, presente solo nella concreta, non diventa automaticamente parte del contratto `IAccount`. I metodi astratti dell'interfaccia sono pubblici anche quando la parola `public` è omessa.

Un metodo `default` non dispone di campi di istanza propri dell'interfaccia, ma può chiamare operazioni del contratto che modificano lo stato dell'implementazione: non implica quindi, in generale, assenza di effetti.

2.4.4 Stato comune e operazione astratta: Libro

```
abstract class Libro {
    private final String autore;
    protected final java.util.List<Pagina> pagine =
        new java.util.ArrayList<>();
    protected Libro(String autore) { this.autore = autore; }
    public String autore() { return autore; }
    public abstract void inserisci(Pagina p);
}
record Pagina(String testo) { }
```

`Libro` conserva autore e pagine, ma rinvia a una sottoclasse concreta la politica di inserimento. `Pagina` è qui un semplice valore testuale. Non si può eseguire `new Libro(...)`; una sottoclasse deve fornire `inserisci`. Lo stesso criterio distingue un `Animale` astratto, che può implementare `respira()` e lasciare astratto `verso()`, da `Cane` e `Gatto`, che specializzano il verso. Un'interfaccia è invece adatta a una capacità comune senza imporre quella gerarchia di stato e implementazione.

2.5 Ereditarietà, specializzazione e riuso

L'*ereditarietà* permette di definire una nuova classe indicando ciò che ha in più o di diverso rispetto a una classe esistente. La classe esistente è detta *superclasse*; la nuova classe è detta *sottoclasse*. In Java si usa la parola chiave `extends`:

```
public class Studente extends Persona {... }
```

Una sottoclasse:

1. eredita metodi e attributi della superclasse e può usarli come se fossero definiti localmente, nel rispetto della visibilità;
2. aggiunge nuovi attributi e metodi;
3. può ridefinire, cioè fare *override*, dei metodi della superclasse;
4. non può eliminare metodi o attributi della superclasse.

La parola chiave `super` permette a una sottoclasse di richiamare un metodo o un costruttore della superclasse. L'annotazione `@Override` comunica al compilatore che il metodo dovrebbe ridefinire un metodo ereditato: se la signature non corrisponde, il compilatore segnala errore, proteggendo da sbagli di nome o parametri.

Il riuso tramite ereditarietà è vantaggioso perché evita di copiare classi esistenti e modificarle, con proliferazione di codice e lavoro manuale. Tuttavia richiede comprensione della superclasse e può aumentare il legame fra superclasse e sottoclasse; per questo, quando non esiste una relazione "è un", può essere preferibile la composizione, cioè costruire una classe che contiene e usa oggetti di altre classi invece di ereditarne l'implementazione.

Il nome della sottoclasse dovrebbe comunicare sia la somiglianza sia la specializzazione rispetto alla base: i nomi alle radici della gerarchia sono solitamente più generali e concisi, quelli delle specializzazioni più specifici. L'ereditarietà riusa l'implementazione senza copiarla e senza richiedere di modificare la classe già funzionante.

2.5.1 Esempio Persona, Studente ed Esame

Si consideri `Persona`, con campi `nome` e `cognome` e metodo `printAll()`. `Studente` ne è una specializzazione: aggiunge `matricola`, un elenco di esami e operazioni `nuovoEsame()` e

`getMedia()`; usa `super(nome, cognome)` nel costruttore e ridefinisce `printAll()` per mostrare anche i dati aggiuntivi. Ogni `Esame` raggruppa `materia` e `voto`; può essere rappresentato mediante un *record*, una classe dedicata a un insieme di componenti di dati. L'annotazione `@Override` permette al compilatore di verificare la compatibilità della ridefinizione.

```
import java.util.ArrayList;
import java.util.List;

class Persona {
    private final String nome;
    private final String cognome;
    Persona(String nome, String cognome) {
        this.nome = nome;
        this.cognome = cognome;
    }
    public void printAll() {
        System.out.println(nome + " " + cognome);
    }
}
record Esame(String materia, int voto) { }
class Studente extends Persona {
    private final String matricola;
    private final List<Esame> esami = new ArrayList<>();
    Studente(String nome, String cognome, String matricola) {
        super(nome, cognome);
        this.matricola = matricola;
    }
    public void nuovoEsame(String materia, int voto) {
        esami.add(new Esame(materia, voto));
    }
    public float getMedia() {
        if (esami.isEmpty()) return 0;
        float somma = 0;
        for (Esame e : esami) somma += e.voto();
        return somma / esami.size();
    }
    @Override public void printAll() {
        super.printAll();
        System.out.println(matricola);
        for (Esame e : esami)
            System.out.println(e.materia() + " " + e.voto());
        System.out.println("Media: " + getMedia());
    }
}
```

Il costruttore di `Studente` inizializza la parte `Persona` mediante `super`; gli esami sono oggetti distinti aggiunti alla lista. Il tipo record espone `materia()` e `voto()`, non getter con prefisso `get`. La media è zero per lista vuota secondo il contratto scelto; per due esami con voti 8 e 7 è 7,5.

L'esecuzione seguente mostra compatibilità fra tipi e late binding:

```
Studente s = new Studente("Alan", "Rossi", "M12345");
s.nuovoEsame("Italiano", 8);
s.nuovoEsame("Fisica", 7);
s.printAll();
Persona p = s;
p.printAll();
```

La variabile `p` è dichiarata `Persona`, ma contiene un riferimento a un oggetto `Studente`; la chiamata a `printAll()` esegue quindi la versione di `Studente`. Non sarebbe invece ammesso chiamare `p.nuovoEsame(...)` senza cast, perché `nuovoEsame` non appartiene al tipo dichiarato `Persona`.

2.6 Compatibilità dei tipi, cast e uguaglianza

L'ereditarietà definisce una classificazione di tipi. Una sottoclasse è un *sottotipo* compatibile con la superclasse: ogni `Studente` è anche una `Persona`, ma non ogni `Persona` è uno `Studente`. Per questo è lecito assegnare un riferimento a `Studente` a una variabile dichiarata `Persona`:

```
Studente s = new Studente("Alan", "Rossi", "M12345");
Persona p = s;
p.printAll();
```

A tempo di compilazione, il tipo dichiarato della variabile determina quali operazioni possono essere invocate. Su `p` si può chiamare `printAll()` se tale metodo esiste in `Persona`; non si può chiamare `nuovoEsame()`, anche se a runtime `p` punta a uno `Studente`, perché il compilatore conosce `p` come `Persona`.

Il *cast* esplicito forza il compilatore a trattare un riferimento come un sottotipo:

```
Persona p = new Studente("Alan", "Rossi", "M12345");
Studente s = (Studente) p;
```

Il cast è accettato dal compilatore quando è plausibile nella gerarchia, ma a runtime riesce solo se l'oggetto è davvero del tipo richiesto. In caso contrario viene generata `ClassCastException`. Per esempio, un cast da `new Persona(...)` a `Studente` fallirebbe a runtime.

2.6.1 Identità degli oggetti e confronto del contenuto

Per gli oggetti, `==` confronta i riferimenti; `equals()` può confrontare il contenuto secondo il contratto del tipo. Due stringhe con lo stesso contenuto possono essere oggetti diversi: il confronto corretto del testo usa `equals()`, eventualmente `Objects.equals(a,b)` se si vuole gestire `null`. La lunghezza della stringa non determina la semantica di `==`; il fatto che alcuni letterali condividano l'istanza spiega perché esempi semplici possano ingannare.

2.7 Polimorfismo, binding e dispatch

Il *polimorfismo* è la proprietà per cui la stessa invocazione di metodo può produrre comportamenti diversi in base al tipo concreto dell'oggetto che la riceve. Nei sistemi a oggetti possono esistere metodi con stesso nome e stessa signature in classi diverse della gerarchia; se una sottoclasse ridefinisce un metodo, la chiamata viene risolta dinamicamente.

Il *late binding* è il collegamento dinamico: a runtime si guarda il tipo effettivo dell'istanza, non soltanto il tipo dichiarato della variabile. Il *early binding*, al contrario, collegherebbe staticamente la chiamata in base al tipo dichiarato. In Java il dispatch dei metodi di istanza ridefinibili è dinamico.

Il *dispatch* è il meccanismo con cui Java decide quale versione di un metodo chiamare. Per un metodo di istanza ridefinibile, il meccanismo si può descrivere così:

1. quando un metodo viene chiamato su un oggetto, si considera il tipo dell'istanza a runtime;
2. si cerca il metodo nella classe concreta dell'istanza;

3. se non viene trovato, si risale la gerarchia delle superclassi finché il metodo viene trovato;
4. il metodo trovato viene eseguito.

Nell'esempio `Account`/`SavingAccount`, una variabile `Account acc = new SavingAccount()` riceve la chiamata `acc.setBalance(1234)`. Il metodo `setBalance()` viene trovato in `Account`; dentro tale metodo viene chiamato `check()`. Poiché l'oggetto reale è `SavingAccount` e `SavingAccount` ridefinisce `check()`, viene eseguito `SavingAccount.check()`. L'esempio mostra che anche da un metodo della superclasse una chiamata a un metodo ridefinito può essere risolta sulla sottoclasse.

Senza polimorfismo, il chiamante dovrebbe contenere uno `switch` o una sequenza di condizioni per distinguere manualmente le classi degli oggetti e chiamare il metodo adatto. Ciò rende il codice più fragile e difficile da manutenere.

Un client può scegliere l'oggetto con una condizione e invocare poi un unico contratto:

```
Persona p = new Persona("Alan", "Rossi");
Studente s = new Studente("Alan", "Rossi", "M12345");
Persona px;
if (i > 10) px = p;
else px = s;
px.printAll();
```

Il valore di `i` decide il riferimento; il tipo dinamico decide l'override. Un terzo livello `StudenteLavor extends Studente` può ridefinire `nuovoEsame` e `printAll`: la ricerca parte da StudenteLavor, risale a Studente per i metodi non ridefiniti e poi a Persona. Non si assume una specifica regola lavorativa non definita dal contratto.

2.7.1 Overload e override

L'*overload* permette di dichiarare più metodi con lo stesso nome e parametri differenti. Non basta cambiare il tipo di ritorno. La scelta dell'overload avviene in compilazione in base ai tipi disponibili; il dispatch dinamico sceglie poi l'*override*, cioè la ridefinizione del metodo di istanza selezionato. Campi, metodi statici e costruttori non seguono questo meccanismo di override dinamico.

2.7.2 Account e SavingAccount: chiamata dalla superclasse alla sottoclasse

```java
public class Account {
   protected float balance = 0;
   public void setBalance(float amount) {
      System.out.println("in Account set-balance");
      if (check(amount))
         balance += amount;
   }
   public boolean check(float amount) {
      System.out.println("in Account check");
      return balance + amount >= 0;
   }
}

public class SavingAccount extends Account {
   public boolean check(float amount) {
      System.out.println("in Saving-Account check");
      return (balance + amount) >= 1000;
   }
}

public class AccTest {
   public static void main(String[] args) {
      Account acc = new SavingAccount();
      acc.setBalance(1234);
   }
}
```

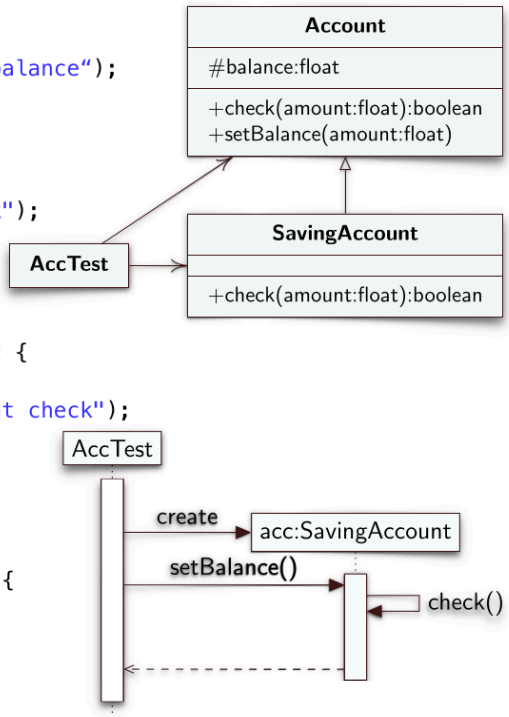


Figura 2.1: Codice e diagrammi del dispatch fra `Account` e `SavingAccount`.

Nell'esempio, `setBalance()` è ereditato da `Account`, mentre `check()` è ridefinito in `SavingAccount`. La chiamata interna a `check()` usa ancora il ricevente reale, cioè `this`: non diventa una chiamata su un altro oggetto della superclasse. È questo il motivo per cui un algoritmo ereditato può usare un passo specializzato, meccanismo che riappare nel Template Method.

2.8 Record e immutabilità dei dati

Un *record* Java è una classe speciale progettata per contenere dati immutabili. Per esempio:

```
public record Esame(String materia, int voto) { }
```

Il record fornisce automaticamente un costruttore che assegna tutti i campi e metodi di accesso con lo stesso nome del campo, per esempio `materia()` e `voto()`, non `getMateria()`. I campi sono `final`, quindi non modificabili dopo l'assegnazione, e non sono presenti setter. I record sono `final`: non possono essere estesi da sottoclassi. Possono implementare interfacce, ma non estendere esplicitamente altre classi; possono inoltre avere campi statici, metodi statici e metodi di istanza aggiuntivi.

Nell'esempio `Studente`, `Esame` è usato come contenitore immutabile di materia e voto. La media dei voti può essere descritta dalla formula:

$$\bar{v} = \frac{1}{n} \sum_{i=1}^{n} v_i, \tag{2.2}$$

dove n è il numero di esami presenti nella lista, v_i è il voto dell'i-esimo esame e $\bar{v}$ è la media. Se si sceglie di rappresentare il caso senza esami con il valore zero, `getMedia()` restituisce `0`; quindi la formula si applica solo per $n > 0$.

I campi finali di un record impediscono di riassegnare i riferimenti, ma non rendono automaticamente immutabili gli oggetti raggiungibili. Se un componente è una lista mutabile, il suo contenuto può ancora cambiare; per un valore davvero protetto occorre scegliere componenti immutabili o copie difensive. Oltre a costruttore e accessori, un record dispone di implementazioni di `equals`, `hashCode` e `toString` coerenti con i componenti.

2.9 Inferenza del tipo delle variabili locali

`var` permette al compilatore di inferire il tipo di una variabile locale dall'inizializzatore; non introduce tipizzazione dinamica. È disponibile da Java 10. Per esempio:

```
var valori = new ArrayList<String>();
// Il tipo statico inferito e' ArrayList<String>.
List<String> contratto = valori;
```

Non si usa `var` per un campo, per dichiarare una variabile senza inizializzarla o con il solo `null`. Inoltre il tipo inferito può essere più concreto dell'interfaccia che si voleva esporre: scegliere `List<String>` resta utile quando rende esplicito il contratto.

2.10 Caso progressivo: leggere e aggregare importi

Un'applicazione deve leggere più file di testo, ciascuno con un numero per riga, eliminare i duplicati e calcolare somma e massimo dei valori distinti. I quattro requisiti vanno controllati separatamente: leggere correttamente un file non dimostra né la deduplicazione né il corretto calcolo degli aggregati.

2.10.1 Dalla lettura monolitica alla separazione dei compiti

La prima versione di `CalcolaImporti` legge e somma nello stesso ciclo:

```
import java.io.*;
import java.util.*;

class CalcolaImporti {
    private final List<String> importi = new ArrayList<>();
    public float calcola(String cartella, String nome) {
        float totale = 0;
        try (BufferedReader f = new BufferedReader(
                new FileReader(new File(cartella, nome)))) {
            String riga;
            while ((riga = f.readLine()) != null) {
                try {
                    float valore = Float.parseFloat(riga);
                    importi.add(riga);
                    totale += valore;
                } catch (NumberFormatException e) {
                    System.err.println("Numero non valido: " + riga);
                }
            }
        } catch (IOException e) {
            System.err.println("Errore di I/O: " + e.getMessage());
        }
        return totale;
    }
}
```

`float` è un tipo numerico primitivo in virgola mobile. `while` ripete il corpo finché la condizione è vera; `readLine()` legge una riga senza il terminatore e restituisce `null` a fine file. L'assegnazione nella condizione conserva la riga prima del controllo. `Float.parseFloat` converte il testo e può sollevare `NumberFormatException`; un problema di apertura o lettura è invece un'`IOException`. I blocchi `try/catch` distinguono i due eventi e il *try-with-resources* chiude il lettore anche in caso di errore. `if` esegue un blocco soltanto quando la propria condizione è vera.

Questa versione realizza lettura e somma, ma conta anche i duplicati e non calcola il massimo. Aggiungere `if (valore > massimo) massimo = valore;` realizza il massimo solo con una corretta inizializzazione: partire da zero è adeguato per importi non negativi, mentre sbaglia un insieme composto da valori negativi. La versione successiva aggiunge prima dell'inserimento `if (!importi.contains(riga))`: il ciclo aggiorna somma e massimo soltanto per righe nuove. Occorre però precisare che `"1"` e `"1.0"` sono testi diversi pur rappresentando lo stesso numero.

`List.add` aggiunge un elemento; `contains` verifica l'uguaglianza degli elementi secondo `equals`, non la loro sola identità. Per una lista di stringhe deduplica quindi il testo; per una lista di `Float` usa il contratto di uguaglianza del tipo numerico. La rappresentazione scelta influenza il significato di "duplicato".

Un'ulteriore versione legge prima tutte le righe distinte e poi svolge due cicli:

```
float totale = 0;
for (String s : importi) totale += Float.parseFloat(s);
float massimo = Float.parseFloat(importi.get(0));
for (String s : importi) {
    float valore = Float.parseFloat(s);
    if (valore > massimo) massimo = valore;
}
```

L'enhanced for assegna a `s` ogni elemento della lista. Il massimo parte dal primo elemento e gestisce anche numeri negativi, ma richiede una lista non vuota. I due cicli separano i calcoli, pur ripetendo le conversioni. La lunghezza può crescere anche se l'intento diventa più leggibile: le versioni con un unico metodo hanno rispettivamente 18 e 23 linee nella convenzione di conteggio a punto e virgola.

Mescolare tutto in un metodo rende difficile osservare ciò che è stato letto, testare separatamente somma e massimo, cambiare la regola sui duplicati o riusare il calcolo senza file. Inesperienza e assenza di progettazione favoriscono questa struttura, soprattutto quando si aggiungono requisiti senza riesaminare responsabilità e dipendenze. Una classe chiamata come una procedura non diventa un buon modello a oggetti per il solo fatto di essere una classe.

2.10.2 Pagamenti con comandi di calcolo e query di lettura

La separazione produce `Pagamenti`: `leggiFile` invoca `inserisci`; `converti` trasforma le righe in numeri; `calcolaSomma` e `calcolaMassimo` aggiornano campi derivati; `getSomma` e `getMassimo` li osservano. I campi sono `importiLetti: List<String>`, `valori: List<Float>`, `totale` e `massimo`.

```
class PagamentiPreparati {
    private final List<String> importiLetti = new ArrayList<>();
    private final List<Float> valori = new ArrayList<>();
    private float totale, massimo;
    public void inserisci(String s) {
        if (!importiLetti.contains(s)) importiLetti.add(s);
    }
    public void leggiFile(String c, String n) throws IOException {
        try (BufferedReader f = new BufferedReader(
                new FileReader(new File(c, n)))) {
```

```
            String riga;
            while ((riga = f.readLine()) != null) inserisci(riga);
        }
    }
    public void converti() {
        valori.clear();
        for (String s : importiLetti) {
            try { valori.add(Float.parseFloat(s)); }
            catch (NumberFormatException e) {
                System.err.println("Numero non valido: " + s);
            }
        }
    }
    public void calcolaSomma() {
        totale = 0;
        for (float v : valori) totale += v;
    }
    public void calcolaMassimo() {
        massimo = 0;
        if (valori.isEmpty()) return;
        massimo = valori.get(0);
        for (float v : valori) if (v > massimo) massimo = v;
    }
    public float getSomma() { return totale; }
    public float getMassimo() { return massimo; }
    public void svuota() {
        importiLetti.clear(); valori.clear();
        totale = 0; massimo = 0;
    }
}
```

Gli import sono quelli del listato precedente. Il nome `PagamentiPreparati` distingue questa variante dalla successiva. Svuotare `valori` prima di convertire impedisce che due conversioni consecutive accumulino copie degli stessi dati. Gli errori di I/O sono qui propagati al client, che decide come presentarli; gli errori di conversione sono segnalati scartando la riga. Queste sono politiche distinte e fanno parte del contratto.

La sequenza del client è `svuota`, una o più `leggiFile`, `converti`, i due comandi di calcolo e infine le query. Se si aggiungono dati dopo il calcolo, i risultati memorizzati diventano obsoleti fino alla nuova esecuzione dei comandi. L'ordine è dunque un vincolo osservabile, non un dettaglio interno.

2.10.3 Pagamenti con risultati calcolati su richiesta

Eliminare righe testuali e campi derivati riduce lo stato da mantenere coerente. Il contratto di questa variante deduplica valori `Float`, calcola gli aggregati quando richiesti e restituisce convenzionalmente zero sulla lista vuota:

```
class Pagamenti {
    private final List<Float> importi = new ArrayList<>();
    public void leggiFile(String c, String n) throws IOException {
        try (BufferedReader f = new BufferedReader(
                new FileReader(new File(c, n)))) {
            String riga;
            while ((riga = f.readLine()) != null)
                inserisci(Float.parseFloat(riga));
        }
```

```
    }
    public void inserisci(float x) {
        if (!importi.contains(x)) importi.add(x);
    }
    public float calcolaSomma() {
        float risultato = 0;
        for (float v : importi) risultato += v;
        return risultato;
    }
    public float calcolaMassimo() {
        if (importi.isEmpty()) return 0;
        float risultato = importi.get(0);
        for (float v : importi)
            if (v > risultato) risultato = v;
        return risultato;
    }
    public void svuota() { importi.clear(); }
}
```

`calcolaSomma` e `calcolaMassimo` sono ora query: restituiscono il risultato senza aggiornare lo stato. La CQS continua a essere rispettata, anche senza una coppia separata comando/getter. `IOException` e `NumberFormatException` raggiungono il chiamante; una lettura fallita dopo alcune righe può aver già inserito una parte dei dati, perché l'operazione non è transazionale. Inizializzare il massimo al primo elemento conserva il significato anche per importi negativi; l'alternativa con accumulatore iniziale zero richiede esplicitamente un dominio non negativo. Le due soluzioni coincidono solo in quel dominio.

2.10.4 Una raccolta comune o una per file?

```
Pagamenti insieme = new Pagamenti();
insieme.leggiFile("./", "Importi.csv");
System.out.println(insieme.calcolaSomma());
insieme.leggiFile("./", "AltriImporti.csv");
System.out.println(insieme.calcolaSomma());

Pagamenti primo = new Pagamenti();
Pagamenti secondo = new Pagamenti();
primo.leggiFile("./", "Importi.csv");
secondo.leggiFile("./", "AltriImporti.csv");
System.out.println(primo.calcolaSomma());
System.out.println(secondo.calcolaSomma());
```

Nel primo caso la seconda lettura aggiunge dati alla raccolta esistente e deduplica anche fra file; nel secondo le raccolte sono indipendenti. Il client deve gestire o dichiarare `IOException`. Per ottenere risultati separati riusando la stessa istanza occorre `svuota()` fra i file. La scelta del numero di oggetti deriva dal requisito: una sola classe non implica una sola istanza e non giustifica da sola Singleton.

Per controllare la comprensione del caso, si devono saper spiegare il ciclo enhanced for, la differenza fra `List` e `ArrayList`, il significato di `contains`, quali operazioni sono query e come verificare la correttezza. Le risposte si collegano: l'interfaccia permette di cambiare contenitore, l'uguaglianza determina i duplicati e le query offrono risultati osservabili da confrontare con un oracle.

2.11 Progettazione a oggetti: procedimento operativo

Per attribuire responsabilità e definire collaborazioni si può procedere in questo modo:

1. individuare le entità del dominio e dare loro nomi nominali, per esempio `Persona`, `Studente`, `Esame`;
2. attribuire a ciascuna classe uno stato coerente e responsabilità limitate;
3. nascondere i dettagli interni con visibilità adeguata, in particolare usando campi `private` quando lo stato non deve essere manipolato direttamente dai client;
4. esporre operazioni pubbliche significative, evitando classi usate solo come contenitori di procedure;
5. usare interfacce quando si vuole dipendere da un contratto e non da una classe concreta;
6. usare ereditarietà solo quando la sottoclasse è davvero un caso particolare della superclasse;
7. sfruttare override, late binding e dispatch per lasciare che il comportamento specifico sia scelto dall'oggetto concreto;
8. quando una struttura cresce o accumula responsabilità, migliorarne la struttura con piccoli refactoring protetti dai test.

La scelta di una struttura è motivata dalle responsabilità e dai cambiamenti attesi: un contratto stabile e un numero contenuto di dipendenze rendono più semplice verificare e modificare il programma.

Capitolo 3

Requisiti, UML e analisi orientata agli oggetti

3.1 Requisiti, specifiche e feature

La progettazione parte dalla comprensione dei bisogni del cliente. Un *requisito* è ciascuna caratteristica che il software deve avere; una *specifica* è la descrizione rigorosa di tali caratteristiche; una *feature* è un insieme di requisiti correlati che permette all'utente di soddisfare un obiettivo. I requisiti tendono a essere granulari, cioè numerosi e piccoli, e possono cambiare nel tempo: per questo l'analisi deve rendere espliciti servizi richiesti, vincoli e criteri di verifica.

UML, acronimo di *Unified Modeling Language*, è una notazione grafica standardizzata per rappresentare visivamente un sistema software. UML si usa sia per comunicare con il cliente, sia per ragionare sulla struttura del programma e sulle interazioni tra oggetti. I diagrammi trattati sono: casi d'uso, attività, stati, collaborazione, classi e sequenza.

L'*analisi dei requisiti* è la fase che porta a definire le specifiche e stabilisce i servizi richiesti e i vincoli del software. L'ingegneria dei requisiti comprende:

1. **studio di fattibilità**, nel quale si valuta se l'azienda possiede risorse e competenze per realizzare il sistema;
2. **analisi dei requisiti**, nella quale si chiariscono le richieste iniziali del cliente;
3. **specifica dei requisiti**, nella quale i requisiti vengono descritti in modo rigoroso;
4. **convalida dei requisiti**, nella quale si verifica che i requisiti siano completi, consistenti, realizzabili e verificabili.

Un requisito è *completo* se non lascia lacune rilevanti, *consistente* se non contraddice altri requisiti, *realizzabile* se può essere soddisfatto con tecnologia, tempo e budget disponibili, e *verificabile* se è possibile controllare se sia stato soddisfatto.

I requisiti si distinguono in due classi principali:

requisiti funzionali indicano che cosa il sistema deve fare, cioè le funzionalità offerte;

requisiti non funzionali indicano come il sistema deve operare o quali vincoli deve rispettare, per esempio affidabilità, efficienza, prestazioni e manutenibilità.

Requisiti e qualità sono collegati: soddisfare e verificare i requisiti permette di valutare la qualità del prodotto, mentre requisiti scritti male rendono difficile o impossibile verificare la correttezza.

La criticità influenza il rigore richiesto alle specifiche: si pensi a sistemi per somministrare farmaci, guida autonoma e gestione di ingenti transazioni. Più gravi sono gli effetti di un errore, più occorre rendere espliciti vincoli, casi limite e criteri di verifica. Lo studio di fattibilità

valuta anche tempo, budget e interesse a realizzare il progetto; durata e condizioni economiche non sono fissate dal metodo.

3.2 Dai requisiti alla struttura del sistema

La progettazione stabilisce la struttura software che realizza le specifiche. Le attività comprendono: suddivisione dei requisiti, identificazione dei sottosistemi o architettura software, specifica delle responsabilità dei sottosistemi, progettazione di interfacce, componenti, strutture dati e algoritmi. Ogni attività produce o integra un documento che descrive un modello: modello degli oggetti, modello di sequenza, modello di transizione degli stati, modello strutturale o modello data-flow.

La suddivisione dei requisiti consiste nel trasformare richieste grandi in problemi più piccoli e gestibili. Questo passaggio deve rispettare coesione e singola responsabilità: attributi e operazioni vanno raggruppati in classi che abbiano uno scopo chiaro, evitando sia classi troppo generiche sia dettagli non necessari.

La suddivisione comprende anche la mappatura dei requisiti ai sottosistemi responsabili. La progettazione architetturale definisce componenti e comunicazioni ad alto livello; quella dettagliata stabilisce interfacce precise, algoritmi e strutture dati interne.

3.3 Casi di uso: attori, confine e obiettivi

Un *diagramma dei casi d'uso* descrive il comportamento del sistema come appare dall'esterno, tramite una sequenza di messaggi fra sistema e attori. Un *attore* è una persona o un sistema esterno che interagisce con il prodotto da sviluppare con un certo ruolo; nei diagrammi è rappresentato come un omino con il nome scritto sotto. Un *caso d'uso* è una funzionalità offerta dal sistema, disegnata come un'ellissi con il nome all'interno; non descrive il comportamento interno né la struttura interna del programma. In un caso d'uso l'attore cerca di ottenere un obiettivo, e lo stesso caso d'uso può avere più *scenari*, cioè percorsi alternativi.

Il rettangolo del sistema delimita il confine: gli attori stanno fuori, i casi d'uso dentro. Una *associazione* fra attore e caso d'uso è una linea che li congiunge. Si distinguono inoltre:

generalizzazione fra attori l'attore più specifico eredita le capacità di quello più generale e ne aggiunge altre;

generalizzazione fra casi d'uso il caso d'uso più specifico eredita da uno più generale e aggiunge caratteristiche;

estensione un caso d'uso aggiunge a un caso base un comportamento incrementale opzionale;

inclusione un caso d'uso incorpora il comportamento di un altro.

La distinzione fra `<<include>>` e `<<extend>>` dipende anche dal verso della freccia tratteggiata: in `<<include>>` la freccia parte dal caso includente e punta al caso incluso; in `<<extend>>` la freccia parte dal caso estendente e punta al caso base.

Il livello di dettaglio deve restare adatto alla comunicazione. Si possono distinguere tre livelli di astrazione: *kite level*, molto astratto e orientato agli obiettivi di business; *sea level*, livello medio e raccomandato perché vicino agli obiettivi dell'utente; *fish level*, troppo dettagliato e quindi potenzialmente enorme.

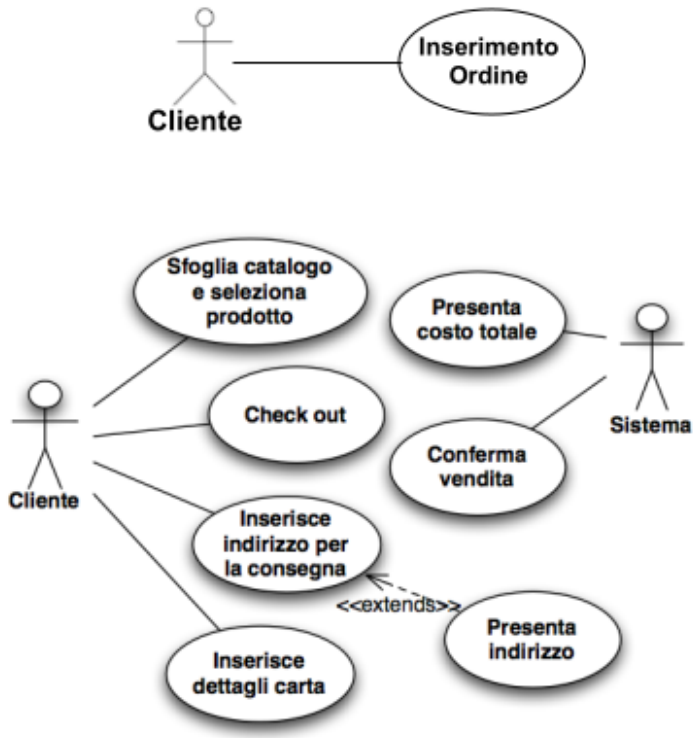


Figura 3.1: Dal caso di uso generale Inserimento Ordine agli obiettivi del cliente e al comportamento opzionale.

3.4 Diagrammi delle attività: decisioni, parallelismo e corsie

Un *diagramma delle attività* rappresenta come si svolge un processo passo per passo. Permette di ragionare sulle operazioni che il sistema dovrà realizzare prima di decidere quali oggetti le svolgeranno. Questa analisi del flusso può guidare la successiva progettazione delle collaborazioni fra oggetti.

La notazione essenziale è: rettangoli arrotondati per i passi di elaborazione, frecce continue per i flussi, rombi per ramificazioni condizionali o merge, cerchio pieno per lo stato iniziale e cerchio pieno circondato da una circonferenza per lo stato finale. Nei diagrammi con flussi paralleli, le barre nere indicano punti di parallelizzazione, quando una freccia entrante si divide in più frecce uscenti, oppure punti di sincronizzazione o *join*, quando più frecce entranti confluiscono in una uscente.

Le *corsie*, o *swimlane*, sono partizioni che indicano il responsabile, attore o modulo, delle attività nella corsia. In questa variante i rettangoli possono rappresentare oggetti o dati, mentre le frecce tratteggiate indicano dati in input o output per un'attività.

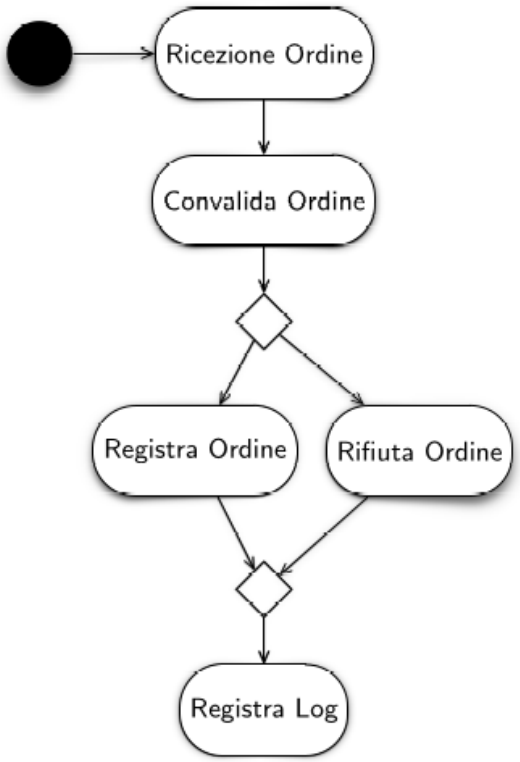


Figura 3.2: Ricezione, convalida, registrazione o rifiuto di un ordine e registrazione del log.

Nel processo di ricezione dell'ordine, il rombo seleziona un'alternativa; il ricongiungimento successivo permette di proseguire al log indipendentemente dal ramo scelto. Un *merge* riunisce alternative, mentre un *join* sincronizza flussi paralleli: non sono la stessa operazione.

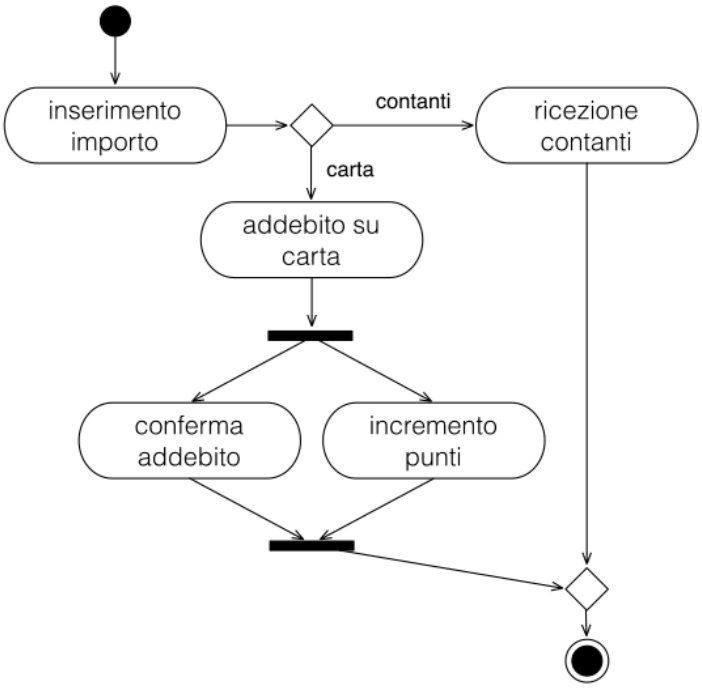


Figura 3.3: Pagamento con alternative e attività concorrenti.

Nel processo di pagamento si separano contanti e carta. Nel ramo carta la barra di fork avvia conferma dell'addebito e incremento dei punti; la barra di join attende i flussi previsti. Una barra può collegare più di due flussi: il numero due è soltanto quello dell'esempio.

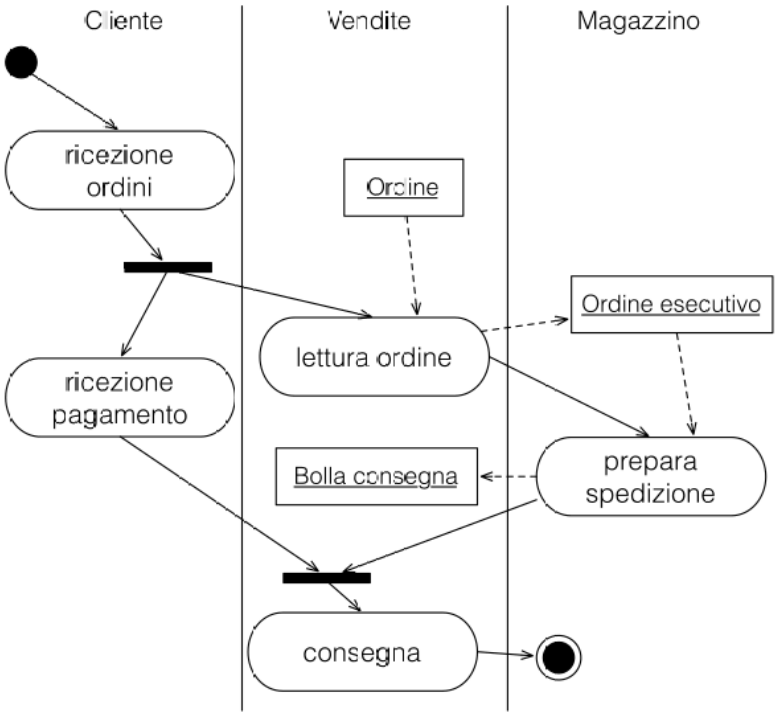


Figura 3.4: Corsie Cliente, Vendite e Magazzino e scambio di documenti.

Le corsie Cliente, Vendite e Magazzino identificano i responsabili delle attività. Ordine e bolla rappresentano invece dati o documenti scambiati fra tali responsabili: distinguerli dalle azioni evita di confondere il contenuto del lavoro con chi lo esegue.

3.5 Diagrammi degli stati: eventi, guardie e azioni

Un *diagramma degli stati* mostra come cambia lo stato di un singolo oggetto nel tempo. Uno *stato* descrive un intervallo della vita di un oggetto, caratterizzato da valori, dall'attesa di eventi o dallo svolgimento di azioni. Una *transizione* permette di lasciare uno stato in risposta a un evento ed è caratterizzata da evento, possibile condizione, azione e stato di arrivo.

L'etichetta di una transizione può essere letta nella forma:

$$\text{evento}\,[\text{condizione}]/\text{azione}. \tag{3.1}$$

Qui *evento* è ciò che può far scattare il passaggio di stato, *condizione* è una guardia booleana opzionale scritta tra parentesi quadre, e *azione* è l'operazione eseguita durante la transizione. Se l'evento accade ma non riguarda una transizione uscente dallo stato corrente, non accade nulla secondo il modello.

Uno *stato composto* è uno stato formato da sottostati sequenziali o concorrenti; uno stato semplice non ha sottostati. Nei sottostati sequenziali solo uno è attivo in un certo momento; negli stati composti concorrenti i sottostati sono paralleli e, se si esce dallo stato composto, si esce da tutti i suoi sottostati. Le indicazioni `entry`, `do` e `exit` indicano rispettivamente operazioni eseguite entrando nello stato, durante la permanenza nello stato e uscendo dallo stato.

Per distinguere evento, condizione e azione si possono confrontare due etichette:

- `ricevi[totale<25]`, dove `ricevi` è l'evento e `totale<25` è la condizione;

- `approvato/debit()`, dove `approvato` è l'evento e `debit()` è l'azione eseguita durante la transizione.

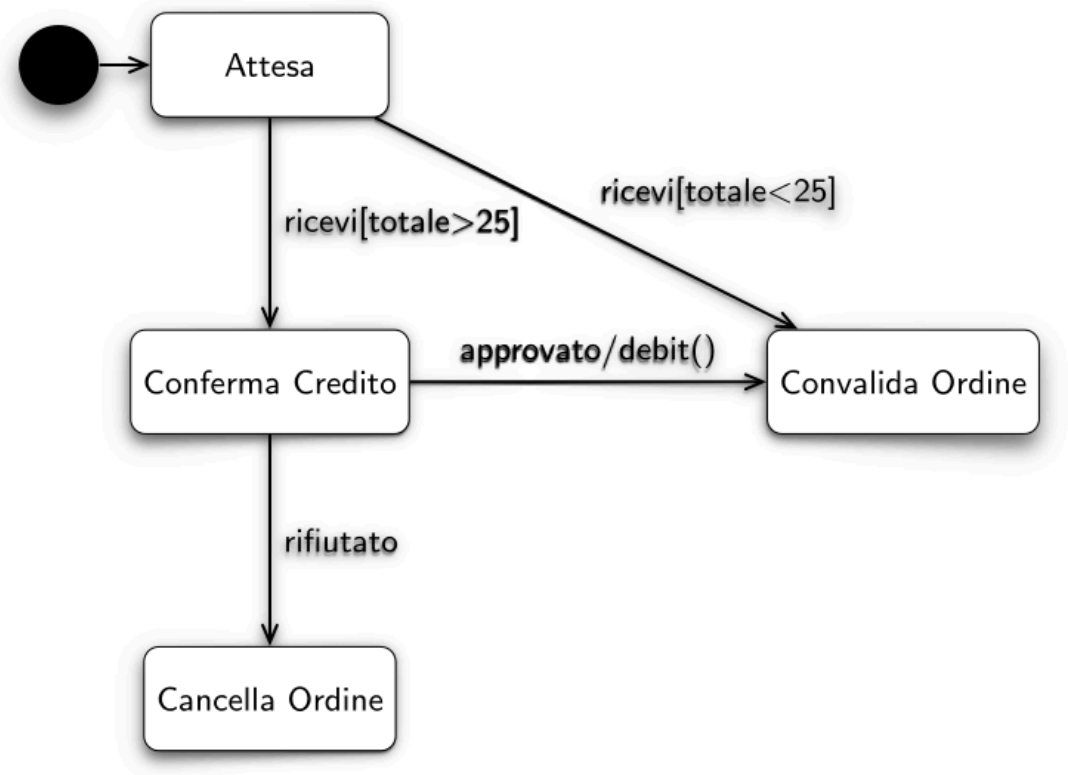


Figura 3.5: Esempio di transizioni con eventi, guardie e azioni.

La guardia va valutata nello stato di origine della transizione pertinente. Il testo dell'esempio presenta `ricevi` con soglie sul totale e un'azione `debit()` associata all'approvazione: evento, condizione e azione hanno ruoli diversi. Le soglie vanno rese complete anche per l'uguaglianza al limite quando si scrive la specifica eseguibile; un diagramma di esempio non sostituisce tale decisione.

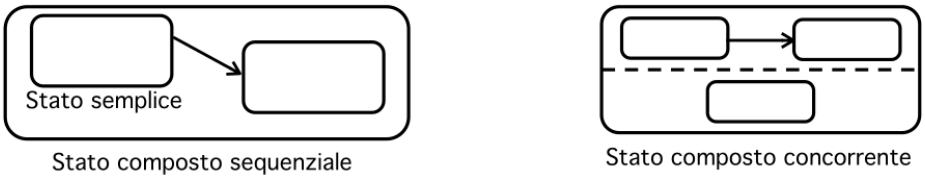


Figura 3.6: Stato composto sequenziale e concorrente.

L'ingresso in uno stato composto può usare un punto iniziale interno oppure una transizione esplicita a un sottostato; l'uscita può attraversare il confine dello stato composto. Le regioni separate dalla linea tratteggiata indicano concorrenza. L'attività svolta durante la permanenza, indicata con `do` o descritta come *doing*, può essere interrotta da una transizione.

Una notazione esemplificativa delle tre attività è:

```
entry / op1()
do    / op2()
exit  / op3()
```

L'ingresso attiva `op1()`, l'uscita attiva `op3()`, mentre `op2()` descrive l'attività durante la permanenza: una transizione può interromperla. I prefissi appartengono al modello di stato, non sono dichiarazioni Java.

3.6 Diagramma delle classi e relazioni strutturali

Un *diagramma delle classi* rappresenta la struttura statica del sistema. Un rettangolo rappresenta una classe ed è suddiviso in tre sezioni: nome, attributi e metodi. Per attributi e operazioni si usa una sintassi del tipo:

```
- nomeAttributo: Tipo
+ nomeMetodo(param: Tipo): TipoRitorno
```

I simboli di visibilità sono `-` per privato, `+` per pubblico e `#` per protetto. Attributi e metodi sottolineati sono statici, cioè appartengono alla classe e non alle singole istanze. Un'interfaccia può essere rappresentata come un rettangolo con la dicitura `<<interface>>` sopra il nome oppure con la notazione a cerchio, detta *lollipop*; nelle interfacce non si indicano attributi, ma solo metodi.

Le frecce principali sono:

ereditarietà freccia con triangolo vuoto rivolta verso la superclasse; il triangolo deve rimanere vuoto;

implementazione di interfaccia freccia tratteggiata con triangolo vuoto rivolta verso l'interfaccia;

associazione linea continua, eventualmente orientata, che rappresenta una relazione strutturale; può avere molteplicità agli estremi;

dipendenza freccia normalmente tratteggiata che esprime un uso, per esempio di un tipo passato come parametro.

Le relazioni parte–tutto distinguono *aggregazione*, con rombo vuoto, quando una parte può esistere indipendentemente dal tutto, e *composizione*, con rombo pieno, quando una parte appartiene esclusivamente al tutto e ne condivide il ciclo di vita. Le molteplicità descrivono quante istanze possono essere collegate: zero o più ordini per un cliente e un cliente per ogni ordine, per esempio, esprimono una relazione uno–a–molti. Nei membri di classe la sottolineatura indica `static`; i nomi di classi e operazioni astratte sono in corsivo.

3.6.1 Dalle relazioni ai campi di Ordine

Nel modello della gestione ordini, `ElencoOrdini` raccoglie gli ordini; ciascun `Ordine` conosce cliente, fornitore e prodotti. `Rubrica` organizza i clienti, mentre `ReportMensile` consulta clienti e fornitori per aggregare gli importi. Cliente conserva nome e provincia; Fornitore il nome; Prodotto nome, costo e linea. Questi collegamenti rendono possibili le operazioni individuate dai requisiti.

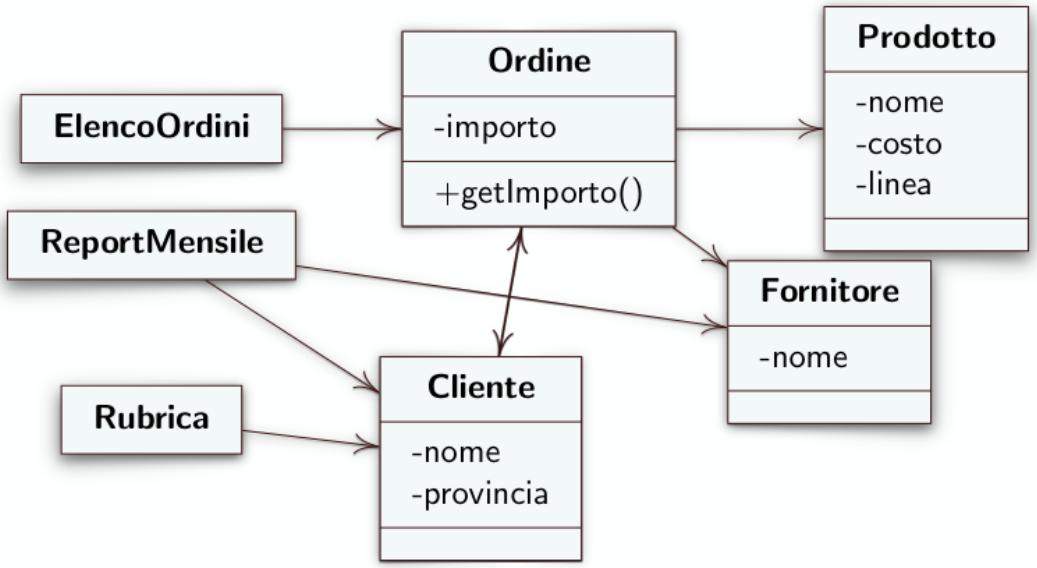


Figura 3.7: Associazioni fra ordini, clienti, prodotti, fornitori e classi di raccolta.

La rappresentazione in Java può esplicitare la corrispondenza fra attributi semplici e riferimenti:

```
class Ordine {
    private int id;
    private java.util.Date data;
    private int importo;
    private boolean archiviato;
    private Cliente cli;
    private Fornitore forn;
    private Stato[] s;
    private Prodotto[] prod;
    public int getImporto() { return importo; }
    // verifica() richiede le regole applicative di validita'.
}
```

Il listato rappresenta la struttura, non l'intera applicazione: Cliente, Fornitore, Stato e Prodotto sono i tipi del modello. `id`, `data`, `importo` e `archiviato` sono dati semplici; `cli` e `forn` rappresentano associazioni a oggetti; gli array rappresentano riferimenti multipli a stati e prodotti. L'array non esprime da solo tutti i limiti di molteplicità, né impone proprietà esclusiva o ciclo di vita comune. Se gli stati appartengono esclusivamente all'ordine si può modellare una composizione; i prodotti condivisi fra ordini non acquistano quel vincolo soltanto perché sono conservati in un campo. Il corpo di `verifica()` dipende da requisiti di validità che devono essere precisati, mentre `getImporto` è già una query definita.

3.7 Interazioni: collaborazione e sequenza

Un *diagramma di collaborazione* mostra le interazioni fra oggetti. Gli oggetti sono rettangoli identificati dalla notazione `nomeIstanza:nomeClasse`; se il nome dell'istanza non importa si scrive solo `:nomeClasse`. Il flusso di messaggi è indicato da frecce accanto alle associazioni fra le istanze. Le etichette dei messaggi contengono un numero sequenziale, eventualmente gerarchico per chiamate annidate, il metodo chiamato e, se necessario, un valore di ritorno.

Un *diagramma di sequenza* mostra le interazioni fra oggetti durante l'esecuzione. L'asse temporale è verticale verso il basso: ciò che avviene prima è in alto. Gli oggetti sono disposti orizzontalmente; la linea di vita (*lifeline*) tratteggiata ne rappresenta l'esistenza e la barra di attivazione un periodo in cui eseguono un'operazione o attendono un ritorno. Un'invocazione è rappresentata da una freccia dal chiamante al ricevente; il ritorno usa una freccia tratteggiata e può essere omesso. La parola `create` indica la creazione di un'istanza. I frame `opt`, `alt` e `loop` racchiudono rispettivamente un comportamento opzionale, alternative e una ripetizione.

Collaborazione e sequenza rendono leggibili aspetti diversi della stessa interazione: la prima evidenzia i collegamenti fra gli oggetti, la seconda l'ordine temporale. Si rappresenta la parte rilevante del comportamento, evitando dettagli che ostacolano la comprensione. Una chiamata a sé stesso (*self-call*), anche ricorsiva, può tornare sulla stessa linea di vita con un'attivazione annidata. È possibile modellare anche la fine della vita di un oggetto; in Java, tuttavia, il recupero della memoria è gestito dal garbage collector della JVM e non da un distruttore esplicito.

Diagramma delle classi e diagramma di sequenza descrivono livelli diversi: il primo mostra tipi e relazioni strutturali, il secondo gli oggetti e i messaggi scambiati in una particolare esecuzione. Non serve quindi una corrispondenza freccia per freccia. Un parametro temporaneo, per esempio, può permettere una chiamata senza richiedere un campo persistente nel chiamante.

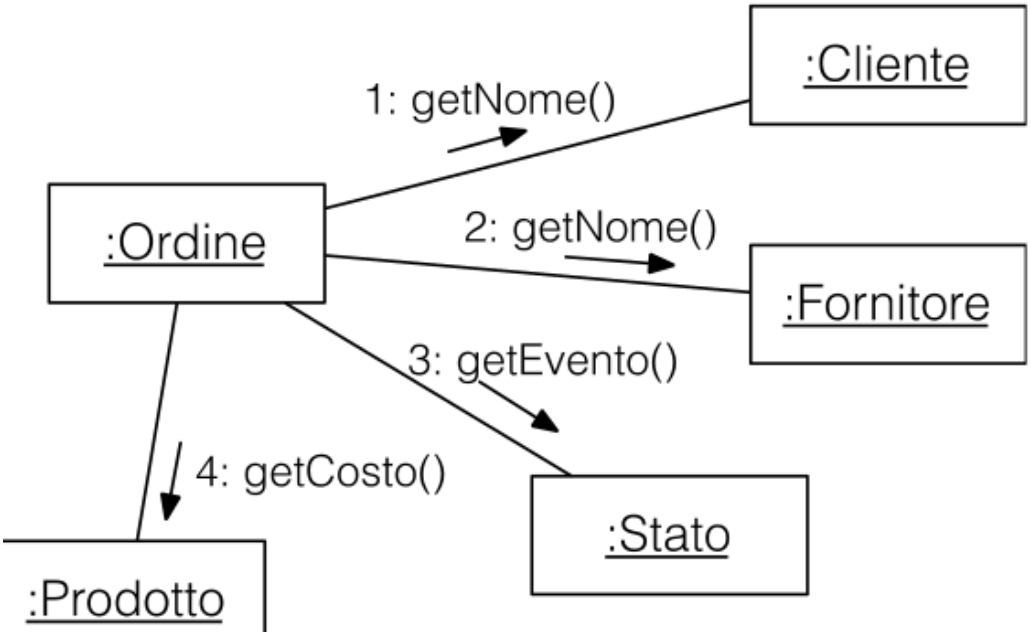


Figura 3.8: Messaggi numerati fra ordine, cliente, fornitore, stato e prodotto.

Nel diagramma di collaborazione gli identificatori delle istanze sono sottolineati. La numerazione rende leggibile l'ordine dei messaggi senza un asse temporale verticale: una notazione gerarchica come `1.1` distingue un messaggio annidato rispetto alla chiamata principale.

Un frame `opt` può racchiudere, per esempio, la creazione di un oggetto richiesta soltanto al primo utilizzo. Il controllo della condizione distingue così un'operazione eseguita sempre da un'inizializzazione differita.

3.7.1 Seguire una sequenza completa: MainEsami

L'esempio Persona–Studente permette di ricostruire tre interazioni collegate. Nel diagramma di creazione MainEsami invoca il costruttore di Studente; la sua inizializzazione richiama il costruttore Persona. Il messaggio `nuovoEsame("Italiano",8)` provoca poi la creazione di un

Esame e l'inserimento nella lista; un secondo messaggio registra Fisica con voto 7. La lifeline di un oggetto creato comincia al messaggio di creazione, non prima.

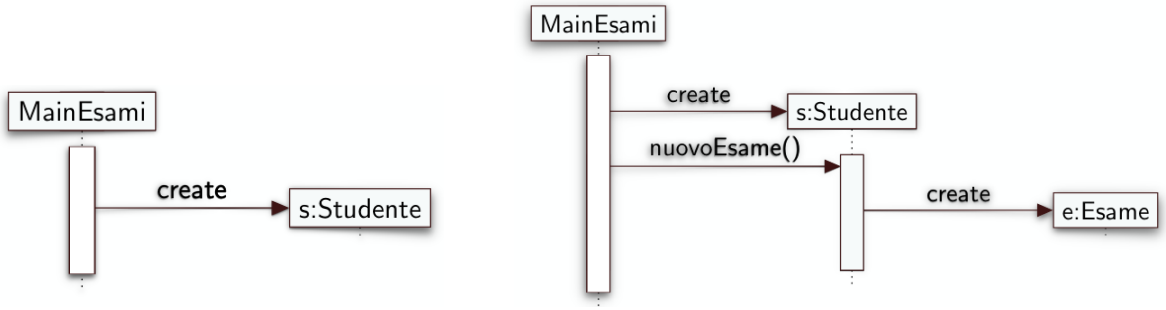


Figura 3.9: Creazione dello Studente e creazione di un Esame attraverso il metodo nuovoEsame.

Nel diagramma di `printAll`, Studente richiama `super.printAll()` per i dati anagrafici e poi stampa matricola ed esami. Il ciclo sugli esami è un frammento `loop`: per ciascun Esame vengono letti materia e voto. La chiamata interna a `getMedia()` è una chiamata dell'oggetto a sé stesso, rappresentabile con un messaggio che rientra sulla stessa lifeline e un'attivazione annidata. Al suo interno un ulteriore ciclo legge `voto()` per accumulare la somma, poi restituisce $15/2 = 7{,}5$. Le frecce di ritorno riportano valori al chiamante; non rappresentano nuovi oggetti.

Infine MainEsami assegna lo stesso riferimento alla variabile Persona e invoca ancora `printAll`. Non nasce una nuova Persona: la lifeline resta quella dello Studente e il dispatch seleziona ancora la ridefinizione. Il diagramma distingue così costruzione, identità dell'istanza, tipo della variabile e metodi effettivamente eseguiti.

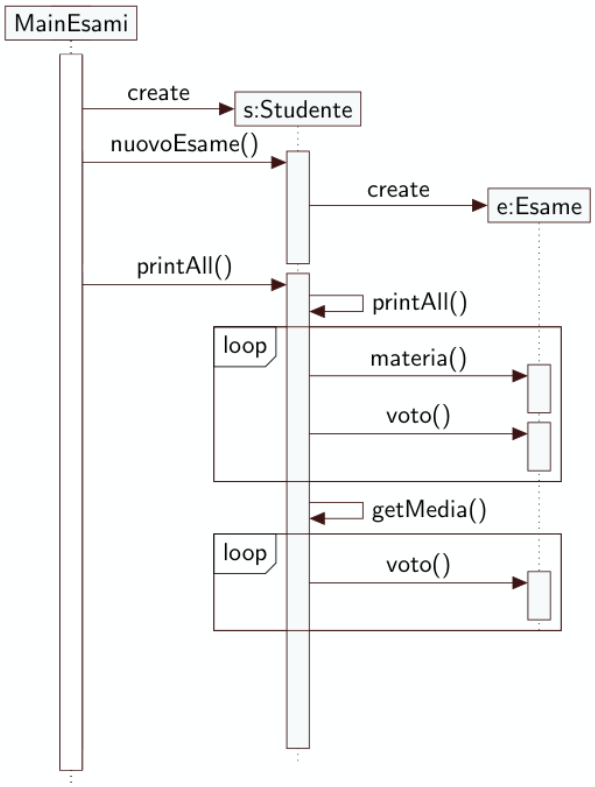


Figura 3.10: Attivazioni, chiamate interne e cicli nella stampa degli esami e nel calcolo della media.

3.8 Individuare classi, attributi e operazioni

L'analisi grammaticale offre un primo procedimento per individuare classi e operazioni nel testo dei requisiti:

1. partire dal testo che descrive i requisiti;
2. individuare i sostantivi: essi suggeriscono classi o attributi;
3. individuare i verbi: essi suggeriscono operazioni o metodi;
4. riconoscere gli oggetti fisici menzionati nel testo come candidati a diventare classi;
5. raggruppare attributi e operazioni nelle classi rispettando coesione, singola responsabilità, KISS e SRP.

Il procedimento è euristico: guida l'analisi ma non produce automaticamente il progetto definitivo. Limitarsi alle parole del cliente può rendere il programma fragile, perché il progettista può dover introdurre classi non nominate nei requisiti. Per distinguere le categorie si possono evidenziare i sostantivi candidati a classi in verde, gli attributi in marrone e i verbi candidati a operazioni in arancione; i colori sono un ausilio visivo, mentre la decisione dipende da responsabilità e coesione.

Si considerino i seguenti requisiti per clienti, fornitori, ordini, prodotti e report mensili:

- deve essere possibile cercare un cliente e mostrare i suoi dati anagrafici;
- la scheda del cliente deve mostrare tutti i dati anagrafici e una lista dei suoi fornitori;
- deve essere possibile selezionare un intervallo di tempo e calcolare l'importo complessivo degli ordini effettuati da un cliente in tale intervallo;
- per ciascun ordine devono essere mostrati nome fornitore, nome cliente, linea di appartenenza dei prodotti e importo complessivo;
- il report mensile deve mostrare per ciascun cliente la provincia e il totale per ciascun fornitore.

L'analisi grammaticale porta alle classi candidate `Cliente`, `Fornitore`, `Ordine`, `Prodotto`, `ReportMensile` e `SchedaCliente`; come attributi candidati dati anagrafici, nome cliente, provincia cliente, linea di appartenenza dei prodotti, importo ordine e nome fornitore; come metodi candidati cercare un cliente, mostrare dati anagrafici e fornitori, calcolare il totale ordini per cliente, selezionare ordini in un intervallo temporale e calcolare il totale mensile per cliente e fornitore.

Capitolo 4

Testing, JUnit e verifica della qualità

4.1 Casi di test, oracle e debugging

Il *testing* è il processo nel quale un programma esercita il codice in determinate condizioni e valuta se il risultato ottenuto è corretto. Un *test case* (caso di test) contiene almeno dati di ingresso, stato iniziale o contesto, risultato atteso e risultato osservato durante l'esecuzione. Il risultato atteso costituisce l'*oracle*: il riferimento con cui confrontare il comportamento effettivo. Per un sistema nuovo l'oracle deriva dai requisiti; durante il refactoring può essere il comportamento della versione precedente.

Un test superato indica che, nelle condizioni controllate, il comportamento osservato coincide con quello atteso; un test fallito segnala una discrepanza da investigare. Dal punto di vista della ricerca dei difetti, un test è utile proprio quando ne rende visibile uno. Il *debugging* consiste nel localizzare e correggere la causa della discrepanza, che può risiedere nel programma ma anche nell'oracle o nella preparazione del test. I test documentano inoltre il comportamento richiesto e proteggono dalle modifiche indesiderate; una suite finita non dimostra normalmente l'assenza di tutti i difetti.

Nel debugging si formula un'ipotesi sulla causa, si prepara un'esecuzione o un'osservazione capace di confermarla o smentirla e si restringe il problema in base all'esito. Cambiare più parti senza un'ipotesi rende difficile attribuire il risultato a una correzione. Un difetto può essere nella specifica, nella logica o nella fixture: anche il test va controllato.

4.2 Verifica, validazione e livelli di test

La *verifica* risponde alla domanda "stiamo costruendo il sistema nel modo corretto?": riguarda il modo in cui il software è implementato, la qualità del codice e il rispetto della progettazione. La *validazione* risponde alla domanda "stiamo costruendo il sistema corretto?": controlla che il prodotto soddisfi bisogni e richieste dell'utente. Le tecniche white-box aiutano a esaminare la struttura interna, mentre quelle black-box controllano il comportamento rispetto alle specifiche. Sono prospettive di progettazione dei test, non definizioni alternative di verifica e validazione.

I difetti possono nascere nella specifica, per esempio da requisiti ambigui, contraddittori o imprecisi; nella progettazione, per esempio da algoritmi, strutture dati o interfacce fra moduli non corretti; nel codice, cioè nell'implementazione; oppure negli stessi casi e piani di test. I test dovrebbero essere affiancati alle *verifiche statiche*, cioè controlli sul codice svolti senza eseguirlo. Possono essere organizzati per livello:

test unitari isolano un singolo componente, quale metodo, oggetto, funzione o piccolo raggruppamento di essi;

test di integrazione esercitano la collaborazione fra componenti e le rispettive interfacce, eventualmente usando database, rete o file system reali;

test di sistema verificano il sistema complessivo, comprese funzionalità e proprietà emergenti dalle interazioni;

alpha test sono condotti dagli sviluppatori usando dati reali del cliente per controllare il soddisfacimento delle richieste;

beta test sono condotti dai clienti su un prodotto quasi completo.

L'uso di dati reali del cliente negli alpha test richiede modalità di accesso e riservatezza concordate. Un *NDA* (accordo di non divulgazione) può disciplinare questo accesso: autorizzare i dati per il test non implica autorizzarne altri usi.

Una *versione alfa* è una versione integrata che può avere difetti e parti mancanti; una *versione beta* include tutte le feature ma può ancora avere difetti; la *versione gold* è il prodotto finale pronto alla commercializzazione. Un *test di regressione* riusa una suite dopo una modifica, una correzione di bug o una nuova funzionalità per accertare che ciò che funzionava continui a funzionare. Una *test suite* è un insieme di test case eseguiti insieme.

Per escludere difetti mediante la sola esecuzione bisognerebbe controllare tutte le condizioni rilevanti con un oracle corretto: un obiettivo normalmente impraticabile. Per una funzione con due parametri di tipo `int`, assumendo 2^{32} valori possibili per ciascun parametro, i casi sono $2^{32} \cdot 2^{32} = 2^{64} \simeq 1{,}8 \cdot 10^{19}$. Se un caso richiedesse $1\,\text{ns} = 10^{-9}\,\text{s}$, il tempo totale sarebbe circa $1{,}8 \cdot 10^{10}\,\text{s}$, cioè circa 600 anni assumendo $1\,\text{anno} \simeq 3 \cdot 10^{7}\,\text{s}$. Si selezionano quindi casi efficaci: quelli tipici dimostrano le capacità richieste, quelli sulle funzionalità esistenti proteggono dalle regressioni e quelli limite esplorano condizioni nelle quali sono frequenti i difetti. L'ordine delle priorità dipende anche dalla criticità delle funzionalità.

Una suite utile comprende molti test unitari rapidi, eseguiti frequentemente, e un numero più contenuto di test d'integrazione su risorse reali, spesso più lenti. La distribuzione dipende dai rischi e dalle dipendenze, non da una proporzione fissa. Come ordine pratico iniziale si può proteggere prima il comportamento già disponibile rispetto alle nuove estensioni e controllare prima i casi tipici rispetto ai limiti; criticità elevate o difetti noti possono motivare priorità diverse.

Esempi aiutano a distinguere le origini dei difetti: un algoritmo che permette una divisione per zero, un campo necessario assente o un parametro di interfaccia di tipo incoerente possono rivelare errori di progetto. Overflow, conversione di tipo errata, precedenza degli operatori, variabili non inizializzate o riassegnate prima dell'uso possono invece derivare dall'implementazione. Il confine non è sempre netto: occorre ricostruire quale decisione ha introdotto il problema. Anche piani e casi di test possono essere difettosi.

4.3 Progettazione black-box e white-box

Un test *black-box* progetta i casi a partire dalle specifiche e dai requisiti, senza usare la struttura interna o l'algoritmo. Chi lo scrive può quindi ragionare sugli scenari normali, particolari e negativi senza essere condizionato dall'implementazione. Un test *white-box*, detto anche *glass-box* o strutturale, richiede invece conoscenza del codice sorgente e sceglie i casi osservando rami, istruzioni e percorsi di controllo.

Nel black-box il dominio degli input può essere partizionato in *classi di equivalenza*: ciascuna è un insieme di dati per i quali si prevede un comportamento uniforme. Si sceglie almeno un rappresentante per classe e, in particolare, valori ai suoi estremi, dove i difetti sono frequenti. Ciò riduce molti possibili input a un insieme gestibile di casi rappresentativi. Occorre includere sia dati validi sia dati non validi; un *fault model*, cioè un'ipotesi sul tipo di difetto cercato, può riguardare un intero intervallo che l'implementazione elabora in modo scorretto.

Nel white-box i *test di percorso* cercano di eseguire almeno una volta ogni percorso indipendente del programma. Nel grafo di controllo i nodi rappresentano condizioni e gli archi i possibili flussi; la complessità ciclomatica misura il numero di percorsi indipendenti da considerare. Coprire i percorsi indipendenti non implica coprire ogni combinazione possibile di condizioni. Un *test sotto stress* esegue il sistema oltre il carico massimo previsto e osserva che il fallimento, se avviene, non sia catastrofico; è rilevante soprattutto per sistemi distribuiti.

4.3.1 Intervallo ammesso, confini e stress

Se una funzione ammette interi da 4 a 20, le partizioni sono $x < 4$, $4 \leq x \leq 20$ e $x > 20$. I casi 3, 4, 12, 20 e 21 coprono due classi non valide, i due confini validi e un valore interno. Scegliere un rappresentante presume un comportamento omogeneo nella partizione: è una strategia per cercare difetti, non una dimostrazione che tutti i valori siano equivalenti nell'implementazione. Errori di disuguaglianza come `x<20` invece di `x<=20` motivano i controlli ai confini.

I difetti possono addensarsi in componenti già problematici: complessità o precedenti fallimenti suggeriscono altri casi mirati, senza escludere le parti mai fallite. Nello stress si cercano conseguenze come perdita di dati o indisponibilità del servizio. Nei sistemi distribuiti conta anche il degrado della rete. I risultati possono richiedere di completare le specifiche sul comportamento oltre il carico previsto e sulle modalità di recupero.

4.4 Osservabilità, contratti e indipendenza dei test

Per testare un oggetto, il suo stato rilevante deve essere *osservabile*, cioè leggibile mediante metodi appropriati, per esempio getter o query. Una *query* restituisce un'informazione senza modificare lo stato. Se non è possibile osservare l'effetto rilevante, il test non può confrontarlo con l'oracle. Un metodo difficile da testare può segnalare che svolge troppi compiti e non rispetta il Principio di Singola Responsabilità (SRP).

Una *precondizione*, nel Design by Contract, è una condizione che deve essere vera prima di eseguire correttamente un metodo. Per accedere all'elemento di indice i di un array o lista di dimensione n, per esempio, deve valere $i \geq 0$ e $n > i$: i è l'indice richiesto e n il numero di elementi. I test devono esplicitare anche i casi negativi, quali lista vuota, indice fuori intervallo e limiti minimo e massimo. I controlli iniziali, o *guard clause*, permettono di trattare il caso negativo prima del percorso principale; l'esito previsto, compresa un'eventuale eccezione, deve essere stabilito dal contratto.

Un buon test rispetta i principi *FIRST*:

Fast è veloce, così la suite può essere eseguita frequentemente;

Isolated/Independent è isolato e indipendente: non dipende dall'ordine di esecuzione né dallo stato lasciato da un altro test;

Repeatable è ripetibile: a parità di input dà lo stesso esito su macchine e momenti diversi;

Self-validating si autovaluta e comunica automaticamente passaggio o fallimento, con un messaggio chiaro;

Timely è scritto nel momento opportuno, idealmente prima del codice quando si usa TDD.

I test devono restare semplici: se un test complesso fallisce, può essere difficile capire se il difetto sia nel test o nel sistema. Come criterio pratico di organizzazione, si prepara almeno una classe di test per ciascuna classe dell'applicazione e almeno un test per ciascun metodo significativo.

Rendere osservabile un effetto non significa esporre indiscriminatamente tutti i campi interni. È preferibile una query pubblica coerente con il dominio o la verifica di un risultato

già previsto dal contratto. Se l'unico modo di testare un metodo è leggere molte variabili intermedie, può essere utile separarne le responsabilità.

Le eccezioni vanno gestite al livello che dispone delle informazioni per recuperare dall'errore, oppure propagate o tradotte secondo il contratto. Un `catch` vuoto può nascondere un difetto; allo stesso modo, restituire `null` nel caso negativo richiede una decisione esplicita e coerente con l'uso del metodo.

Quando una condizione negativa è prevista e ordinaria, è spesso più chiaro controllarla con una guard clause anziché usare un'eccezione come flusso abituale. Una libreria va inoltre incapsulata se i suoi errori espongono dettagli estranei al dominio: il livello applicativo può tradurli o gestirli. Ciò non introduce un divieto generale di propagare eccezioni Java; la scelta va resa esplicita nel contratto e affidata al livello capace di decidere il recupero.

4.5 TDD, BDD e struttura del caso di test

Il *Test-Driven Development* (TDD) è una pratica di sviluppo in cui si scrive prima il test e poi il codice minimo necessario a soddisfare il requisito. Il suo ciclo è:

1. **Red**: si scrive un test non ancora superato; nello scegliere nomi di classi e metodi si compie già progettazione;
2. **Green**: si implementa il minimo codice che fa passare il test;
3. **Refactor**: si migliora la struttura senza cambiare il comportamento osservabile, quindi si riesegue la suite.

Il principio YAGNI limita l'implementazione alle funzionalità richieste, evitando estensioni speculative. I test guidano il passo corrente, ma averli tutti superati non dimostra che ogni requisito sia stato formulato o verificato correttamente.

La struttura *AAA* separa un test in *Arrange*, preparazione dello stato; *Act*, chiamata dell'operazione; *Assert*, confronto con il risultato atteso. Il *Behaviour-Driven Development* (BDD) esprime invece il comportamento in forma *Given–When–Then*: dato un contesto, quando avviene un'azione, allora il sistema produce un effetto atteso. BDD rende il test leggibile come documentazione dal punto di vista dell'utente ed è complementare al TDD, che descrive soprattutto un processo di sviluppo.

Given–When–Then, abbreviato GWT, corrisponde ad Arrange–Act–Assert: Given prepara il contesto, When esegue l'azione e Then esprime il risultato atteso. Si possono usare descrizioni BDD anche senza scrivere il test prima del codice; in quel caso la descrizione del comportamento non comporta, da sola, l'adozione del ciclo TDD.

4.5.1 Dallo scheletro compilabile al test Red

Il TDD richiede di distinguere l'errore di compilazione dal fallimento di un'asserzione. Se il test nomina una classe o un metodo ancora inesistente, prima si prepara lo scheletro minimo che consente di compilare: costruttore, firme e risultati provvisori. Si esegue quindi il test e si verifica che fallisca per il comportamento mancante, poi si implementa la funzionalità e si raggiunge Green. Uno scheletro che ritorna sempre zero non è ancora una soluzione: serve a rendere osservabile il fallimento atteso.

4.6 Copertura, percorsi e criteri di arresto

Nel grafo di flusso usato dai test di percorso la complessità ciclomatica è:

$$cc = A - N + 2. \tag{4.1}$$

Qui cc è la complessità ciclomatica, A il numero di archi e N il numero di nodi di un grafo di controllo connesso. Il valore indica la dimensione di un insieme di percorsi indipendenti: orienta la progettazione dei test strutturali, senza rendere esaustive tutte le combinazioni di esecuzione. Una sequenza priva di diramazioni ha $cc = 1$. Una condizione, rappresentata da un nodo decisionale e uno di uscita con due archi alternativi, ha $cc = 2 - 2 + 2 = 2$: occorrono casi che esercitino entrambi gli esiti.

La *code coverage* misura quale parte del codice viene esercitata dalla suite secondo un criterio dichiarato. Per la copertura delle linee si usa:

$$C = \frac{L_{\text{eseguite}}}{L_{\text{totali}}} \cdot 100\%. \tag{4.2}$$

Qui C è la copertura, L_{eseguite} il numero di linee raggiunte almeno una volta e L_{totali} il numero totale di linee considerate. Per esempio, $C = 70\%$ indica che è stato eseguito il 70% delle linee. Anche il 100% lascia aperta la possibilità di input, combinazioni o requisiti non verificati. Soglie del 90–95% possono orientare il lavoro, ma vanno motivate in base al sistema e non garantiscono correttezza; le parti critiche possono richiedere controlli più rigorosi. Il tracciamento della copertura può essere integrato nell'ambiente di sviluppo.

Criteri diversi misurano aspetti diversi: la copertura di funzioni conta le funzioni eseguite; quella di costrutti le istruzioni; quella di archi i flussi attraversati nel grafo di controllo. La copertura di rami richiede di esercitare le alternative, per esempio il ramo `if` e il ramo `else`; quella di condizioni richiede che ogni sottoespressione booleana assuma almeno una volta valore vero e falso. La copertura di decisioni riguarda gli esiti delle espressioni che determinano il flusso; l'esecuzione dei punti di ingresso e uscita è un controllo ulteriore, da esplicitare quando si combinano più criteri.

Per `if (x && y) a=1; else a=0;`, i casi $(x, y) = (vero, vero)$ e $(falso, falso)$ raggiungono entrambi gli esiti della decisione. In Java, nel secondo caso `y` non viene valutato per il corto circuito: per osservare anche `y=false` serve, per esempio, $(vero, falso)$. Occorre quindi distinguere valori assegnati alle variabili e sottoespressioni realmente valutate. La copertura delle decisioni controlla i due esiti della decisione composta; combinandola con la copertura delle funzioni e dei punti di ingresso e uscita si ottiene un criterio strutturale più ampio. Quando il termine è usato per tale criterio composto, vanno dichiarate esplicitamente tutte queste componenti; non implica la verifica di ogni possibile combinazione di input.

Il rapporto di copertura vale per l'unità scelta: linee eseguite su linee totali, rami eseguiti su rami totali, e così via. Una soglia del 60% può illustrare un obiettivo iniziale, come quelle del 90–95% illustrano obiettivi più elevati: nessuna delle percentuali costituisce una garanzia. Una build notturna, o *nightly build*, può compilare e rieseguire una suite ampia come controllo di regressione, affiancando i test rapidi eseguiti durante le modifiche.

Il *bug trend* misura la frequenza con cui sono scoperti difetti. Copertura desiderata e bug trend sono condizioni di arresto del testing: quando la frequenza tende a zero può indicare sia assenza di nuovi difetti sia un aumento eccessivo del costo per trovarli; non costituisce, da sola, prova di correttezza.

Nell'IDE, il tracciamento della copertura usa il verde per linee eseguite almeno una volta e il rosso per linee non raggiunte. Questo colore riguarda l'esecuzione, mentre Red e Green del TDD riguardano l'esito dei test: una linea eseguita può contenere un errore che l'oracle non rileva. I test dell'usabilità dell'interfaccia richiedono inoltre valutazioni che non si riducono facilmente a unit test automatizzati.

4.7 JUnit: annotazioni, asserzioni e fixture

JUnit è il framework per scrivere ed eseguire test di applicazioni Java. Fornisce un motore di esecuzione, o *test runner*, e API per definire test e asserzioni. Un'*annotazione* Java è un

metadato associato a elementi del codice, come classi, metodi e campi, che fornisce informazioni aggiuntive agli strumenti. In JUnit:

`@Test` identifica un metodo di test;

`@BeforeEach` identifica un metodo eseguito prima di ogni test, utile per ottenere uno stato fresco;

`@BeforeAll` identifica un metodo eseguito una sola volta prima di tutti i test della classe; nella forma usata qui il metodo è `static`;

`@ParameterizedTest` indica un test eseguito più volte con input differenti; `@CsvSource` può fornire tali input in forma tabellare CSV.

Le *asserzioni* della classe `Assertions` effettuano il confronto automatico con l'oracle. `assertEquals`, applicata agli argomenti `expected` e `actual`, verifica l'uguaglianza fra valore atteso e valore effettivo; `assertTrue` verifica una condizione booleana vera; `assertNull` verifica un riferimento `null`. Per risultati in virgola mobile occorre una tolleranza: se r è il risultato, a il valore atteso e $\varepsilon = 0{,}01$ la tolleranza, una verifica esplicita della distanza fra i valori è

$$|r - a| < \varepsilon. \tag{4.3}$$

4.7.1 Catalogo: test di somma e massimo

Un `Catalogo` conserva prodotti inseriti come stringhe CSV, cioè campi separati da virgole: nome, prezzo, responsabile e data. `svuota()` elimina i prodotti, `inserisci(...)` ne aggiunge uno, `calcolaSomma()` aggiorna il totale letto da `getSomma()`, mentre `calcolaMassimo()` aggiorna il massimo letto da `getMassimo()`. La preparazione condivisa del test, detta *fixture*, usa quattro prezzi: 12,00, 10,50, 2,00 e 21,50. Il totale atteso è 46,00 e il massimo 21,50. `@BeforeAll` prepara i dati una sola volta; i metodi `@Test` controllano le due operazioni.

```
public class AppTest {
    private static final Catalogo catalog = new Catalogo();

    @BeforeAll
    public static void init() {
        catalog.svuota();
        catalog.inserisci("album,12.00,Rick,2025-01-15");
        catalog.inserisci("libro,10.50,Rick,2025-01-15");
        catalog.inserisci("matite,2.00,Rick,2025-01-15");
        catalog.inserisci("zaino,21.50,Rick,2025-01-15");
    }

    @Test
    public void sommaValori() {
        catalog.calcolaSomma();
        assertEquals(46.00f, catalog.getSomma(), 0.01f);
    }

    @Test
    public void massimoValori() {
        catalog.calcolaMassimo();
        assertEquals(21.50f, catalog.getMassimo(), 0.01f);
    }
}
```

L'esempio è auto-validante perché l'asserzione sostituisce la lettura manuale di messaggi `OK`/`FAILED`. Poiché lo stato è condiviso e inizializzato una sola volta, esso mostra l'uso dell'annotazione, ma per l'indipendenza FIRST è necessario verificare che ciascun test non modifichi

lo stato in modo da influenzare l'altro; quando ciò non è garantito è preferibile preparare uno stato nuovo con `@BeforeEach`.

Il metodo di inserimento CSV offre casi concreti da osservare nella copertura:

```
String[] campi = riga.split(",");
if (campi.length != 4) {
    System.err.println("Riga malformata: " + riga);
    return;
}
if (campi[1].trim().equals("valore")) return; // intestazione
String nome = campi[0].trim();
float prezzo = Float.parseFloat(campi[1].trim());
```

Dopo i controlli il metodo costruisce il prodotto con gli altri due campi; un errore di conversione richiede un esito previsto dal contratto. Se la suite usa solo righe corrette, il ramo della riga malformata può restare rosso anche quando la lettura del file e il ciclo sono verdi. Vanno aggiunte righe con numero di campi diverso da quattro, intestazione e prezzo non numerico, controllando l'effetto atteso. Il colore di una linea non dimostra che il parser riconosca tutti i CSV possibili: lo `split` semplice non gestisce per sé virgolette o virgole interne ai campi.

La fixture a due prodotti può essere inizializzata una volta con un campo `static` e `@BeforeAll`, ottenendo somma 33,50 e massimo 21,50. La variante seguente con `@BeforeEach` cambia intenzionalmente il ciclo di preparazione: ogni test riceve un catalogo nuovo invece di condividere la medesima istanza.

Il test parametrizzato della somma rende visibili i quattro casi nella classe CalcTest, definita insieme al metodo da controllare. Ogni riga CSV fornisce due operandi e il risultato atteso.

4.7.2 Variante con due prodotti e preparazione indipendente

Una fixture alternativa contiene soltanto album da 12,00 e zaino da 21,50: il totale atteso diventa 33,50. Preparare un nuovo catalogo con `@BeforeEach` rende esplicita l'indipendenza dei test. Valori attesi e dati iniziali devono sempre descrivere lo stesso scenario:

```
private Catalogo catalog;

@BeforeEach
void preparaCatalogo() {
    catalog = new Catalogo();
    catalog.inserisci("album,12.00,Rick,2025-01-15");
    catalog.inserisci("zaino,21.50,Rick,2025-01-15");
}

@Test
void sommaDueProdotti() {
    catalog.calcolaSomma();
    assertEquals(33.50f, catalog.getSomma(), 0.01f);
}
```

Le annotazioni di base appartengono a `org.junit.jupiter.api`. Per usare i frammenti si importano i tipi e le asserzioni necessari:

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;
```

La tolleranza rende esplicita la politica di confronto dei floating-point. Lista vuota, un solo elemento, valori ai limiti e input non validi richiedono casi ulteriori, con un comportamento atteso definito dal contratto.

4.7.3 Test eseguibili di Pagamenti e lettura da file

Il caso `Pagamenti` permette di distinguere un controllo manuale dell'output da un test che si autovaluta. La fixture inserisce 321,01, 531,7 e 1234,5: somma attesa 2087,21, massimo 1234,5. La seguente classe usa la variante con query e si può eseguire senza un framework:

```
import java.io.IOException;

class TestPagamenti {
    private final Pagamenti pgm = new Pagamenti();
    private void initLista() {
        pgm.svuota();
        pgm.inserisci(321.01f);
        pgm.inserisci(531.7f);
        pgm.inserisci(1234.5f);
    }
    private void verifica(boolean ok, String nome) {
        System.out.println((ok ? "OK " : "FAILED ") + nome);
    }
    void testSommaValori() {
        initLista();
        verifica(Math.abs(pgm.calcolaSomma() - 2087.21f) < 0.01f,
                 "test somma val");
    }
    void testMaxValore() {
        initLista();
        verifica(Math.abs(pgm.calcolaMassimo() - 1234.5f) < 0.01f,
                 "test massimo val");
    }
    void testVuoto() {
        pgm.svuota();
        verifica(pgm.calcolaSomma() == 0, "somma vuota");
        verifica(pgm.calcolaMassimo() == 0, "massimo vuoto");
    }
    void testLeggiFile() {
        try {
            pgm.leggiFile("csv", "Importi.csv");
            verifica(true, "test leggi file");
        } catch (IOException e) {
            verifica(false, "test leggi file");
        }
    }
    public static void main(String[] args) {
        TestPagamenti t = new TestPagamenti();
        t.testLeggiFile();
        t.testSommaValori();
        t.testMaxValore();
        t.testVuoto();
    }
}
```

Con un file leggibile e composto da numeri validi le prime tre righe iniziano con `OK`; se il file non è trovato, la prima diventa `FAILED test leggi file`, mentre somma e massimo passano

perché ricreano la propria fixture. Il test di lettura controlla soltanto l'assenza di `IOException`: per verificare il contenuto occorre confrontare anche i dati o gli aggregati attesi. Una riga non numerica richiede un caso distinto, perché non è un errore di I/O. Il confronto esatto con un letterale decimale non è appropriato per questi aggregati in virgola mobile: la tolleranza va applicata sia alla somma sia al massimo.

Nella variante con risultati memorizzati la stessa fixture si inserisce come testo, poi si eseguono `converti`, `calcolaSomma` e `calcolaMassimo`; le verifiche leggono `getSomma` e `getMassimo`. I valori attesi rimangono identici se rappresentazione dei duplicati, dominio e politica sugli errori coincidono. Per entrambi i contratti si aggiungono casi con un duplicato, un solo valore, letture successive e lista svuotata.

Un comportamento BDD dello stesso caso è: *Given* una raccolta contenente i tre importi distinti; *When* ne richiedo la somma; *Then* ottengo 2087,21 entro la tolleranza dichiarata. L'oracle viene così espresso prima dell'implementazione scelta.

4.7.4 Un metodo con input intero: scheletro, test e soluzione

Per provare un metodo che riceve un intero si può specificare prima l'addizione: `add(2,3)` deve restituire 5. Lo scheletro `int add(int a,int b) { return 0; }` compila ma fallisce il test; sostituire il corpo con `return a+b;` realizza il comportamento per il dominio considerato. Un test parametrizzato completo rende espliciti i quattro casi:

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.assertEquals;

class Calc {
    int add(int a, int b) { return a + b; }
}
class CalcTest {
    private final Calc calculator = new Calc();
    @ParameterizedTest
    @CsvSource({"1,1,2", "2,3,5", "10,5,15", "-1,1,0"})
    void testAdd(int a, int b, int expected) {
        assertEquals(expected, calculator.add(a, b));
    }
}
```

Questi casi non specificano il trattamento dell'overflow degli interi: per estendere il requisito oltre l'aritmetica `int` occorrerebbe un contratto ulteriore.

4.8 Maven: progetto, dipendenze e ciclo di build

Apache Maven automatizza build, dipendenze, test, packaging e rilascio. La struttura convenzionale pone il codice applicativo in `src/main/java`, i test in `src/test/java` e la descrizione del progetto nel file `pom.xml`, il Project Object Model (POM). Il POM elenca nome, versione, dipendenze, plugin e configurazioni; una dipendenza con scope `test` è disponibile solo per i test. Le fasi `mvn compile`, `mvn test`, `mvn package` e `mvn verify` eseguono, nell'ordine, compilazione, test unitari, produzione del file distribuibile `.jar` e controlli di qualità; l'esecuzione di una fase include le precedenti.

Una dipendenza è identificata da coordinate e versione; Maven scarica anche le dipendenze transitive. Per esempio, una configurazione che usa JUnit 6 può fissare esplicitamente la versione `6.0.3`, mantenendo coerenti librerie, JDK e plugin. Test di integrazione e controlli aggiuntivi richiedono una configurazione adeguata: la presenza della fase `verify` non attiva automaticamente qualsiasi verifica.

```
progetto/
  pom.xml
  src/main/java/com/example/   # codice applicativo
  src/test/java/com/example/   # classi di test
```

4.8.1 Creare il progetto e fissare la configurazione

Si può creare la struttura con l'IDE oppure con `mvn archetype:generate`, scegliendo un archetipo Java e le coordinate del progetto. Per aggiungere una libreria si cercano in Maven Central gruppo, artefatto e versione; si copia il blocco `dependency` nella sezione `dependencies` del POM e si assegna lo scope appropriato. Le dipendenze transitive sono le librerie richieste a loro volta dalla dipendenza dichiarata: non occorre ricopiare manualmente ogni archivio.

Un POM completo per gli esempi con Java 25 e JUnit 6.0.3 è:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>prj-junit6</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>6.0.3</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>6.0.3</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <version>6.0.3</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.14.1</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
```

```
      </plugin>
    </plugins>
  </build>
</project>
```

`groupId`, `artifactId` e `version` identificano il progetto; `modelVersion` identifica il modello del POM. `release` seleziona livello del linguaggio, bytecode e API della piattaforma obiettivo: per questo non occorre ripetere anche `source` e `target` con lo stesso valore. Il JDK usato deve supportare la release selezionata. L'API permette di scrivere i test, l'engine li esegue e `params` abilita le sorgenti dei test parametrizzati. Il plugin compiler applica la configurazione Java; Surefire avvia i test nel ciclo Maven. Fissare le versioni rende la configurazione riproducibile, senza sostenere che siano le uniche possibili.

Un JAR, *Java Archive*, è un archivio compresso di classi compilate e risorse, eventualmente con metadati di avvio. `mvn package` lo produce nella cartella `target`; non implica che ogni JAR sia avviabile direttamente. La verifica della suite si ottiene con `mvn test`, mentre `verify` comprende le fasi precedenti e le ulteriori verifiche effettivamente configurate.

4.9 Procedimento per progettare ed eseguire un test

Per progettare ed eseguire un test si può seguire questa sequenza:

1. identificare il requisito, il componente e il comportamento da controllare;
2. stabilire oracle, precondizioni, stato iniziale e input validi, non validi e di confine; per un black-box scegliere rappresentanti delle classi di equivalenza;
3. rendere osservabile il risultato mediante l'interfaccia pubblica o query appropriate;
4. organizzare il caso in Arrange–Act–Assert, predisponendo uno stato indipendente dagli altri test;
5. formulare l'asserzione; per valori floating-point applicare una tolleranza anziché uguaglianza esatta;
6. eseguire la suite con il runner, interpretare un fallimento come segnale da investigare e correggere con debugging;
7. dopo ogni modifica rieseguire la suite come regressione e osservare la copertura senza confonderla con la correttezza.

Capitolo 5

Refactoring e miglioramento della struttura del codice

5.1 Obiettivo e limite del refactoring

Il *refactoring* è il processo di trasformazione di un sistema software che ne migliora la struttura interna senza modificarne il comportamento esterno: dopo la trasformazione restano soddisfatti gli stessi requisiti funzionali. Per *struttura interna* si intendono, per esempio, la ripartizione delle responsabilità fra classi e metodi, la duplicazione e le dipendenze; per *comportamento esterno* ciò che osserva chi invoca il sistema, quali risultati, effetti e interfacce pubbliche previste dai requisiti.

Non è dunque refactoring né la correzione di un difetto né l'aggiunta di una funzionalità: entrambe cambiano il comportamento desiderato. Il refactoring è manutenzione del codice esistente e si applica a codice che già funziona. Una modifica che altera il comportamento per adattare il programma a nuove librerie o piattaforme è invece una diversa attività di manutenzione, anche se può essere accompagnata da refactoring.

La progettazione non è necessariamente adeguata al primo tentativo e tende a deteriorarsi quando si aggiungono modifiche. Il refactoring la migliora gradualmente durante e dopo l'implementazione, così che requisiti futuri siano più facili da incorporare. Può ridurre la quantità di codice, eliminare duplicazioni, rendere più leggibili strutture logiche con molti condizionali annidati, separare classi con troppe responsabilità e diminuire l'accoppiamento, cioè la dipendenza fra componenti.

Un beneficio del refactoring è preservare e migliorare la *manutenibilità*, cioè la facilità di comprendere, correggere e modificare il software. In particolare, il principio DRY (*Don't Repeat Yourself*) richiede di non duplicare codice: se un frammento copiato contiene un difetto, la correzione va ripetuta in ogni copia e le copie possono divergere. Una buona struttura rende anche più semplice individuare difetti, non perché il refactoring li corregga per definizione, ma perché richiede di comprendere più profondamente il codice.

Il *debito tecnico* è la conseguenza futura di una soluzione rapida ma poco curata: nel tempo si pagano "interessi" sotto forma di maggiore tempo per comprendere e modificare il codice. Una scadenza può indurre a scrivere una soluzione poco leggibile; non migliorarla quando possibile aumenta tale debito. Il refactoring è quindi un intervento preventivo contro il degrado della progettazione, non una garanzia che il sistema sia corretto o che il codice diventi sempre più breve.

Il refactoring va distinto dal *reverse engineering*, l'analisi di un sistema per ricavarne informazioni su comportamento o struttura, e dal *re-engineering*, la ricostruzione di un sistema con struttura differente. Queste attività possono essere utili su software esistente, ma hanno obiettivi distinti: il tratto essenziale del refactoring è la preservazione del comportamento.

Prima della trasformazione occorre avere test superati sul comportamento che deve rimanere invariato; dopo ciascun piccolo passo si compila ed esegue nuovamente la suite di test. Una *suite* è l'insieme dei casi di test eseguiti insieme. Se un test fallisce, il passo ha presumibilmente compromesso un comportamento già funzionante: si interviene subito, invece di accumulare molte modifiche e rendere difficile localizzare l'errore.

I test sono una rete di sicurezza, non una dimostrazione assoluta: proteggono soltanto i comportamenti che osservano e il cui oracle è corretto. L'*oracle* è il riferimento per decidere se un risultato è atteso; nel refactoring, l'output della versione precedente può svolgere questo ruolo. Se il codice iniziale non è corretto, il refactoring non è il modo per correggerlo: prima si chiarisce il comportamento richiesto e si corregge il difetto con test adeguati, poi si migliora la struttura.

Nel ciclo TDD il refactoring segue il test superato: Red individua un comportamento mancante, Green lo realizza e Refactor ne migliora la struttura mantenendo verdi i test. La revisione fra due programmatori che lavorano insieme, detta *pair programming*, può aiutare a discutere le trasformazioni e ad affrontarle con maggiore fiducia. Questa collaborazione, i test frequenti e il design semplice si sostengono reciprocamente.

5.1.1 Invarianza e sequenza di lavoro

La conservazione del comportamento può essere espressa come un'invariante: se P è il programma prima della trasformazione, P' quello dopo e x un input del dominio considerato, l'obiettivo è

$$\mathrm{obs}(P, x) = \mathrm{obs}(P', x). \tag{5.1}$$

La funzione $\mathrm{obs}(P, x)$ rappresenta il comportamento osservabile di P sull'input x, compresi risultato ed effetti previsti. L'uguaglianza deve valere per tutti gli input rilevanti; una suite finita la controlla soltanto per i casi che esegue.

Un procedimento operativo comune alle tecniche presentate è:

1. identificare un sintomo di debolezza del progetto e il comportamento che deve restare invariato;
2. verificare che il codice attuale funzioni e che esista una suite di regressione significativa;
3. scegliere una tecnica e compiere un solo piccolo cambiamento strutturale;
4. compilare per intercettare errori di tipo, visibilità e riferimenti;
5. eseguire tutti i test; se falliscono, correggere o annullare subito l'ultimo passo;
6. ripetere fino a ottenere una struttura più semplice, senza introdurre funzionalità nuove.

5.2 Code smell e scelta della trasformazione

Un *code smell*, o *bad smell*, è un sintomo nel codice che suggerisce una possibile debolezza della progettazione e invita a esaminarla; non è, da solo, la prova di un difetto. La scelta della trasformazione dipende dal sintomo:

codice duplicato lo stesso comportamento appare in più punti; si può usare *Estrai Metodo*;

metodo lungo o con commenti interni il commento spesso segnala un blocco con un intento nominabile; si usa *Estrai Metodo*, spesso preceduto da *Sostituisci Temp con Query*;

classe grande una classe con molti attributi, molte linee o più gruppi di responsabilità suggerisce *Estrai Classe*;

lista di parametri lunga si può usare *Introduci Oggetto Parametro*, cioè raggruppare dati correlati in un oggetto parametro;

cambiamenti divergenti parti della stessa classe cambiano per ragioni diverse; si usa *Estrai Classe* per separare le responsabilità.

Una *responsabilità* è un compito o motivo di cambiamento coerente affidato a un elemento del programma. La scelta è guidata dall'intento e dalle dipendenze effettive, non da una soglia numerica universale di righe, parametri o attributi.

5.3 Estrai Metodo

Estrai Metodo raggruppa un frammento in un nuovo metodo il cui nome ne comunica l'intenzione. È adatto a metodi lunghi, codice duplicato o blocchi che richiedono commenti per essere capiti. I metodi piccoli e ben nominati sono più riusabili; il metodo di alto livello diventa una sequenza leggibile di chiamate e l'override, cioè la ridefinizione di un metodo in una sottoclasse, risulta più agevole.

Il procedimento è il seguente:

1. individuare il frammento e scegliere un nome che descriva ciò che fa, non soltanto come lo fa;
2. copiare il frammento nel nuovo metodo;
3. trasformare in parametri le variabili locali del metodo sorgente che il frammento legge;
4. dichiarare nel nuovo metodo le variabili usate soltanto dal frammento;
5. se il frammento modifica una variabile locale del metodo sorgente, valutare se il nuovo metodo possa restituire il valore aggiornato come query;
6. sostituire il frammento nel metodo sorgente con la chiamata al nuovo metodo, quindi compilare e testare.

Una *query* restituisce un'informazione senza modificare lo stato osservabile dell'oggetto. Il passaggio 5 non è automatico: se non è possibile esprimere correttamente il risultato mediante un valore restituito, bisogna riesaminare il confine dell'estrazione.

La classificazione delle variabili precede lo spostamento del frammento. Se gli output sono molti, prima conviene separare le responsabilità: aggiungere campi alla classe soltanto per aggirare lo scope locale introduce stato non necessario. La lunghezza è un indizio: tagliare righe senza ottenere un'operazione coerente non è sufficiente.

5.3.1 Esempio: separare stampa, calcolo e dettagli

Si consideri `printDovuto`, che stampa un banner, calcola il totale degli ordini e stampa i dettagli. La classe Ordine di questo esempio espone l'importo come query; il costruttore permette di preparare una fixture.

```
import java.util.List;

class Ordine {
    private final double importo;
    Ordine(double importo) { this.importo = importo; }
    public double getImporto() { return importo; }
}
class Stampe {
    private final List<Ordine> ordini;
```

```
    private final String nome = "Mike";
    Stampe(List<Ordine> ordini) { this.ordini = ordini; }
    public void printDovuto() {
        System.out.println("----------------");
        System.out.println("- Cliente Dare -");
        System.out.println("----------------");
        double tot = 0;
        for (Ordine o : ordini) tot += o.getImporto();
        System.out.println("nome: " + nome);
        System.out.println("tot: " + tot);
    }
}
```

Dopo l'estrazione, la stessa sequenza si legge nei nomi dei metodi:

```
class Stampe2 {
    private final List<Ordine> ordini;
    private final String nome = "Mike";
    Stampe2(List<Ordine> ordini) { this.ordini = ordini; }
    public void printDovuto() {
        printBanner();
        double tot = getTotale();
        printDettagli(tot);
    }
    public double getTotale() {
        double tot = 0;
        for (Ordine o : ordini) tot += o.getImporto();
        return tot;
    }
    public void printBanner() {
        System.out.println("----------------");
        System.out.println("- Cliente Dare -");
        System.out.println("----------------");
    }
    public void printDettagli(double somma) {
        System.out.println("nome: " + nome);
        System.out.println("tot: " + somma);
    }
}
```

`tot` resta locale al calcolo e `somma` diventa parametro della stampa. Nessun campo temporaneo è stato aggiunto per far comunicare i metodi. Con gli stessi ordini si confronta l'intero testo prodotto dalle due classi: banner, nome e totale devono restare uguali.

5.3.2 Applicare l'estrazione a Pagamenti

Nel metodo monolitico di lettura degli importi si riconoscono blocchi con confini diversi: apertura e lettura del file, controllo dei duplicati, conversione, somma e massimo. Estrarre `inserisci` localizza la politica dei duplicati; estrarre `leggiFile` separa l'I/O; `converti` isola la trasformazione del testo. I cicli di somma e massimo diventano operazioni autonome, mentre i getter osservano i risultati. L'ordine delle dipendenze chiarisce quali dati devono già esistere a ciascun passo.

Il passaggio successivo alle query elimina `totale` e `massimo`: il client non deve più ricordare di ricalcolarli dopo ogni inserimento. Per dimostrare che una trasformazione è solo strutturale si confrontano risultati ed effetti sugli stessi input e sulla stessa sequenza ammessa dal contratto precedente. Il passaggio da deduplicazione testuale a numerica, la modifica della propagazione

delle eccezioni o la correzione del massimo per dati negativi cambiano invece il contratto o il comportamento in alcuni casi: vanno trattati come decisioni separate, con test specifici. La sola somiglianza dei risultati su tre importi non prova equivalenza per tutte le stringhe e tutti gli errori.

5.4 Sostituisci Temp con Query

Sostituisci Temp con Query si applica quando una variabile temporanea conserva il risultato di un'espressione. Una *variabile temporanea* è una variabile locale al metodo nel quale è dichiarata; perciò altri metodi non possono leggerla direttamente. Si estrae l'espressione in una query e si sostituiscono gli usi della variabile con chiamate alla query. Il dato diventa così disponibile anche ad altri metodi e ciò facilita una successiva estrazione di metodi.

La tecnica è appropriata quando la temporanea viene assegnata una sola volta. Si dichiara dapprima `final` per far controllare al compilatore che non vi siano riassegnazioni, si compila, si estrae la parte destra dell'assegnazione in un metodo e si sostituiscono tutte le occorrenze. Se la variabile cambia valore, una query non rappresenta più un valore fisso: si può dividere la variabile in valori distinti oppure non applicare questa tecnica. In particolare, una variabile di accumulo, quale un contatore o una somma in un ciclo, non va trasformata meccanicamente in query. Un eventuale costo di ricalcolo va ottimizzato solo se diventa un problema reale osservato.

La sostituzione richiede anche che rivalutare l'espressione conservi il significato: se i dati da cui dipende cambiano tra due utilizzi, una query può dare un risultato diverso dalla fotografia conservata nella temporanea. Va quindi controllata l'assenza di effetti indesiderati della rivalutazione.

L'esempio seguente sostituisce le temporanee `prezzoBase` e `sconto` con query. `quantita` è il numero di unità e `prezzo` il prezzo unitario; il prodotto `quantita * prezzo` è il prezzo base. Poiché entrambe le espressioni dipendono dallo stato dell'oggetto e non da una variabile locale che cambia, possono essere richiamate dal metodo pubblico.

```
public double getPrezzo() {
    return prezzoBase() * sconto();
}

private double prezzoBase() {
    return quantita * prezzo;
}

private double sconto() {
    if (prezzoBase() > 1000) return 0.95;
    return 0.98;
}
```

5.5 Dividi Variabile Temp

Dividi Variabile Temp riguarda invece una temporanea riassegnata per rappresentare valori concettualmente diversi, ma non una variabile aggiornata in un ciclo né un accumulatore. Si dà un nome e una variabile separata a ciascun significato: ciascuna responsabilità diventa leggibile e può essere dichiarata `final`. Si rinomina la prima dichiarazione e i suoi riferimenti fino alla successiva assegnazione, si introduce allora la nuova variabile, si compila e testa, e si ripete per passi piccoli.

Un diverso esempio mostra perché dividere una temporanea: la stessa variabile non deve prima indicare il perimetro e poi l'area. Con larghezza `width` e altezza `height`, i valori distinti

sono

$$p = 2(\texttt{width} + \texttt{height}), \qquad a = \texttt{width} \cdot \texttt{height}, \tag{5.2}$$

dove p è il perimetro e a l'area. Nel codice i nomi `perim` e `area`, dichiarati `final`, rendono visibili i due significati. L'eccezione resta l'accumulatore di un ciclo: lì la riassegnazione rappresenta intenzionalmente un unico valore progressivo.

5.5.1 Due fasi del moto e due accelerazioni

Un corpo di massa `mass` è soggetto inizialmente a `primaryForce`; dopo `delay` si aggiunge `secondaryForce`. Il metodo calcola la distanza dopo `time`. Nel dominio fisico considerato massa positiva, tempi non negativi e ritardo intero permettono di leggere direttamente le formule; i cast a `int` del listato troncano invece eventuali frazioni di tempo e fanno parte del suo comportamento.

```
private double primaryForce, secondaryForce, mass, delay;

public double getDistanceTravelled1(int time) {
    double acc = primaryForce / mass;
    int primaryTime = (int) Math.min(time, delay);
    double result = 0.5 * acc * primaryTime * primaryTime;
    int secondT = (int) (time - delay);
    if (secondT > 0) {
        double primaryVel = acc * delay;
        acc = (primaryForce + secondaryForce) / mass;
        result += primaryVel * secondT
                + 0.5 * acc * secondT * secondT;
    }
    return result;
}
```

`acc` indica prima l'accelerazione iniziale e poi quella della seconda fase. La prima è $a_1 = F_1/m$; la seconda $a_2 = (F_1 + F_2)/m$. Per tempi interi coerenti con il ritardo, posto $t_1 = \min(t, d)$ e $t_2 = t - d > 0$, la distanza è $s = \frac{1}{2}a_1 t_1^2$ prima del cambio; dopo il cambio si aggiunge $v_1 t_2 + \frac{1}{2}a_2 t_2^2$, con $v_1 = a_1 d$. La velocità iniziale della seconda fase non deve essere persa.

Si divide la temporanea senza cambiare gli altri calcoli:

```
public double getDistanceTravelled2(int time) {
    final double primAcc = primaryForce / mass;
    int primaryTime = (int) Math.min(time, delay);
    double result = 0.5 * primAcc * primaryTime * primaryTime;
    int secondT = (int) (time - delay);
    if (secondT > 0) {
        double primaryVel = primAcc * delay;
        final double secondAcc = (primaryForce + secondaryForce) / mass;
        result += primaryVel * secondT
                + 0.5 * secondAcc * secondT * secondT;
    }
    return result;
}
```

`primAcc` e `secondAcc` hanno significati separati e sono finali. `result` resta un accumulatore: le sue assegnazioni costruiscono intenzionalmente la stessa distanza e non richiedono di dividerlo. Test prima del ritardo, al ritardo e dopo il ritardo verificano che la trasformazione conservi entrambi i rami.

5.6 Sposta Metodo

Sposta Metodo si applica quando un metodo usa più attributi e operazioni di un'altra classe che della classe nella quale risiede: il comportamento mostra una dipendenza eccessiva verso la classe destinataria. Si esaminano prima attributi e metodi usati, valutando se quelli impiegati soltanto dal comportamento trasferito debbano seguirlo; si controlla inoltre che il metodo non sia dichiarato in superclassi o sottoclassi. Si dichiara e copia il metodo nella classe destinataria, si adattano i riferimenti alle due classi e si mantiene provvisoriamente il vecchio metodo come *delegato*, cioè un metodo che inoltra la chiamata al nuovo. Dopo compilazione e test si decide se rimuovere il delegato o conservarlo.

Il polimorfismo richiede cautela. Se il metodo è dichiarato anche nella gerarchia di ereditarietà e il comportamento polimorfico appartiene concettualmente alla classe destinataria, lo spostamento dell'intero comportamento diventa un refactoring più complesso. Se invece quel comportamento deve rimanere nella classe originaria, il metodo non va spostato integralmente; si può estrarre soltanto parte della logica.

5.7 Estrai Classe

Estrai Classe separa una classe che svolge il lavoro di due classi. Il Principio di Singola Responsabilità (SRP) richiede che una classe abbia un solo motivo per cambiare. Sono indizi un sottogruppo di dati e metodi che stanno insieme, dati che cambiano insieme o dipendono fra loro, oppure una sottoclasse che usa solo poche caratteristiche della superclasse. Si decide la responsabilità da separare, si crea una nuova classe, si inserisce nella classe originaria un attributo che ne conserva un'istanza e si spostano attributi e metodi, iniziando da quelli di più basso livello. La nuova relazione è una dipendenza fra le due classi.

Una classe `Persona` può contenere prefisso e numero di telefono, dati che costituiscono un concetto separabile. *Estrai Classe* crea, per esempio, `Telefono` e lascia a `Persona` un riferimento a quell'oggetto:

```
class Telefono {
    private String prefisso;
    private String numero;

    String formatoCompleto() { return prefisso + " " + numero; }
}

class Persona {
    private Telefono telefono;
}
```

L'esempio non afferma che ogni attributo debba essere estratto: la separazione è giustificata perché i dati e le operazioni sul recapito sono una responsabilità coerente che può cambiare indipendentemente da quelle della persona.

5.8 Oggetto Parametro e sostituzione dei condizionali

Una lunga lista di parametri correlati può essere ridotta mediante *Introduci Oggetto Parametro*. Si raggruppano dati che formano un concetto coerente: invece di ripetere data iniziale e finale in più firme, si può introdurre un valore `Periodo`, concentrando nel suo costruttore la validazione dell'ordine delle date. Un contenitore di parametri non correlati nasconde la lunga firma senza chiarire la responsabilità. Durante il refactoring si possono scoprire bug, ma correggerli cambia il comportamento: conviene distinguerne il passo, aggiornare l'oracle e proteggere la correzione con un caso di regressione.

Quando molti metodi ripetono lo stesso `switch` per distinguere gli stati di un oggetto, si può trasferire ogni comportamento in una classe di stato e delegare attraverso un contratto comune. La selezione passa così dal confronto su un codice al polimorfismo. La trasformazione è detta *Replace Conditional with Polymorphism*; il caso specifico *Replace Type Code with State* porta alla struttura del pattern State.

Capitolo 6

Java funzionale, lambda, collection e stream

6.1 Dallo stile imperativo allo stile dichiarativo

Java è un linguaggio imperativo: in questo stile il programmatore descrive esplicitamente i passi da eseguire, per esempio il ciclo che attraversa una lista e la variabile che accumula un risultato. A partire da Java 8 il linguaggio include caratteristiche della *programmazione funzionale*, in genere più concise, espressive e più facilmente parallelizzabili. Lo stile funzionale è dichiarativo: si esprime *che cosa* si vuole ottenere, delegando alla libreria i dettagli dell'iterazione. Non sostituisce la programmazione a oggetti, ma la affianca.

Per esempio, cercare un nome con un ciclo esterno richiede un flag booleano e la gestione dell'uscita dal ciclo; la stessa intenzione dichiarativa è espressa da `nomi.contains("Nobi")`. Nel secondo caso l'iterazione è *interna* all'implementazione della lista. Gli stream estendono questa idea: consentono di descrivere una successione di trasformazioni e selezioni sugli elementi.

Il confronto completo della ricerca è:

```
List<String> nomi = List.of("Nobita", "Nobi", "Suneo", "Honekawa",
    "Shizuka", "Minamoto", "Takeshi", "Gouda");
boolean trovato = false;
for (String nome : nomi) {
    if (nome.equals("Nobi")) { trovato = true; break; }
}
boolean interno = nomi.contains("Nobi");
```

Entrambe le variabili risultano vere. Nel ciclo il programmatore controlla il flag e l'uscita; `contains` concentra nella collezione la stessa richiesta. Questo è un confronto fra controllo dell'iterazione e intenzione dichiarata, non una promessa di differenza prestazionale.

Le principali tappe del linguaggio comprendono classi anonime da Java 1.1, enhanced for da Java 5, diamond `<>` da Java 7 e lambda e stream da Java 8. Il contesto è anche l'uso di processori con più core: esprimere trasformazioni e selezioni senza stato condiviso facilita la divisione del lavoro.

6.2 Lambda, interfacce funzionali e funzioni di ordine superiore

Una *classe anonima* è una classe priva di nome, definita e istanziata nel punto in cui compare una forma estesa dell'operatore `new`; può implementare un'interfaccia per un uso puntuale. Quando il comportamento richiesto è espresso da un solo metodo astratto, da Java 8 si può usare una *espressione lambda*, cioè una funzione anonima.

Data l'interfaccia `Hello`, con il solo metodo `greetings(String s)`, una lambda può fornire direttamente quel comportamento:

```
Sera sr = new Sera(s -> System.out.println("Ciao, " + s));
sr.saluti();
```

Il compilatore determina dal contesto che `s` è una `String`: questa determinazione automatica è detta *inferenza del tipo*. `Sera` riceve un collaboratore Hello e il suo metodo `saluti()` ne usa l'operazione di saluto. La lambda fornisce proprio tale comportamento al costruttore; non costituisce una sintassi generica per implementare interfacce con più metodi astratti.

La forma generale è

parametri `->` *corpo*. (6.1)

I *parametri*, a sinistra della freccia, sono gli ingressi; il *corpo*, a destra, calcola l'azione e l'eventuale valore restituito. Se il parametro è uno solo si possono omettere sia il suo tipo sia le parentesi; con nessun parametro o con più parametri le parentesi sono necessarie. Un corpo con più istruzioni richiede le parentesi graffe e un `return` esplicito quando deve restituire un valore:

```
s -> System.out.println("Ciao " + s)
(x, y) -> x + y
() -> System.out.println("Buongiorno")
(x, y) -> { System.out.println("x: " + x); return x + y; }
```

Nella seconda lambda, `x` e `y` sono gli ingressi e `x + y` è il risultato. Nei linguaggi funzionali puri una lambda è *pura* se il risultato dipende soltanto dagli ingressi e non possiede stato. In Java una lambda può accedere a stato esterno e quindi può non essere pura: tale possibilità richiede attenzione, soprattutto in parallelo.

Un'*interfaccia funzionale* è un'interfaccia con un unico metodo astratto. L'annotazione `@FunctionalInterface` chiede al compilatore di verificare questo vincolo e impedisce che modifiche successive aggiungano per errore un secondo metodo astratto. Una lambda viene interpretata come implementazione di tale unico metodo.

Una *funzione di ordine superiore* riceve una funzione, ne crea una oppure ne restituisce una. I metodi degli stream sono esempi importanti: ricevono lambda che specificano il comportamento da applicare agli elementi. In Java questi comportamenti sono rappresentati tramite interfacce funzionali: passarli o restituirli significa fornire un oggetto conforme al contratto atteso. La notazione funzionale si integra quindi con il sistema di tipi a oggetti.

Le interfacce funzionali ricorrenti distinguono predicati, trasformazioni, produttori di valori, consumatori e criteri di confronto. I parametri generici `T` e `R` indicano rispettivamente il tipo di ingresso e il tipo di risultato.

Tabella 6.1: Interfacce funzionali usate nelle pipeline.

Interfaccia	Metodo	Significato ed esempio
`Predicate<T>`	`test(T)`	Restituisce un booleano; `x -> x >= 0`.
`Function<T,R>`	`apply(T)`	Trasforma un `T` in un `R`; `x -> x.length()`.
`Supplier<T>`	`get()`	Non riceve parametri e fornisce un valore `T`; `() -> Math.random()`.
`Consumer<T>`	`accept(T)`	Riceve un elemento, compie un'azione e non restituisce valore; è usato da `peek()` e `forEach()`.
`Comparator<T>`	`compare(T,T)`	Stabilisce l'ordine fra due oggetti dello stesso tipo.

6.2.1 Lo stesso comportamento in tre forme

Un oggetto `Sera` riceve il comportamento da eseguire attraverso il contratto `Hello`. Una classe nominata lo realizza così:

```
interface Hello { void greetings(String s); }
class Sera {
    private final Hello myh;
    Sera(Hello h) { myh = h; }
    void saluti() { myh.greetings("buonasera"); }
}
class Saluti implements Hello {
    public void greetings(String s) {
        System.out.println("Ciao, " + s);
    }
}
class MainSal {
    public static void main(String[] args) {
        Sera sr = new Sera(new Saluti());
        sr.saluti();
    }
}
```

Il client dipende da `Sera`; `Sera` conserva `Hello`; `Saluti` implementa `Hello`. La classe anonima elimina il nome della concreta mantenendo lo stesso contratto:

```
Sera sr = new Sera(new Hello() {
    @Override public void greetings(String s) {
        System.out.println("Ciao, " + s);
    }
});
sr.saluti();
```

Con una sola operazione astratta basta la lambda:

```
Sera sr = new Sera(s -> System.out.println("Ciao, " + s));
sr.saluti();
```

Le tre alternative stampano `Ciao, buonasera`. Non si eliminano né il contratto né il collaboratore; si elimina il codice necessario a nominare e dichiarare la classe che realizza un comportamento semplice.

6.2.2 Riferimenti a metodi e a costruttori

La sintassi `::` denota un *riferimento a metodo*: indica un comportamento senza invocarlo immediatamente. `Integer::sum`, per esempio, corrisponde all'addizione `(a, b) -> a + b`; il metodo somma due valori primitivi `int`, con le eventuali conversioni boxing/unboxing richieste dall'interfaccia funzionale. `Persona::getEta` indica invece un metodo da invocare sulla persona ricevuta come argomento. Per gli esempi sulle persone si assumono nome, età e ruolo accessibili tramite `getNome()`, `getEta()` e `getRuolo()`.

Un riferimento può avere la forma `Classe::metodoStatico`, `istanza::metodo` oppure `Classe::metodoDiIstanza`; nell'ultimo caso il ricevente è fornito fra gli argomenti del comportamento funzionale. La notazione non invoca immediatamente il metodo e deve essere compatibile con il contratto funzionale atteso. Un Supplier conserva un comportamento da invocare con `get()`, non necessariamente un valore già calcolato: può leggere uno stato aggiornato al momento della chiamata.

`Function<T,R>` definisce `apply(T)`. La funzione `stringLength = x -> x.length()`, con tipo `Function<String,Integer>`, riceve una stringa e ne restituisce la lunghezza: `stringLength.apply("truth")` vale cinque.

Un `Supplier<T>` definisce `get()`, non riceve argomenti e restituisce un `T`: per esempio `Supplier<String> sup = () -> "Ciao ciao";` e `sup.get()` restituisce la stringa. Un riferimento a costruttore, quale `ProdottoA::new`, può essere un `Supplier<Prodotto>`; una mappa da nomi a supplier permette quindi di ritardare la creazione fino alla chiamata a `get()`. Se il costruttore richiede parametri, il solo `Supplier` non è appropriato, perché non ne accetta.

6.3 Collection: strutture dati, costi e origine dello stream

La gerarchia delle *collection* offre strutture dati e operazioni quali `add()`, `remove()`, `size()`, `contains()` e `containsAll()`. `Collection` è l'interfaccia generale; `List` aggiunge, fra gli altri, `get()`, `indexOf()` e `set()`.

- `ArrayList` è un array espandibile: i riferimenti agli elementi sono memorizzati nell'array interno e l'accesso per indice è veloce; un incremento del 50% quando manca spazio è un possibile dettaglio implementativo, non una garanzia del contratto List.
- `LinkedList` è una lista i cui elementi mantengono riferimenti al precedente e al successivo.
- `Vector` è simile a `ArrayList`, ma sincronizza le singole operazioni, con un costo prestazionale; una sequenza di chiamate non diventa per questo automaticamente atomica.
- `TreeSet` mantiene elementi ordinati e offre operazioni principali con costo logaritmico.

La notazione $O(f(n))$ descrive un limite all'ordine di crescita del costo al crescere degli elementi: $O(1)$ indica costo costante, $O(n)$ lineare e $O(\lg n)$ logaritmico, dove $\lg n$ è il logaritmo di n. Per confrontare le strutture bisogna anche specificare quale operazione si esegue e se la posizione interessata è già nota.

Tabella 6.2: Complessità delle collection: inserimento in coda; * indica posizione già individuata.

Struttura	`get`	`add`	`contains`	`remove`
`ArrayList`	$O(1)$	$O(1)$ amm.	$O(n)$	$O(n)$
`LinkedList`	$O(n)$	$O(1)$*	$O(n)$	$O(1)$*
`TreeSet`	—	$O(\lg n)$	$O(\lg n)$	—

L'aggiunta in coda ad ArrayList ha costo costante *ammortizzato*: alcune aggiunte richiedono di riallocare e copiare l'array, ma il costo medio sulla sequenza di aggiunte resta costante. Inserire o rimuovere nel mezzo richiede spostamenti. In LinkedList il collegamento o scollegamento è costante quando il nodo è già individuato; raggiungerlo per indice o cercarne il valore costa invece $O(n)$.

Java 8 ha introdotto i *metodi di default* nelle interfacce per aggiungere metodi a interfacce esistenti senza rompere la compatibilità con applicazioni precedenti. `stream()` è un metodo di default di `Collection`: su una collection di elementi `T` restituisce `Stream<T>`, una sequenza di elementi `T` su cui applicare operazioni. `List.of(...)` (da Java 9) crea invece una lista non modificabile; `Stream.of(...)` crea direttamente uno stream.

	get	add	contains	remove
ArrayList	costante	costante	O(n)	O(n)
LinkedList	O(n)	costante	O(n)	costante
TreeSet		O(lg n)	O(lg n)	

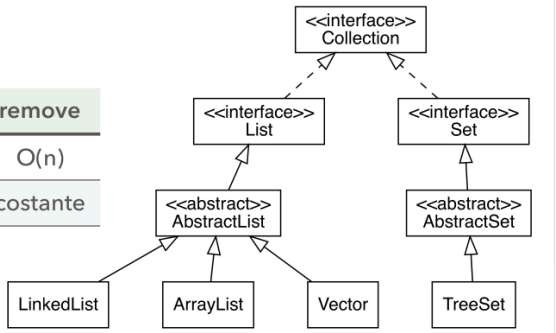


Figura 6.1: Gerarchia Collection/List/Set e confronto dei costi delle operazioni nelle condizioni specificate.

6.4 Pipeline: sorgente, passaggi lazy e operazione terminale

Uno *stream* non è una collection né un contenitore che memorizza dati: è un flusso di elementi provenienti da un'origine, descritto utilmente come un nastro trasportatore. Una *pipeline* è la catena formata dall'origine, da zero o più operazioni intermedie e da un'operazione terminale. Le trasformazioni non modificano la collection di origine ma producono stream distinti.

Un'operazione che restituisce uno stream è *intermedia*; è normalmente *lazy*, cioè la sua elaborazione è rinviata. Un'operazione *terminale* produce il valore finale o esegue l'azione finale, consuma lo stream ed è *eager*: forza l'esecuzione delle operazioni precedenti. Senza terminale una pipeline lazy non viene valutata. Per esempio:

```
List<String> nomi = List.of("Nobita", "Nobi", "Suneo");
long c = nomi.stream()
.filter(s -> s.equals("Nobi"))
.count();
```

`filter` seleziona gli elementi per cui il predicato è vero e restituisce uno stream; `count` è terminale e restituisce il numero `long` di elementi selezionati. La pipeline equivale ai passaggi espliciti `s1 = nomi.stream()`, `s2 = s1.filter(...)` e `c = s2.count()`.

Concatenare due `filter` non richiede di costruire prima una lista per il primo e poi una seconda lista per il secondo. In una pipeline, un elemento attraversa i filtri finché viene scartato o raggiunge il terminale. La valutazione lazy permette di fondere il lavoro di attraversamento e di evitare materializzazioni inutili; non è necessario riscrivere i predicati come un unico AND perché questa proprietà valga. È la libreria a organizzare il passaggio degli elementi; la valutazione lazy non garantisce una particolare riscrittura del compilatore.

Uno stream può essere consumato una sola volta; per una nuova elaborazione si chiede un nuovo stream alla sorgente. `map` ha una corrispondenza uno-a-uno fra elementi attraversati in ingresso e in uscita, pur potendo cambiare il tipo; `filter` seleziona zero oppure un elemento per ciascun ingresso. Non si modificano automaticamente gli elementi o la collezione di origine, ma una lambda che modifica un oggetto mutabile può comunque produrre effetti osservabili: tale effetto dipende dal codice passato all'operazione.

6.5 Filter e map: selezione e trasformazione

`filter(Predicate<T> p)` riceve un predicato `p`: per ciascun elemento `T`, il risultato di `p.test(...)` è un booleano: se vale `true`, l'elemento prosegue; altrimenti è scartato. Per contare i nomi di cinque caratteri si scrive `nomi.stream().filter(s -> s.length() == 5).count()`; per le stringhe vuote, `filter(s -> s.isEmpty())`.

`map(Function<T,R> mapper)` applica la funzione `mapper` a ogni elemento di tipo `T` e restituisce uno `Stream<R>`. Per esempio `map(x -> x * 2)` raddoppia gli interi; `map(Persona::getEta)` trasforma uno stream di `Persona` in uno stream delle rispettive età. Entrambe sono operazioni intermedie.

6.5.1 Dati completi per trasformazioni e ricerche

Per gli esempi sulle età si usa una classe con costruttore e getter:

```
class Persona {
    private final String nome;
    private final int eta;
    Persona(String nome, int eta) { this.nome = nome; this.eta = eta; }
    public String getNome() { return nome; }
    public int getEta() { return eta; }
    public static Persona getMax(Persona p1, Persona p2) {
        return p1.getEta() > p2.getEta() ? p1 : p2;
    }
}
List<Persona> amici = List.of(new Persona("Saro", 24),
    new Persona("Taro", 21), new Persona("Ian", 19), new Persona("Al", 16));
```

Due filtri concatenati `filter(p -> p.getNome().equals("Taro"))` e `filter(p -> p.getEta()==21)` selezionano Taro. Cambiare l'età di Al da 16 a 21 in un'altra fixture modifica i dati, pur lasciando invariato questo risultato. Una ricerca su sole stringhe, come `nomi.contains("Saro")`, controlla invece soltanto il nome e non una proprietà della persona.

Un predicato dichiarato separatamente può essere applicato a una sorgente concreta:

```
Predicate<Integer> positive = x -> x >= 0;
List<Integer> nonNegativi = Stream.of(2, 5, 10, -1)
    .filter(positive).toList(); // [2, 5, 10]
List<Integer> eta = new ArrayList<>();
for (Persona p : amici) eta.add(p.getEta());
```

Il nome `positive` non cambia la condizione: qui è incluso anche zero. L'ultimo ciclo costruisce la stessa sequenza di età di `map(Persona::getEta).collect(Collectors.toList())`, cioè 24, 21, 19, 16. `Stream` è un'interfaccia: il client ne usa il contratto senza nominare le classi interne che realizzano filtri, trasformazioni e riduzioni.

La selezione di valori maggiori di cinque può produrre risultati diversi senza cambiare il predicato: `numeri.stream().filter(n -> n>5).count()` restituisce un conteggio, mentre `numeri.stream().filter(n -> n>5).toList()` restituisce i valori selezionati. Il terminale determina il tipo di risultato e richiede uno stream nuovo per ogni elaborazione.

6.6 Collect e toList: materializzare il risultato

Il metodo `collect()` raccoglie gli elementi finali secondo un `Collector`. Il collector `Collectors.toList()` li accumula in una `List`. Da Java 16 è disponibile anche `Stream.toList()`. Un esempio completo ricava le età:

```
List<Integer> eta = amici.stream()
.map(Persona::getEta)
.collect(Collectors.toList());
```

`Stream.toList()` restituisce una lista non modificabile. `Collectors.toList()` non promette un'implementazione concreta né un contratto di mutabilità specifico. Per ottenere un ArrayList modificabile si specifica il contenitore:

```
.collect(Collectors.toCollection(ArrayList::new))
```

Gli elementi contenuti non diventano per questo immutabili.

6.7 Reduce: accumulazione, identità e associatività

Una *riduzione* combina molti elementi in un solo valore. La forma `reduce(T identity, BinaryOperator<T> accumulator)` riceve un'identità `identity`, dello stesso tipo degli elementi, e un accumulatore binario. Formalmente, dato lo stream finito $v_1, \ldots, v_n$, con $a_0 = I$, dove I è l'identità, e con g accumulatore, si calcola

$$a_i = g(a_{i-1}, v_i) \quad (1 \le i \le n), \qquad \text{risultato} = a_n. \tag{6.2}$$

Qui v_i è l'i-esimo elemento e a_i il valore accumulato dopo averlo elaborato. Per la somma, $I = 0$ e $g(a, v) = a + v$: `reduce(0, (accum, v) -> accum + v)`. Se lo stream è vuoto, questa variante restituisce l'identità, quindi 0 nell'esempio. `reduce` è terminale.

La variante `reduce(BinaryOperator<T>)` non riceve identità e restituisce `Optional<T>`, perché non esiste un risultato da estrarre quando lo stream è vuoto. `Optional<T>` rappresenta un valore di tipo `T` che può essere presente oppure assente; si può usare `isPresent()` per verificare la presenza e `get()` per estrarre il valore solo nel ramo in cui è presente.

La formula della riduzione precedente descrive l'accumulazione sequenziale. In parallelo, gruppi di elementi possono essere combinati secondo un altro raggruppamento: l'operatore deve essere associativo e l'identità deve essere neutra. Per la somma, $0 + x = x$; scegliere 10 come presunta identità può aggiungere più volte quel valore quando si combinano partizioni. La sottrazione non è associativa: $(a - b) - c$ e $a - (b - c)$ possono differire.

6.7.1 Esempi di riduzione

`Function<T,R>` definisce `apply(T)`: riceve un oggetto di tipo `T` e restituisce un risultato di tipo `R`. Nell'esempio seguente `stringLength` calcola la lunghezza di una stringa; passato a `map`, consente di sommare le lunghezze e di ottenere 31.

```
Function<String, Integer> stringLength = x -> x.length();
int totale = Stream.of("truth", "flows", "to", "them",
                       "sweetly", "by", "nature")
.map(stringLength).reduce(0, Integer::sum);
```

6.7.2 Esempio: somma delle età

Per una lista `amici` di `Persona`, la somma delle età è una trasformazione seguita da una riduzione:

```
int somma = amici.stream()
.map(Persona::getEta)
.reduce(0, Integer::sum);
```

Il primo passaggio produce le età, il secondo le combina a partire dall'identità zero.

Con la fixture delle quattro persone, le tre forme equivalenti della somma sono:

```
int sommaImperativa = 0;
for (Persona p : amici) sommaImperativa += p.getEta();
int sommaLambda = amici.stream().map(p -> p.getEta())
    .reduce(0, (acc, eta) -> acc + eta);
int sommaMetodo = amici.stream().map(Persona::getEta)
    .reduce(0, Integer::sum);
```

Tutte restituiscono 80. Il primo stile controlla l'iterazione e aggiorna una variabile; gli altri descrivono trasformazione e combinazione. Per gli importi `List<Float> valori`, lo stesso confronto dà:

```
float totale = 0;
for (float v : valori) totale += v;
float ridotto = valori.stream().reduce(0f, Float::sum);
```

Sono due corpi alternativi di `calcolaSommaImper` e `calcolaSomma`. La virgola mobile può rendere significativo il raggruppamento delle somme: non si deduce l'identità bit per bit fra qualsiasi esecuzione parallela e sequenziale.

6.8 Comparator, max e sorted: confronto e ordine

Un `Comparator<T>` definisce `compare(o1,o2)` per ordinare oggetti: restituisce 0 se `o1` e `o2` sono equivalenti secondo il criterio scelto, un valore positivo se `o1` è maggiore di `o2`, negativo nel caso opposto. `Comparator.comparing(Persona::getEta)` costruisce il criterio estraendo l'età come chiave. `max(comparator)` è terminale, seleziona il massimo e restituisce un `Optional`; `sorted(comparator)` è intermedia e restituisce gli elementi ordinati.

Un Comparator restituisce zero quando i due elementi sono equivalenti rispetto al criterio, anche se non sono lo stesso riferimento e non sono uguali secondo `equals`. Se si confrontano le età con `p1.getEta()-p2.getEta()`, per 10 e 25 il risultato è -15, coerentemente con il fatto che dieci precede venticinque in ordine crescente. Per evitare anche i problemi di overflow di una sottrazione fra interi generici, si può scrivere:

```
Comparator<Persona> perEta =
    Comparator.comparingInt(Persona::getEta);
```

`max(perEta)` restituisce la Persona massima secondo quel criterio, non direttamente l'età; per ottenere il valore si trasforma prima lo stream o si legge la Persona risultante dall'Optional.

```
Optional<Persona> pmax = amici.stream()
.filter(p -> p.getEta() < 20)
.max(Comparator.comparing(Persona::getEta));
if (pmax.isPresent())
    System.out.println(pmax.get().getNome());
```

La versione imperativa del massimo sotto i vent'anni mantiene un candidato inizialmente assente:

```
Persona candidato = null;
for (Persona p : amici) {
    if (p.getEta() < 20
            && (candidato == null || p.getEta() > candidato.getEta()))
        candidato = p;
}
if (candidato != null) System.out.println(candidato.getNome());
Optional<Persona> conReduce = amici.stream()
    .filter(p -> p.getEta() < 20).reduce(Persona::getMax);
conReduce.ifPresent(p -> System.out.println(p.getNome()));
```

La riduzione senza identità rappresenta l'assenza con `Optional`; il metodo statico `getMax` sceglie fra due persone. Con Ian 19 e Al 16 il risultato è Ian; se nessuno passa il filtro, non si stampa alcun nome. `max(Comparator)` esprime direttamente lo stesso obiettivo. In caso di pari età occorre distinguere massimo rispetto al criterio e identità della persona selezionata: una politica specifica di spareggio richiede un criterio ulteriore.

Il metodo di ordinamento di Stream si chiama `sorted()`, mentre `sort()` appartiene ad altri contesti, per esempio alla lista mutabile.

6.8.1 Esempio: selezione e ordinamento delle persone

Per il team si usa una definizione alternativa del tipo Persona: un record con nome, età e ruolo. Gli accessori diventano `nome()`, `eta()` e `ruolo()`, senza prefisso `get`.

```
record Persona(String nome, int eta, String ruolo) { }
List<Persona> team = List.of(new Persona("Kent",29,"CTO"),
    new Persona("Luis",25,"Programmer"), new Persona("Al",25,"Programmer"),
    new Persona("Andrea",26,"GroupLeader"), new Persona("Joe",26,"Programmer"));
team.stream().filter(p -> p.eta() < 26)
    .filter(p -> p.ruolo().equals("Programmer"))
    .sorted(Comparator.comparing(Persona::nome))
    .forEach(p -> System.out.print(p.nome() + " "));
```

L'output è `Al Luis`: Joe ha 26 anni e viene escluso dalla condizione stretta. I due filtri e l'ordinamento sono intermedi; `forEach` è il terminale.

6.9 Distinct, findFirst, findAny e limit

`distinct()` è un'operazione intermedia che elimina i duplicati secondo l'uguaglianza degli elementi. `findAny()` e `findFirst()` sono terminali che restituiscono `Optional`. Sono operazioni di *short-circuiting*: per restituire un elemento non devono necessariamente esaminare l'intero stream. Anche `limit(k)` è intermedia e tronca lo stream a k elementi, dove k è il limite richiesto.

`findFirst()` rispetta l'ordine d'incontro quando esiste; `findAny()` può scegliere un elemento qualsiasi. La differenza è rilevante in parallelo, ma non va intesa come promessa che una specifica istanza di thread vinca sempre. `limit(1)` restituisce ancora uno stream, non l'elemento né un Optional; occorre un terminale. Inoltre `limit(k)` pone un massimo, non garantisce k risultati se a monte ce ne sono meno.

6.9.1 Esempio: ruoli distinti

I ruoli diversi sono ottenuti proiettando ogni persona sul ruolo e rimuovendo i duplicati:

```
team.stream().map(Persona::ruolo).distinct()
.forEach(r -> System.out.print(r + " "));
```

Con il team appena definito si stampa `CTO Programmer GroupLeader`, nell'ordine del primo incontro dei ruoli. La ricerca di un Programmer può essere espressa come:

```
Optional<Persona> qualsiasi = team.stream()
    .filter(p -> p.ruolo().equals("Programmer")).findAny();
qualsiasi.ifPresent(p -> System.out.println(p.nome()));
```

Luis, Al e Joe sono tutti risultati ammessi; `findAny` non ne impone uno fisso. `findFirst` sullo stesso stream ordinato per incontro seleziona Luis. Per una ricerca con due requisiti diversi, un tipo `PersonaAlta(String nome,int altezza)` permette `filter(p -> p.nome().equals("Mario"))`, seguito da `filter(p -> p.altezza()>=170)` e da `findAny()` o `findFirst()`: l'Optional è vuoto se non esiste una persona che soddisfa entrambe le condizioni.

6.10 Operazioni stateless e stateful

Un'operazione intermedia *stateless* può elaborare ogni elemento senza conservare informazioni sugli altri: `map` e `filter` ne sono esempi. Un'operazione intermedia *stateful* richiede invece

informazioni sull'insieme degli elementi: `sorted` deve ordinarli e `distinct` deve riconoscere quelli già incontrati. Lo stato necessario può quindi crescere con la dimensione dello stream.

Le riduzioni terminali hanno comunque un accumulatore. Per somma (`reduce`) e massimo (`max`), il risultato parziale può avere dimensione costante rispetto al numero di elementi; una riduzione che costruisce una struttura crescente può richiedere sempre più memoria. Conservare un accumulatore e classificare un'operazione intermedia come stateful sono quindi aspetti da tenere distinti.

6.11 Generare sequenze con iterate e generate

`Stream.iterate(seme, f)` genera uno stream infinito ordinato: produce il `seme`, poi f(`seme`), poi $f(f$(`seme`)), e così via. Ogni valore dipende dal precedente; la generazione presenta quindi una dipendenza sequenziale, anche se la pipeline può essere configurata come parallela. Occorre limitarlo quando si desidera un risultato finito:

```
Stream.iterate(2, n -> n * 2)
.limit(10)
.forEach(System.out::println); // 2, 4, 8, ..., 1024
```

`Stream.generate(supplier)` genera uno stream infinito: il Supplier fornisce ogni valore senza ricevere quello precedente. Per esempio, `Stream.generate(Math::random).limit(5)` produce cinque `double` casuali maggiori o uguali a zero e minori di uno. Un generatore che arrotonda il valore scalato può essere definito così:

```
Supplier<Long> intero = () -> Math.round(Math.random() * 10);
```

I valori ottenuti sono interi di tipo `long` fra zero e dieci, estremi inclusi.

Per uno stream infinito, la posizione del limite conta:

```
Stream.generate(() -> 0)
.limit(10).filter(n -> n < 0).count(); // termina: 0
// Con filter(n -> n < 0) prima di limit(10),
// non arriverebbe mai un elemento al limite.
```

Una generazione illimitata non garantisce che il terminale finisca soltanto perché esiste un `limit` dopo un filtro che non produce risultati.

6.12 Debug della pipeline con peek

`forEach` è terminale: esegue un'azione per ogni elemento e non permette di proseguire la pipeline. Per osservare i valori intermedi in fase di debug si usa `peek(Consumer<T>)`: riceve un elemento, compie per esempio una stampa, non ne modifica il passaggio e restituisce lo stream. È lazy: senza un terminale non viene eseguito.

```
List<Integer> numbers = List.of(2, 3, 4, 5);
List<Integer> risultato = numbers.stream().map(x -> x + 17)
    .peek(x -> System.out.println("after map: " + x))
    .filter(x -> x % 2 == 0)
    .peek(x -> System.out.println("after filter: " + x))
    .limit(3)
    .peek(x -> System.out.println("after limit: " + x)).toList();
```

La traccia sequenziale è `after map: 19`; poi, per 20, `after map`, `after filter` e `after limit`; poi `after map: 21`; infine i tre passaggi per 22. Il risultato è `[20,22]`; il limite 3 non

crea un terzo valore. Il processamento procede elemento per elemento attraverso la pipeline, non completando prima un'intera operazione su tutti gli elementi.

`peek()` è destinato all'osservazione durante il debug. Il corto circuito e le ottimizzazioni possono far sì che non tutti gli elementi lo attraversino: non vi si collocano aggiornamenti indispensabili alla correttezza. Per osservare più fasi si colloca un peek dopo ciascuna trasformazione di interesse, conservando l'assenza di effetti sullo stato applicativo.

6.13 Stream primitivi e conversioni

`IntStream` è uno stream del tipo primitivo `int`. `IntStream.rangeClosed(a,b)` genera gli interi da a a b, estremi inclusi, con incremento uno. Perciò il calcolo seguente restituisce $1+2+3+4+5=15$:

```
int somma = IntStream.rangeClosed(1, 5).sum();
```

`mapToInt` trasforma uno stream generico in `IntStream`; `boxed` esegue la conversione opposta, da `IntStream` a `Stream<Integer>`; `mapToObj` crea uno stream di oggetti a partire da un `IntStream`.

Le specializzazioni standard sono `IntStream`, `LongStream` e `DoubleStream`; non esiste `FloatStream` nell'API standard. `IntStream` non estende `Stream<Integer>`: entrambi fanno parte della famiglia degli stream, ma hanno operazioni e tipi diversi. `range(a,b)` esclude il limite superiore, `rangeClosed(a,b)` lo include.

Tre conversioni concrete collegano i contratti:

```
IntStream.rangeClosed(1,6).map(x -> x*x).forEach(System.out::println);
int lunghezze = Stream.of("truth","flows","to","them","sweetly","by","nature")
    .mapToInt(String::length).sum(); // 31
List<Integer> dieci = IntStream.rangeClosed(1,10).boxed().toList();
List<Persona> primiQuattro = IntStream.rangeClosed(0,3)
    .mapToObj(i -> amici.get(i)).toList();
```

Si stampano i quadrati 1, 4, 9, 16, 25, 36. La selezione per indice richiede che `amici` contenga almeno quattro elementi; generare indici non controlla automaticamente il limite della lista.

6.13.1 Esempio: verificare l'ordinamento di una lista

Per verificare se una lista di interi è ordinata si generano gli indici delle coppie contigue, si cerca una violazione e si nega la sua presenza:

```
boolean ordinata = !IntStream.rangeClosed(0, lista.size() - 2)
.filter(i -> lista.get(i) > lista.get(i + 1))
.findAny().isPresent();
```

L'indice `i` varia da 0 a `lista.size()` $-\,2$, così `i+1` resta un indice valido. `findAny` può fermarsi alla prima coppia non ordinata.

6.13.2 Esempio: terne contigue che formano triangoli

Data una lista di numeri interi positivi, una terna (a,b,c) forma un triangolo se ciascun lato è minore della somma degli altri due:

$$a < b+c \ \wedge \ b < a+c \ \wedge \ c < a+b, \tag{6.3}$$

dove a, b e c sono le tre lunghezze positive. Per le terne contigue di una lista si generano gli indici iniziali, si costruisce la terna e si applicano le tre selezioni:

```
IntStream.rangeClosed(0, lista.size() - 3)
.mapToObj(i -> new int[]{lista.get(i), lista.get(i+1), lista.get(i+2)})
.filter(t -> t[0] < (long) t[1] + t[2])
.filter(t -> t[1] < (long) t[0] + t[2])
.filter(t -> t[2] < (long) t[1] + t[0])
.forEach(t -> System.out.println(t[0] + " " + t[1] + " " + t[2]));
```

La promozione a `long` prima della somma evita overflow nell'addizione di due interi positivi. La lista $\{2, 3, 5, 7, 8\}$ produce le terne valide `3 5 7` e `5 7 8`. L'uso di `rangeClosed` fino a `size()-3` assicura che gli accessi `i`, `i+1`, `i+2` esistano.

6.14 Esercizio svolto: buste paga e raggruppamento per ruolo

Si deve calcolare il pagamento mensile di sei lavoratori, il totale e un elenco per ruolo. La tariffa dipende dal ruolo e viene moltiplicata per 25; in mancanza di una tariffa associata si usa 50. I record appartengono a questo esempio e usano accessori senza prefisso `get`:

```
import java.util.*;
import static java.util.Comparator.comparing;

record Persona(String nome, int eta, String ruolo) { }
record BustaPaga(Persona dipendente, int totale) { }
class ListaPersone {
    private final List<Persona> team = List.of(
        new Persona("K Gray", 29, "CTO"),
        new Persona("L Rose", 25, "Programmer"),
        new Persona("Al White", 25, "Programmer"),
        new Persona("Joe Black", 26, "Programmer"),
        new Persona("Jill Purple", 26, "Tester"),
        new Persona("Bill Green", 26, "Analyst"));
    private final Map<String, Integer> roleCost = Map.of(
        "Programmer", 50, "CTO", 65, "Tester", 56, "Analyst", 54);
    private List<BustaPaga> buste = List.of();
    public void genListaPagamenti() {
        buste = team.stream()
            .map(p -> new BustaPaga(p,
                roleCost.getOrDefault(p.ruolo(), 50) * 25))
            .sorted(comparing(b -> b.dipendente().nome())).toList();
    }
    public int calcolaSommaBuste() {
        return buste.stream().map(BustaPaga::totale)
            .reduce(0, Integer::sum);
    }
    public void stampaBuste() {
        buste.forEach(b -> System.out.println(
            b.dipendente().nome() + "\t" + b.totale()));
    }
    public void stampaRuoliPersone() {
        roleCost.keySet().stream().sorted()
            .peek(System.out::println)
            .forEach(ruolo -> team.stream()
                .filter(p -> p.ruolo().equals(ruolo))
                .sorted(comparing(Persona::nome))
                .forEach(p -> System.out.println("  " + p.nome())));
    }
}
```

Prima di sommare o stampare le buste si chiama `genListaPagamenti`. `map` produce un nuovo record per lavoratore; `sorted` ordina per nome della persona contenuta nel record e `toList` materializza il risultato. La riduzione successiva somma soltanto i totali. Si ottiene:

Dipendente	Pagamento
Al White	1250
Bill Green	1350
Jill Purple	1400
Joe Black	1250
K Gray	1625
L Rose	1250
Totale	8125

`stampaRuoliPersone` visita le chiavi delle tariffe in ordine alfabetico: Analyst, CTO, Programmer, Tester. Per ciascuna stampa l'intestazione e avvia una seconda pipeline che seleziona e ordina i lavoratori di quel ruolo. Il `peek` è qui una traccia del flusso per gruppi; se l'intestazione è un effetto richiesto dall'applicazione è più chiaro stamparla nel corpo del `forEach`, insieme alla pipeline interna. Un ruolo non presente nella mappa riceve comunque tariffa 50 nel calcolo, ma non compare in questa stampa basata sulle sole chiavi della mappa: per elencare tutti i ruoli effettivi la sorgente deve derivare dai ruoli del team con `distinct()`.

6.15 Stream paralleli: condizioni di correttezza e prestazioni

La programmazione parallela sfrutta più core ma è più complessa di quella sequenziale: con `Thread` occorre in generale gestire condizioni di corsa e strumenti quali `synchronized`, `wait`, `notify`, lock ed esecutori. `Collection.parallelStream()` può invece fornire uno stream parallelo, che ripartisce il lavoro in blocchi; `stream()` produce la variante sequenziale.

Le operazioni senza effetti sullo stato condiviso e l'assenza di modifiche alla sorgente facilitano la parallelizzazione. Una lambda che aggiorna uno stato globale richiede invece coordinamento e può compromettere correttezza e prestazioni. Su milioni di elementi e hardware multicore, sorgenti come ArrayList, Vector e TreeSet possono beneficiare della divisione del lavoro; LinkedList può offrire vantaggi inferiori, perché la rappresentazione concatenata è meno agevole da dividere in parti indipendenti. Il risultato dipende dall'operazione e dalla macchina, e `forEach` parallelo non garantisce l'ordine di esecuzione.

```
long c = nomi.parallelStream()
.map(String::toUpperCase)
.filter(s -> s.equals("NOBI"))
.count();
```

Il lavoro parallelo comprende suddivisione, coordinamento e ricomposizione: per pochi elementi l'overhead può superare il guadagno. Un cambiamento del JDK può modificare le prestazioni senza cambiare l'API, ma non garantisce sempre un miglioramento. Una sorgente facilmente partizionabile e operazioni senza stato condiviso aiutano; ordinamento, duplicati e ordine richiesto possono aumentare il costo.

6.15.1 Leggere un confronto sperimentale

Due esperimenti confrontano ricerca su cinque milioni di elementi con ciclo esterno, stream sequenziale e stream parallelo, usando ArrayList, Vector, LinkedList e TreeSet. Il primo

impiega hardware a quattro core e Java 11.0.2; il secondo hardware a otto core e Java 18.0.1. I grafici inferiori mostrano anche il costo di riempimento delle strutture.

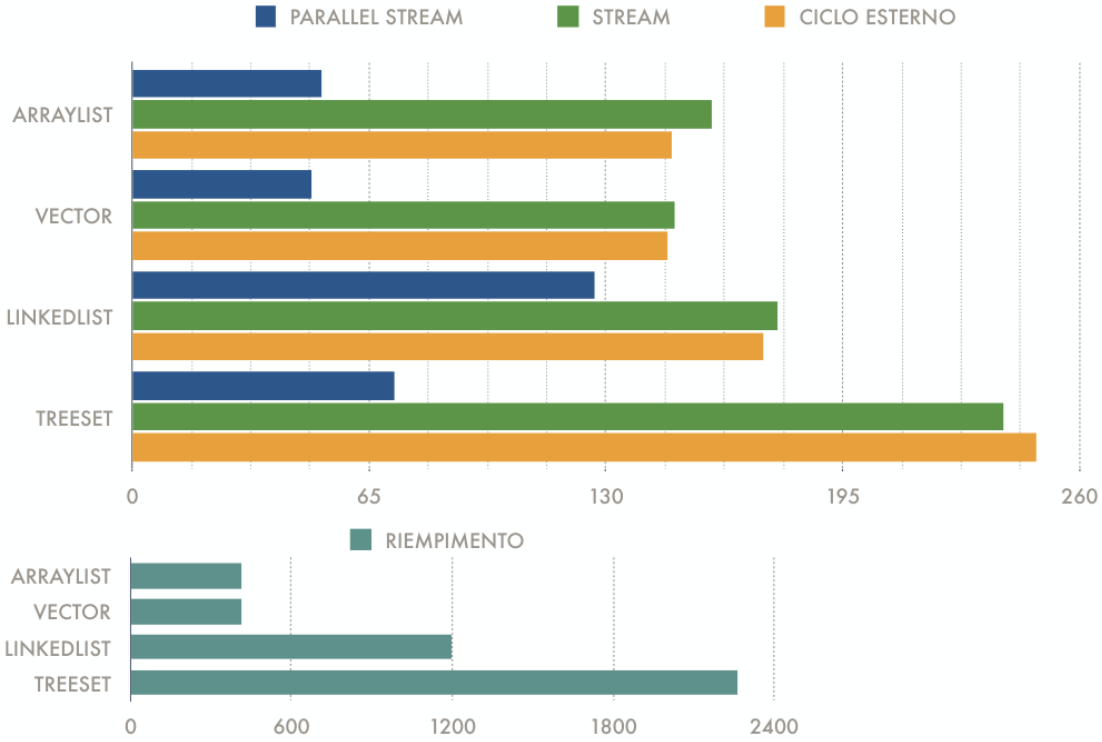


Figura 6.2: Ricerca su cinque milioni di elementi e riempimento: quattro core, Java 11.0.2.

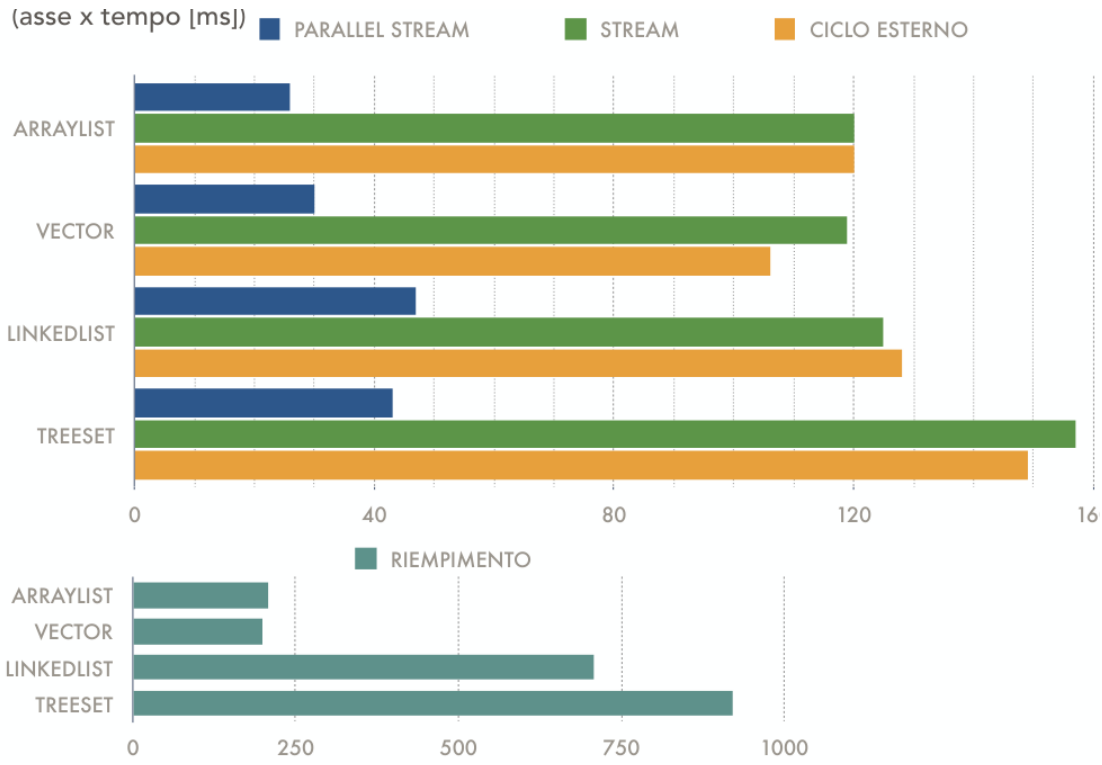


Figura 6.3: Ricerca su cinque milioni di elementi e riempimento: otto core, Java 18.0.1; tempi in millisecondi.

Nel primo confronto il parallelo riduce sensibilmente il tempo di ricerca su ArrayList, Vector e TreeSet, con un beneficio minore su LinkedList. Nel secondo il parallelo resta il più rapido nei casi illustrati; i tempi di riempimento delle strutture restano diversi. Le barre consentono un confronto qualitativo e una lettura approssimativa dell'asse, non misure esatte dei singoli casi. Cambiano insieme hardware e JDK: non si può attribuire la differenza fra i due esperimenti a una sola di queste variabili, né trasformarla in una garanzia per ogni pipeline.

Capitolo 7

Design pattern: principi e pattern creazionali

7.1 Design pattern: problemi ricorrenti, ruoli e conseguenze

Un *design pattern* è una micro-architettura: una struttura software, in genere formata da poche classi e oggetti, che descrive una soluzione collaudata, generica ma specializzabile, a un problema di progettazione ricorrente in un dato contesto. Non è una libreria né codice da copiare: identifica ruoli, responsabilità e collaborazioni che devono essere adattati al problema. Il suo scopo riguarda soprattutto le proprietà non funzionali di una soluzione—per esempio riuso, modificabilità, testabilità, affidabilità e sicurezza—oltre a ciò che il programma fa.

I pattern danno un vocabolario comune al gruppo di sviluppo e consentono di riusare soluzioni anziché reinventarle. Non sono però una garanzia automatica di qualità né vanno applicati senza un problema che ne giustifichi il costo: introducono classi, indirezioni e responsabilità ulteriori. Esistono cataloghi per contesti diversi (sistemi centralizzati, concorrenti, distribuiti e real-time); la progettazione orientata agli oggetti ne costituisce uno degli ambiti principali.

Per comprendere e applicare un pattern occorre precisarne cinque aspetti.

Nome identifica la soluzione e ne richiama lo scopo a un livello di astrazione alto.

Intento esprime brevemente ciò che il pattern vuole ottenere.

Problema comprende motivazione e applicabilità: precisa il contesto e le condizioni in cui la soluzione è appropriata.

Soluzione presenta classi, oggetti, responsabilità e relazioni; non è una singola implementazione rigida.

Conseguenze rendono espliciti risultati, vantaggi, svantaggi e compromessi.

Nel problema agiscono le *forze*, cioè obiettivi e vincoli spesso contrastanti che la progettazione deve bilanciare. Esempi di utilizzo e codice possono completare la descrizione.

I pattern si classificano secondo lo scopo: i *creazionali* organizzano la creazione di istanze; gli *strutturali* la composizione di classi e oggetti; i *comportamentali* gli algoritmi, la comunicazione e la distribuzione delle responsabilità. Questa classificazione distingue il problema affrontato, anche quando diagrammi di pattern diversi hanno una struttura simile.

Nel catalogo GoF i creazionali sono Abstract Factory, Builder, Factory Method, Prototype e Singleton; gli strutturali Adapter, Bridge, Composite, Decorator, Facade, Flyweight e Proxy; i comportamentali Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method e Visitor. La classificazione colloca anche i nomi non sviluppati in dettaglio; Object Pool, Dependency Injection, MVC e Null

Object appartengono a schemi o tecniche ulteriori e non vanno conteggiati automaticamente nei 23 pattern del catalogo.

Una soluzione elementare può presupporre un sistema centralizzato e un singolo thread logico. Concorrenza, distribuzione e vincoli real-time introducono forze ulteriori: applicare un pattern in tali contesti richiede di riesaminarne le condizioni di funzionamento.

7.2 Quale aspetto della costruzione deve variare?

I pattern creazionali astraggono il processo di creazione: rendono il sistema meno dipendente da come gli oggetti sono creati, composti e rappresentati. Invece di disseminare espressioni `new ClasseConcreta(...)` nel codice client, spostano in un punto dedicato la conoscenza della classe concreta e della costruzione. Il *client* è il codice che richiede e usa un oggetto; una *classe concreta* è una classe istanziabile che realizza un'interfaccia o una classe astratta.

Il client riceve così un oggetto pronto e lo usa attraverso un'astrazione, cioè un'interfaccia o un tipo astratto. Questo favorisce il disaccoppiamento: una dipendenza da una concreta può essere sostituita senza modificare ogni client. Una gerarchia di ereditarietà è fissata in compilazione; una maggiore flessibilità può derivare dalla *composizione*, cioè della costruzione di un oggetto mediante riferimenti ad altri oggetti configurabili a runtime. Non significa che ogni `new` sia errato: l'istanziazione concreta resta necessaria da qualche parte; il punto è localizzarla quando la scelta o la costruzione devono variare.

Un vincolo di unicità conduce a Singleton; la variazione del prodotto a Factory Method; la coerenza di una famiglia ad Abstract Factory; la costruzione per passi a Builder; la creazione per copia a Prototype. Object Pool aggiunge una politica di riuso delle istanze, mentre Dependency Injection separa la costruzione dei collaboratori dal loro utilizzo.

7.3 Singleton: unicità e accesso controllato

L'intento di *Singleton* è assicurare che una classe abbia una sola istanza e fornire un punto di accesso globale a quell'istanza. È appropriato quando il dominio richiede effettivamente un solo gestore condiviso nell'applicazione, per esempio uno spooler di stampa, un file system, un window manager, un registro di log o il gestore della lista dei clienti. Una variabile globale rende un oggetto raggiungibile ma, da sola, non impedisce a qualcuno di costruire altre istanze: il vincolo deve essere controllato dalla classe.

La classe Singleton conserva un riferimento statico all'istanza, rende privato il costruttore e offre un metodo statico `getInstance()`. "Statico" qui significa che il membro appartiene alla classe, perciò `getInstance()` è invocabile prima che esista un oggetto. Il costruttore privato impedisce alle altre classi di usare `new`; il metodo della classe è l'unico punto autorizzato a farlo.

Con inizializzazione *eager*, l'istanza è creata durante l'inizializzazione della classe; con inizializzazione *lazy*, alla prima richiesta. La seconda evita di costruire un oggetto che potrebbe non essere usato, ma il controllo elementare sul riferimento nullo non è sicuro con più thread: due chiamanti possono superarlo contemporaneamente e creare due oggetti. L'inizializzazione della classe protegge invece la creazione eager; nella variante lazy un `getInstance()` sincronizzato può serializzare controllo e costruzione.

Il *Multiton* è la variante che consente un numero finito di istanze, eventualmente associandone una a una chiave, quale una cartella nel caso di `Pagamenti`. Il beneficio è il principio delle *conseguenze locali*: cambiando il numero ammesso di istanze si modifica il fornitore, mentre i client continuano a chiedere l'istanza con la stessa operazione. Rispetto a rendere tutte le operazioni statiche, Singleton conserva un oggetto e consente di variare centralmente la politica sul numero di oggetti; la soluzione interamente statica non crea invece alcuna istanza.

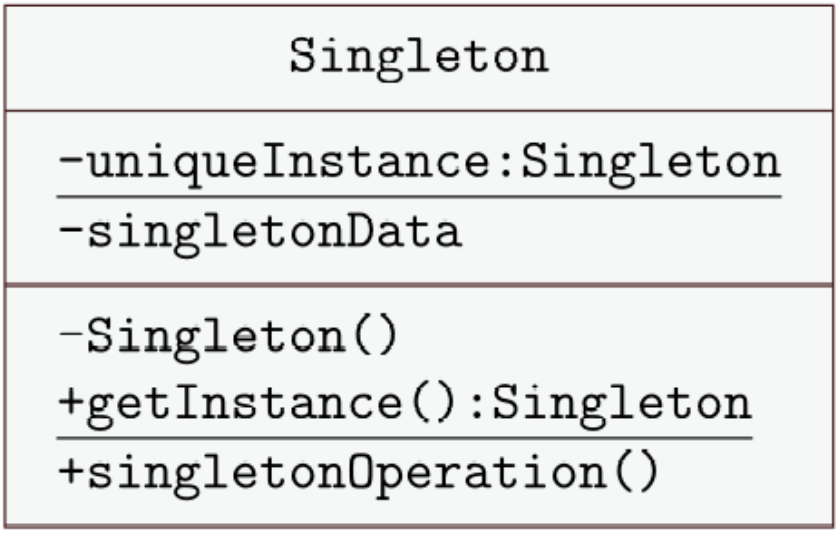


Figura 7.1: Costruttore privato, riferimento e accesso statici; operazioni sullo stato dell'unica istanza.

La sicurezza della creazione non rende automaticamente sicuri i metodi che modificano lo stato condiviso. Il passaggio a Multiton mantiene invariati i client soltanto se il contratto di accesso e le loro assunzioni sull'identità restano validi.

Il vincolo generale sul numero di istanze della classe C è

$$|I(C)| \leq 1, \tag{7.1}$$

dove $I(C)$ è l'insieme delle istanze. Per una creazione lazy può essere inizialmente vuoto; dopo l'inizializzazione riuscita ha un elemento. La sola visibilità globale non impone tale vincolo.

7.3.1 Logs lazy, Fib eager e variante Multiton

L'esempio `Logs` conserva i messaggi in una lista e crea l'unica istanza alla prima richiesta. La condizione sul riferimento evita una seconda costruzione nell'esecuzione a singolo thread considerata dall'esempio.

```
import java.util.List;
import java.util.ArrayList;

public final class Logs {
    private static Logs obj;
    private final List<String> righe = new ArrayList<>();
    private Logs() { }

    public static Logs getInstance() {
        if (obj == null) obj = new Logs();
        return obj;
    }
    public void record(String s) { righe.add(s); }
    public void empty() { righe.clear(); }
    public String dumpLast() { return righe.get(righe.size() - 1); }
    public String dumpAll() {
        String risultato = "";
        for (String s : righe) risultato += s;
```

```
        return risultato;
    }
}
```

`Logs a = Logs.getInstance()` e `Logs b = Logs.getInstance()` ricevono lo stesso riferimento. Questo codice presuppone un solo thread e richiede un adattamento per gestire richieste concorrenti.

`Fib`, che svolge il ruolo di Singleton per i numeri di Fibonacci, inizializza il riferimento statico direttamente alla dichiarazione: `private static Fib obj = new Fib();`. `Logs` rimanda invece la costruzione. Il nome del ruolo non è una keyword Java. Nel Multiton `Pagamenti` può associare una diversa istanza a ciascuna cartella; il criterio è se esista già un'istanza per la chiave richiesta.

`dumpLast` richiede almeno un messaggio; `dumpAll` concatena senza inserire separatori. Se si registrano `"one "`, `"two "`, `"three "`, l'ultimo messaggio è `"three "` e l'intero testo è `"one two three "`, con spazi finali significativi. Un test deve chiamare `empty` prima di preparare ciascuna fixture. Un secondo riferimento ottenuto da `getInstance()`, registrando poi `"uno"` e `"due"`, rende `"due"` visibile anche dal primo: l'identità condivisa può essere controllata con `assertSame(a,b)` e l'effetto con `assertEquals("due",a.dumpLast())`. Questi sono controlli distinti dell'unicità e dello stato.

I tre controlli si possono esprimere con una fixture indipendente per ogni test:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class LogsTest {
    private final Logs l = Logs.getInstance();
    @BeforeEach void prepara() {
        l.empty();
        l.record("one "); l.record("two "); l.record("three ");
    }
    @Test void testLast() { assertEquals("three ", l.dumpLast()); }
    @Test void testAll() { assertEquals("one two three ", l.dumpAll()); }
    @Test void testSingl() {
        Logs l2 = Logs.getInstance();
        assertSame(l, l2);
        l2.record("uno"); l2.record("due");
        assertEquals("due", l.dumpLast());
    }
}
```

La variante eager di `Fib` rende visibile la stessa condivisione tramite un indice:

```
final class Fib {
    private static final Fib obj = new Fib();
    private final int[] x = {1,2,3,5,8,13,21,34,55,89,144};
    private int i = 3;
    private Fib() { }
    static Fib getInstance() { return obj; }
    int getValue() {
        if (i < x.length) i++;
        return x[i - 1];
    }
    void revert() { i = 0; }
}
```

Con due riferimenti `f` e `f2`, due chiamate consecutive sul primo restituiscono 5 e 8; le due successive sul secondo restituiscono 13 e 21. Le righe stampate sono quindi `f 5 8` e `f2 13 21`.

Raggiunto l'ultimo elemento si continua a restituire 144; `revert` riporta l'indice a zero e la chiamata successiva restituisce 1. L'indice non appartiene alle variabili locali dei client.

Esercizio: Pagamenti unico e Pagamenti per cartella

Per trasformare la classe `Pagamenti` in Singleton si aggiungono, al suo interno, i seguenti membri e si mantengono le operazioni già definite:

```
private static final Pagamenti unico = new Pagamenti();
private Pagamenti() { }
public static Pagamenti getInstance() { return unico; }
```

Il client usa `Pagamenti.getInstance()` invece di `new Pagamenti()`. I file letti da client diversi confluiscono nello stesso stato; `svuota` agisce su tutti gli utilizzatori. L'unicità è quindi appropriata solo se quel comportamento è richiesto.

Una variante alternativa associa le istanze a chiavi di cartella; sostituisce il riferimento unico con una mappa e conserva il costruttore privato:

```
private static final Map<String, Pagamenti> istanze = new HashMap<>();
private Pagamenti() { }
public static synchronized Pagamenti getInstance(String cartella) {
    Pagamenti p = istanze.get(cartella);
    if (p == null) {
        p = new Pagamenti();
        istanze.put(cartella, p);
    }
    return p;
}
```

La stessa chiave restituisce la stessa istanza; chiavi diverse danno raccolte indipendenti. Il dominio delle chiavi determina quante istanze sono possibili: questa mappa non impone da sola una capacità massima prestabilita. Per un Multiton limitato occorre limitare le chiavi ammesse o rifiutare le nuove richieste al raggiungimento della capacità. Va anche stabilito se nomi diversi della stessa cartella debbano essere normalizzati. La sincronizzazione dell'accesso protegge la mappa, mentre gli aggiornamenti degli importi richiedono una valutazione distinta.

7.4 Factory Method: delegare la scelta del prodotto

L'intento di *Factory Method* è definire un'interfaccia per creare un oggetto lasciando alle sottoclassi la scelta della classe da istanziare. Un *framework*, nel senso usato qui, è una struttura di classi astratte che definisce lo scheletro di un'applicazione e lascia dettagli a chi lo specializza. Il framework deve restare generico e quindi conosce il tipo astratto `Product`, che non può istanziare, ma deve produrre oggetti concreti: il factory method rinvia questa decisione al `ConcreteCreator` mediante dispatch dinamico o *late binding*, cioè la scelta a runtime della ridefinizione del metodo in base al tipo dinamico dell'oggetto.

I ruoli sono:

Product l'interfaccia comune degli oggetti creati;

ConcreteProduct una sua implementazione;

Creator la classe, spesso astratta, che dichiara `factoryMethod()` con risultato di tipo `Product`; può anche fornire una creazione predefinita;

ConcreteCreator la sottoclasse che implementa o ridefinisce il factory method e istanzia il prodotto concreto.

Un'operazione concreta di `Creator` può chiamare `factoryMethod()` su `this`: se l'oggetto dinamico è un `ConcreteCreator`, viene eseguito il suo override, che restituisce un `ConcreteProduct`; il client conserva il risultato come `Product` e invoca il contratto comune. Tipicamente vi è una corrispondenza uno-a-uno fra concrete creator e concrete product, ma un creator può selezionare più prodotti con logica interna. Il vantaggio è l'intercambiabilità dei prodotti e l'estensione senza modificare i client che conoscono il contratto; il prezzo è la proliferazione di classi. Con un solo prodotto concreto e nessuna variazione prevista, la struttura può essere sovradimensionata.

Sono possibili varianti semplificate: creator e concrete creator nella stessa classe, factory statico e parametri che suggeriscono il prodotto. Queste varianti centralizzano comunque la creazione, ma il Factory Method classico è quello basato sulla delega alle sottoclassi; non si devono confondere le due descrizioni.

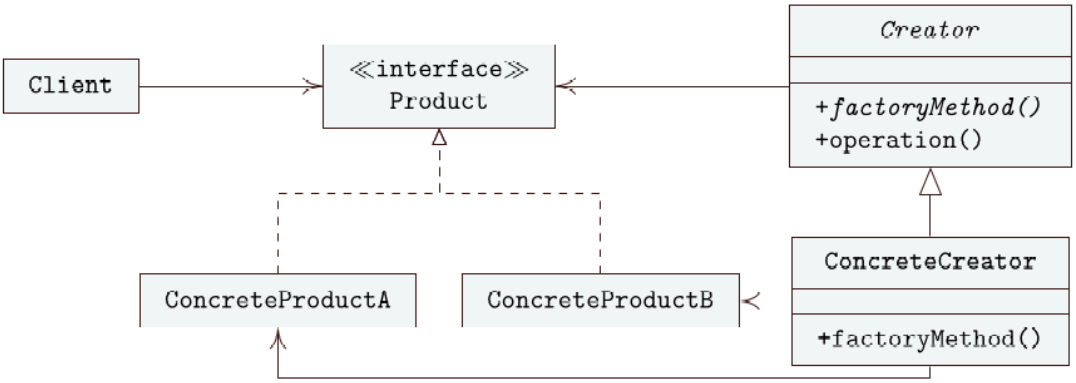


Figura 7.2: Ruoli del Factory Method nel diagramma delle classi.

Il Client richiede e usa il prodotto. Renderlo esplicito nel diagramma chiarisce la collaborazione senza imporre una classe con quel nome: il ruolo può essere svolto da qualunque parte del programma che utilizzi il servizio del creator.

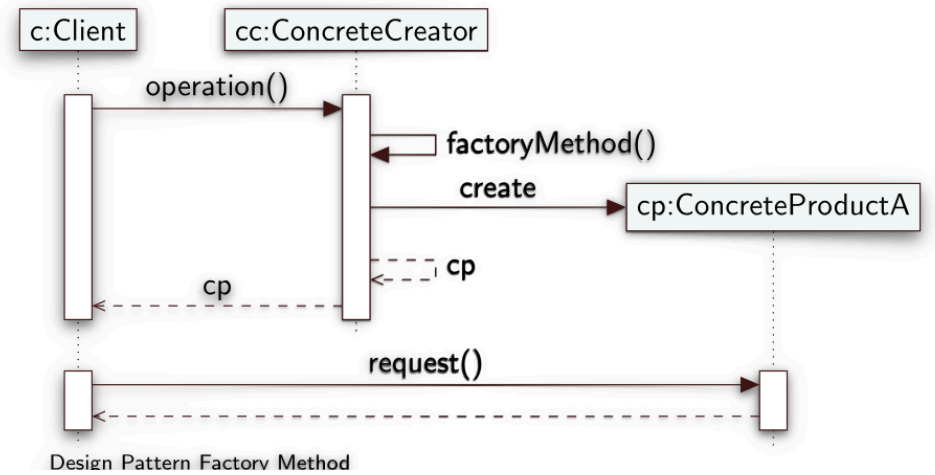


Figura 7.3: Creazione delegata al ConcreteCreator e uso del prodotto.

Il client invoca `operation()` sul creator; il metodo ereditato richiama il factory method sulla stessa istanza; il dispatch seleziona l'override del ConcreteCreator; il prodotto viene restituito e usato attraverso Product. Il diagramma delle classi può evidenziare soltanto il disaccoppiamento essenziale e lasciare implicita una dipendenza di configurazione presente nella

sequenza. Per ricostruire le dipendenze occorre quindi distinguere il collegamento necessario all'uso da quello necessario alla configurazione.

La scelta di rappresentare Product come interfaccia e Creator come classe astratta ha una motivazione specifica. Il prodotto può richiedere soltanto un contratto, senza stato comune; il creator può avere un contatore delle istanze, un metodo concreto `final` che lo aggiorna e un factory method astratto che crea il prodotto. Un'interfaccia non fornisce campi d'istanza né metodi d'istanza finali. Non è però un vincolo universale del pattern: il ruolo Product può anche essere una classe astratta se il dominio richiede stato e comportamento comuni.

7.4.1 Creator con contatore e sospensione degli studenti

Questo esempio usa `Studente` come interfaccia Product, distinta dalla gerarchia anagrafica. `StudenteAttivo` registra esami; `StBloccato` conserva la media al momento della sospensione e rifiuta nuovi esami.

```
import java.util.*;

interface Studente {
    boolean nuovoEsame(String materia, int voto);
    float getMedia();
}
record Esame(String materia, int voto) { }
class StudenteAttivo implements Studente {
    private final List<Esame> esami = new ArrayList<>();
    public boolean nuovoEsame(String m, int v) {
        esami.add(new Esame(m, v));
        return true;
    }
    public float getMedia() {
        if (esami.isEmpty()) return 0;
        float somma = 0;
        for (Esame e : esami) somma += e.voto();
        return somma / esami.size();
    }
}
class StBloccato implements Studente {
    private final float media;
    StBloccato(float media) { this.media = media; }
    public boolean nuovoEsame(String m, int v) { return false; }
    public float getMedia() { return media; }
}
abstract class Creator {
    private int numIstanze;
    public final Studente creaStudente() {
        numIstanze++;
        return getStudente();
    }
    public int getNum() { return numIstanze; }
    public abstract Studente getStudente();
    public abstract Studente sospendi(Studente s);
    public abstract Studente ripristina(Studente s);
}
class StCreator extends Creator {
    private boolean attivo = true;
    private final Map<Studente, Studente> bloccati = new HashMap<>();
    public Studente getStudente() {
```

```
        if (attivo) return new StudenteAttivo();
        return new StBloccato(0);
    }
    public Studente sospendi(Studente s) {
        Studente sospeso = new StBloccato(s.getMedia());
        bloccati.put(sospeso, s);
        return sospeso;
    }
    public Studente ripristina(Studente s) {
        return bloccati.remove(s);
    }
}
```

La sequenza è `client.creaStudente` sul creator, chiamata interna a `getStudente`, costruzione di `StudenteAttivo`, restituzione come `Studente` e invocazione di `nuovoEsame`. Il metodo finale impedisce di saltare il conteggio ridefinendolo. Il contatore registra però le chiamate a `creaStudente`: chiamare direttamente `getStudente` o costruire un bloccato in `sospendi` non lo incrementa.

La mappa associa il sostituto bloccato all'originale: `s = creator.sospendi(s)` cambia il riferimento del client; `s = creator.ripristina(s)` recupera gli esami originali e rimuove l'associazione. Un riferimento estraneo restituisce `null`. Altri client che conservano il riferimento allo studente attivo non vengono automaticamente bloccati. Il flag `attivo`, inizializzato a vero e privo qui di un'operazione di modifica, seleziona sempre lo studente attivo; l'eventuale configurazione alternativa richiede di completarne il contratto.

Lo stesso schema vale per un framework di documenti: `createDocument()` restituisce il tipo astratto `Document`, mentre creator specializzati producono `PdfDocument` o `WordDocument`. Il codice che apre, conserva e usa il documento resta dipendente dal contratto comune.

7.4.2 Posti del teatro: client, creatore e prodotto

`Posto` è Product; `PostoPalco` e `PostoPlatea` sono ConcreteProduct; `Posizioni` è Creator; `CreatorPosti` è ConcreteCreator. `Main` e `Biglietto` usano i posti attraverso il contratto comune.

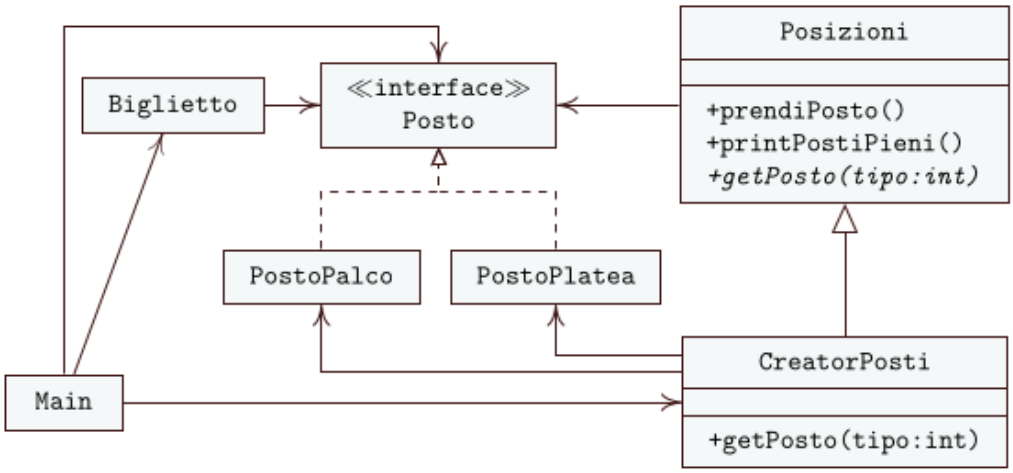


Figura 7.4: Nel teatro il Main configura il creatore e usa Posto; Biglietto conserva il posto assegnato.

Nel teatro, `Main` chiede un posto al creator, lo tratta come `Posto` e lo inserisce in un `Biglietto`; il biglietto dipende anch'esso dal contratto Posto. La variante che nomina `CreatorPosti` nel Main conserva una dipendenza concreta nel punto di configurazione. Dichiarare il riferimento come `Posizioni` disaccoppia l'uso; ricevere il creator già costruito nel costruttore disaccoppia anche la scelta del creatore dal client. Il parametro numerico di `prendiPosto(tipo)` sostituisce la conoscenza dei nomi `PostoPalco` e `PostoPlatea` con una convenzione da documentare.

Assegnazione, biglietto e condizioni di terminazione

`Posto` offre `getPosizione(): String`, `getCosto(): int` e `getSettore(): String`. Un `PostoPalco` sceglie nel costruttore un numero casuale fra 1 e 20; la posizione è quel numero convertito in testo. Per numeri da 1 a 10 il costo è 40 e il settore Blu; da 11 a 19 il costo è 50 e il settore Verde; il numero 20 costa 50 ed è Centrale.

`Posizioni` conserva un insieme ordinato di identificativi occupati e una lista dei posti assegnati, con capacità dichiarata 100. La logica di richiesta è:

```
public Posto prendiPosto(int tipo) {
    if (postiPieni.size() == 100) return null;
    Posto p;
    do { p = getPosto(tipo); }
    while (postiPieni.contains(p.getPosizione()));
    postiPieni.add(p.getPosizione());
    posti.add(p);
    return p;
}
public abstract Posto getPosto(int tipo);
```

`CreatorPosti.getPosto` restituisce `new PostoPalco()` per `tipo==1` e `new PostoPlatea()` negli altri casi. Il framework decide quando chiedere un prodotto e come registrarlo; la sottoclasse decide quale concreta costruire. `printPostiPieni` percorre la lista e stampa posizione, settore e costo.

Il client chiede prima `prendiPosto`, controlla che non sia `null`, poi costruisce `Biglietto(nome, posto)`. Il biglietto conserva il contratto `Posto`; `stampaDettagli` ne richiama posizione, settore e costo. Il messaggio finale riunisce intestatario e dati del posto senza riconoscerne la concreta.

La capacità globale 100 non garantisce che il ciclo termini: dopo aver occupato tutte le 20 posizioni di palco, nuove estrazioni di palco continuerebbero a produrre duplicati anche con meno di 100 posti totali. Occorre quindi distinguere disponibilità per categoria e capacità globale. Inoltre gli identificativi devono distinguere posti di settori diversi se i numeri coincidono. Senza una specifica completa della platea e dell'esaurimento per categoria non è possibile dedurre una politica unica: il frammento illustra Factory Method sotto la precondizione che esista ancora un posto ottenibile con identificativo nuovo; un gestore completo deve rendere esplicita e controllare tale precondizione.

7.5 Object Pool: acquisizione, rilascio e riuso

Un *Object Pool* è un deposito di istanze già create. Quando un client richiede un oggetto, il pool ne restituisce uno libero; alla fine dell'uso il client lo rilascia. Un pool a dimensione fissa non crea un oggetto se nessuno è disponibile; uno estendibile può crearne uno quando il deposito è vuoto. L'uso è esclusivo: un oggetto consegnato non deve restare disponibile nel pool. Prima del riuso può essere necessario reimpostarne lo stato, altrimenti il nuovo client vede dati del precedente. Un ConcreteCreator di Factory Method può implementare

questa politica senza cambiare l'interfaccia di richiesta; il pool stesso può essere unico e quindi combinato con Singleton.

Un pool può precaricare gli oggetti nel costruttore oppure crearli su richiesta. In un pool limitato, il numero di istanze create complessivamente va distinto dal numero di istanze attualmente libere: la sola dimensione della lista dei liberi non indica quante siano in uso. Quando si raggiunge la capacità, la politica di indisponibilità deve essere esplicita; non va introdotta una nuova istanza violando il limite.

Un `CreatorPool` estendibile rimuove un oggetto libero con `remove(0)` e, se il deposito è vuoto, costruisce un cerchio: rappresenta un pool estendibile, non un pool a capacità fissa. `releaseShape(s)` restituisce la risorsa. Occorre evitare rilasci duplicati, oggetti estranei e consegna simultanea dello stesso oggetto a due client. Prima di consegnare una risorsa riusata si ripristina lo stato previsto. La combinazione con Singleton controlla l'unicità del gestore del pool, non il numero delle risorse gestite.

Il codice rende visibile la politica: se la lista non è vuota riusa una `Shape`, altrimenti crea un `Circle`; `releaseShape` restituisce l'oggetto.

```
public class CreatorPool {
    private List<Shape> pool = new ArrayList<>();
    public Shape getShape() {
        if (pool.size() > 0) return pool.remove(0);
        return new Circle();
    }
    public void releaseShape(Shape s) { pool.add(s); }
}
```

In un pool con stato significativo, il rilascio o la nuova acquisizione deve aggiungere il ripristino, o *reset*, delle condizioni previste per un nuovo utilizzatore.

7.6 Dependency Injection: costruzione e wiring

La *Dependency Injection* (DI) è la tecnica per cui un oggetto riceve dall'esterno gli oggetti da cui dipende, anziché costruirli. Una *dipendenza* è un oggetto o servizio necessario al funzionamento di un altro oggetto. Nella *constructor injection* la dipendenza è parametro del costruttore: la costruzione è separata dall'uso e l'oggetto può essere completo dal primo istante. La *setter injection* configura invece la dipendenza dopo la costruzione: richiede di gestire uno stato inizialmente incompleto e il rischio di dimenticare il setter. L'*injector*, o *composition root*, crea e collega le istanze (*wiring*) in un punto. Poiché il factory method è un punto di creazione, può essere un posto naturale per iniettare le dipendenze nel prodotto; anche il creator scelto può essere iniettato nel client con tipo statico astratto.

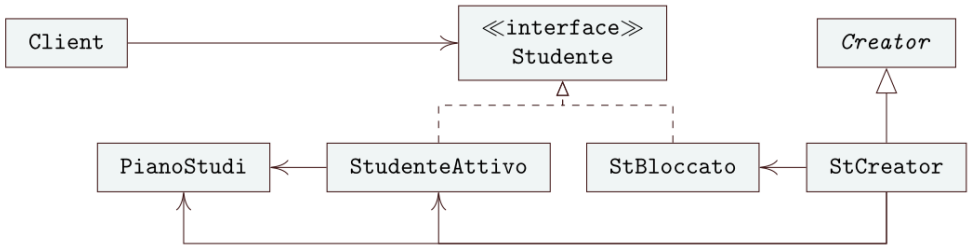


Figura 7.5: `StCreator` costruisce `StudenteAttivo` e gli fornisce `PianoStudi`.

La dipendenza deve essere disponibile quando l'oggetto inizia a essere usato. Un parametro nel costruttore rende esplicito il requisito e può essere conservato in un campo finale. Non

garantisce da solo un valore non nullo: tale invariante richiede un controllo o un contratto effettivamente rispettato. Per esempio:

```
final class StudenteAttivo {
    private final PianoStudi piano;
    StudenteAttivo(PianoStudi piano) {
        this.piano = java.util.Objects.requireNonNull(piano);
    }
}
```

Nell'esempio, il factory method realizza il wiring del prodotto. Anche il Main può fungere da composition root, creando creator, prototipi e client e collegandoli. Non ogni dipendenza dell'intera applicazione deve essere fabbricata da un'unica factory di prodotti: la responsabilità della factory deve rimanere coerente con il proprio dominio.

7.7 Abstract Factory: prodotti coerenti di una famiglia

L'intento di *Abstract Factory* è fornire un'interfaccia per creare famiglie di oggetti correlati senza specificarne le classi concrete. Una *famiglia* raccoglie prodotti di tipi diversi ma appartenenti alla stessa variante del sistema; per esempio un pulsante e una barra di scorrimento nello stesso look-and-feel. Il problema richiede sia di cambiare l'intera famiglia, sia di evitare di mescolare prodotti di famiglie diverse, che produrrebbero un'interfaccia incoerente.

`AbstractFactory` dichiara un'operazione di creazione per ciascun tipo di prodotto. Ogni `ConcreteFactory` implementa tali operazioni e produce esclusivamente la propria famiglia. Per ogni tipo di prodotto esiste un `AbstractProduct`, il contratto che nasconde le implementazioni; i concrete product lo implementano. Il client riceve una factory attraverso il tipo `AbstractFactory` e usa solo essa e gli `AbstractProduct`. In una rappresentazione a griglia, le righe sono i tipi di prodotto e le colonne le famiglie: una concrete factory seleziona una colonna intera.

A differenza del Factory Method, che disaccoppia dalla concreta di un prodotto, Abstract Factory disaccoppia da un'intera famiglia e ne preserva la coerenza. Le famiglie sono facilmente scambiabili; aggiungere un nuovo tipo di prodotto è invece oneroso, perché richiede un metodo nuovo nell'astrazione e in ogni factory concreta. L'Open/Closed Principle prescrive che un'entità software sia aperta all'estensione e chiusa alla modifica: qui si osserva nel poter aggiungere una famiglia attraverso una nuova factory, non nell'assenza assoluta di modifiche a tutte le astrazioni.

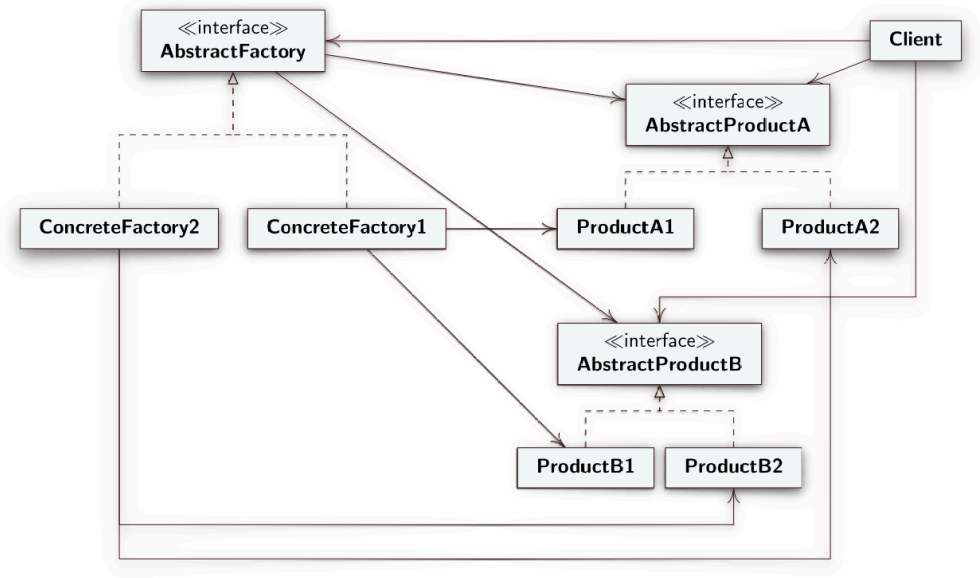


Figura 7.6: Due tipi di prodotto e due famiglie nell'Abstract Factory.

Le lettere A e B identificano tipi distinti di prodotto; i numeri 1 e 2 identificano le famiglie. Nell'esempio grafico, Button, Checkbox e Scrollbar sono tipi, Windows e macOS sono famiglie. Aggiungere Linux significa aggiungere una factory e i relativi prodotti; aggiungere un nuovo tipo di widget richiede invece di estendere il contratto di tutte le factory. Il client dipende direttamente dai contratti dei prodotti perché ne usa le operazioni, oltre che da AbstractFactory perché le richiede.

La coerenza è ottenuta facendo produrre alla factory selezionata soltanto oggetti della famiglia corretta e usando tale factory con disciplina. Il diagramma non impedisce magicamente a codice esterno di mescolare istanze ottenute da factory diverse. Ricorrere a cast ai prodotti concreti nel client indebolisce l'astrazione e rende meno agevole cambiare famiglia.

Per una Abstract Factory con m tipi di prodotto e n famiglie, la griglia concettuale ha $m \cdot n$ concrete product; una factory seleziona una delle n colonne. Questa relazione descrive la classificazione dei partecipanti, non il costo di esecuzione:

$$N_{\text{prodotti}} = mn, \tag{7.2}$$

ove N_{prodotti} è il numero di combinazioni concrete, m il numero di tipi e n il numero di famiglie.

7.7.1 Esempio: famiglie di icone e testi

Nell'esempio di Abstract Factory, `Creator` dichiara `getIcon()` e `getText()`; `CreatorEng` restituisce `Box` e `English`, mentre `CreatorJ` restituisce `Circle` e `Japanese`. Il client conserva la factory come `Creator`, ottiene un'icona e un testo dalla medesima famiglia e può cambiare variante cambiando la factory ricevuta.

```
interface Icon {
    void sad(); void happy(); void surprise();
}
class Circle implements Icon {
    public void sad() { System.out.print(" (> _ <) "); }
    public void happy() { System.out.print(" (^ _ ^) "); }
    public void surprise() { System.out.print(" (o _ o) "); }
}
```

```
class Box implements Icon {
    public void sad() { System.out.print(":-[ "); }
    public void happy() { System.out.print(":-] "); }
    public void surprise() { System.out.print(":-O "); }
}
interface Text {
    void welcome(); void attention(); void surprise();
}
class English implements Text {
    public void welcome() {
        System.out.println("::: Welcome. Nice to meet you. :::::");
    }
    public void attention() {
        System.out.println("::: Be quiet please! :::::");
    }
    public void surprise() { System.out.println("::: Oh, my dear! :::::"); }
}
class Japanese implements Text {
    public void welcome() {
        System.out.println("Youkoso. Konnichiwa! Hajimemashite.");
    }
    public void attention() {
        System.out.println("Shizuka ni shite kudasai!");
    }
    public void surprise() { System.out.println("Bikkuri shita!"); }
}
interface Creator {
    Icon getIcon(); Text getText();
}
class CreatorEng implements Creator {
    public Icon getIcon() { return new Box(); }
    public Text getText() { return new English(); }
}
class CreatorJ implements Creator {
    public Icon getIcon() { return new Circle(); }
    public Text getText() { return new Japanese(); }
}
```

Il client richiede i due prodotti e ne usa le operazioni:

```
Icon ic = c.getIcon();
Text t = c.getText();
ic.happy(); t.welcome();
ic.surprise(); t.surprise();
ic.sad(); t.attention();
```

Con `CreatorEng` ottiene le faccine Box seguite dai testi inglesi; sostituendo soltanto la factory con `CreatorJ` ottiene Circle e testi giapponesi. I due assi sono concreti: non si cambia un'icona isolata, ma la famiglia di oggetti richiesta dall'applicazione.

Nei diagrammi una freccia dalla concreta factory al prodotto astratto può essere omessa per non duplicare il contratto già dichiarato dall'interfaccia factory. L'omissione grafica non significa che il metodo concreto possa ignorare il tipo di ritorno. Analogamente, la dipendenza del client dai prodotti è diretta perché il client ne invoca i metodi: non si ricava una generale transitività di tutte le relazioni UML.

Nel toolkit grafico, `WindowsButton` e `MacScrollBar` appartengono a famiglie diverse. Mescolarli vanifica la coerenza che la selezione di una ConcreteFactory deve preservare. Una

nuova famiglia richiede i prodotti della nuova colonna; un nuovo tipo di prodotto richiede una nuova operazione in ciascuna factory.

7.8 Builder: costruire passo per passo un oggetto complesso

L'intento di *Builder* è separare la costruzione di un oggetto complesso dalla sua rappresentazione, affinché lo stesso processo costruttivo possa produrre rappresentazioni diverse. È applicabile quando l'algoritmo di costruzione deve restare indipendente dalle parti dell'oggetto e dal loro assemblaggio. Un caso frequente è un oggetto con molti attributi, alcuni opzionali, che devono diventare immutabili, cioè non modificabili dopo la costruzione.

Il costruttore *telescopico* offre molti costruttori con un numero crescente di parametri: consente di omettere valori, ma rende difficile al chiamante ricordare ordine e significato degli argomenti. I setter separati evitano la lunga lista di parametri ma impediscono agli attributi di essere `final` e non permettono di verificare nel costruttore che lo stato sia già consistente. Con Builder il client: (1) crea il builder con i dati richiesti; (2) configura passo per passo i dati opzionali con metodi che possono restituire il builder stesso; (3) invoca `build()` o l'equivalente, che verifica i dati necessari e crea il product immutabile.

I partecipanti sono `Builder`, interfaccia per costruire le parti; `ConcreteBuilder`, che assembla e conserva la rappresentazione in costruzione e fornisce il product; `Director`, che costruisce usando l'interfaccia Builder; e `Product`, l'oggetto complesso risultante. Il Director è un ruolo utile, non un obbligo del semplice builder fluente mostrato nell'esempio. Builder nasconde la struttura del product, isola costruzione e rappresentazione e dà controllo fine, passo dopo passo, fino alla consegna dell'oggetto completo.

Per Builder, `BigliettoBuilder` conserva dati provvisori quali titolare, agente, data e prezzo; metodi come `titolare(...)` restituiscono il builder, mentre `getBiglietto()` controlla che il titolare sia presente prima di costruire `Biglietto`. Il Director `InitBiglietto` mostra la sequenza di configurazione e la consegna finale dell'oggetto. L'esempio evidenzia che la validazione deve avvenire prima della consegna; uno stato provvisorio rappresentato da `null` deve essere controllato esplicitamente, perché Builder non garantisce automaticamente che tutti i dati richiesti siano presenti.

7.8.1 Tre modi di configurare Biglietto

Nella variante telescopica i campi di costruzione sono `titolare`, `agente`, `data` e `prezzo`. Il costruttore completo riceve tutti i valori; quello senza prezzo delega con `this(t,a,d,100)`, quello senza data usa `LocalDateTime.now()` e quello con il solo titolare usa l'agente `"BBBSelling"`. La catena `this(...)` concentra l'assegnazione in un costruttore, ma moltiplica le combinazioni da ricordare.

La variante con costruttore vuoto sostituisce i campi finali con campi assegnabili e offre `setTitolare`, `setAgente`, `setData` e `setPrezzo`. Fra due setter l'oggetto può essere incompleto, e il chiamante può dimenticare una configurazione necessaria. Il builder raccoglie invece i dati prima di consegnare il prodotto:

```
import java.time.LocalDateTime;

class Biglietto {
    final String titolare, agente;
    final LocalDateTime data;
    final double prezzo;
    private double acconto;
    Biglietto(String t, String a, double p, LocalDateTime d) {
        titolare = t; agente = a; prezzo = p; data = d;
```

```
    }
}
interface BBuilder {
    BBuilder agente(String a);
    BBuilder titolare(String t);
    BBuilder sconto(double sc);
    BBuilder data(LocalDateTime d);
    Biglietto getBiglietto();
}
class BigliettoBuilder implements BBuilder {
    private String titolare, agente = "SingleSeller";
    private LocalDateTime data;
    private double prezzo = 50;
    public BigliettoBuilder agente(String a) {
        agente = agente + " " + a;
        return this;
    }
    public BigliettoBuilder titolare(String t) {
        titolare = t; return this;
    }
    public BigliettoBuilder data(LocalDateTime d) {
        data = d; return this;
    }
    public BigliettoBuilder sconto(double sc) {
        if (prezzo >= 40 && sc <= 10) prezzo = (1 - sc / 100) * prezzo;
        return this;
    }
    public Biglietto getBiglietto() {
        if (titolare == null) return null;
        return new Biglietto(titolare, agente, prezzo, data);
    }
}
class InitBiglietto {
    Biglietto getBiglietto() {
        return new BigliettoBuilder().agente("Tim Seller")
            .titolare("Mario Rossi").sconto(5)
            .data(LocalDateTime.now()).getBiglietto();
    }
}
```

`InitBiglietto` svolge il ruolo Director: stabilisce l'ordine dei passi e ottiene il prodotto. La catena produce agente `"SingleSeller Tim Seller"` e prezzo $50(1 - 5/100) = 47{,}5$. Il controllo dello sconto richiede contemporaneamente prezzo corrente almeno 40 e percentuale al massimo 10; non impone un limite inferiore alla percentuale e non verifica tutti i dati del prodotto. Se il dominio esclude percentuali negative o date mancanti occorre un vincolo aggiuntivo, che non va dato per implicito. Due sconti consecutivi agiscono sul prezzo già ridotto. `getBiglietto` rifiuta soltanto il titolare nullo, restituendo `null`.

I quattro campi di costruzione sono finali, ma `acconto` è distinto e mutabile: se il contratto permette di aggiornarlo, il biglietto nel suo complesso non è immutabile. Builder controlla la costruzione; l'immutabilità successiva dipende da tutti i campi e dalle operazioni esposte.

7.9 Prototype: creazione mediante copia

L'intento di *Prototype* è specificare i tipi degli oggetti da creare mediante un'istanza prototipo e creare nuovi oggetti copiandola. È utile quando lo stato iniziale è complesso e simile a

quello di un oggetto già configurato, oppure quando scrivere una concreta nell'espressione `new` accoppierebbe il client a quella classe. Il client usa `prototipo.clone()` attraverso il contratto `Prototype`: il tipo emerge dall'istanza, non dal nome della concreta scritto nel client.

`Prototype` dichiara la clonazione; `ConcretePrototype` la realizza; il client crea copie senza nominare la concreta. Un `PrototypeRegistry` è un registro separato, tipicamente una mappa da chiave semantica a prototipo: permette di registrare o rimuovere prototipi a runtime. Il registro può essere Singleton o una dipendenza iniettata. Così, per introdurre una variante non occorre necessariamente una gerarchia di creator: si può registrare un nuovo oggetto prototipo. Una composizione, per esempio un pannello con controlli, può essere clonata come struttura intera.

La semantica della copia deve essere esplicita. Una *copia superficiale* duplica l'oggetto ma conserva i riferimenti agli oggetti raggiunti dai suoi campi. Una *copia profonda* duplica anche gli oggetti mutabili che devono appartenere allo stato indipendente del clone. La scelta dipende dal contratto: risorse condivise e oggetti immutabili possono restare comuni.

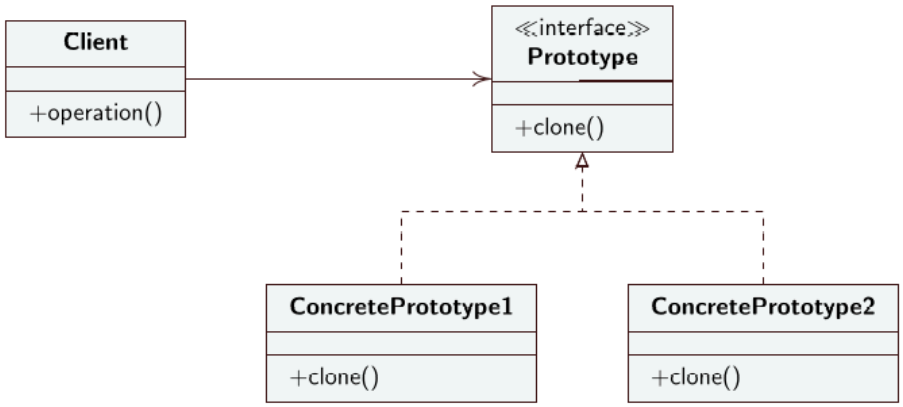


Figura 7.7: Il client chiede la copia tramite l'interfaccia Prototype.

Il registro contiene prototipi già esistenti: qualcuno deve crearli e configurarli prima della prima clonazione. Una nuova variante di stato, per esempio un cerchio rosso rispetto a uno blu, può richiedere soltanto una diversa istanza registrata; un nuovo tipo strutturale, come un triangolo con attributi propri, richiede comunque la sua implementazione di ConcretePrototype. Prototype evita la corrispondente gerarchia di creator, non elimina le classi necessarie al dominio.

`addAll()` inserisce gli elementi in una collezione di destinazione senza clonare gli oggetti referenziati. Una lista nuova evita di condividere il contenitore, ma può ancora condividere tutti gli elementi mutabili. Per copiare un grafo con cicli occorre memorizzare in una mappa ogni coppia originale–clone prima di proseguire sui riferimenti: incontrando di nuovo l'originale, si riusa il clone già creato. Si evita così la ricorsione indefinita e si preservano le relazioni di condivisione previste.

La copia può essere seguita da un'inizializzazione specifica: un contratto usato come prototipo può conservare identificativo, data e cliente del modello, che devono essere sostituiti per il nuovo caso. Copiare l'oggetto non decide automaticamente quali dati debbano cambiare.

Rispetto alle classi `CerchioCreator`, `QuadratoCreator` e `TriangoloCreator`, un registro di prototipi evita una gerarchia parallela di creatori. Le varianti `CerchioRosso`, `CerchioBlu` e `CerchioGrande` possono essere sostituite da stati diversi di uno stesso tipo Cerchio. Un pannello composto da Button, Label e TextField mostra invece una copia di struttura: si deve decidere quali parti clonare e quali mantenere condivise.

7.9.1 Esempio: agenda di impegni e quaderno di note

Una nuova agenda può partire dagli impegni già inseriti, oppure un quaderno dalle note precedenti. Se originale e clone condividono la stessa collezione, aggiungere o rimuovere un elemento tramite uno modifica anche ciò che vede l'altro. Se la collezione è copiata ma gli elementi sono condivisi, cambiano indipendentemente gli elenchi, mentre modificare un impegno mutabile resta visibile a entrambi. Una copia profonda separa anche gli elementi per i quali il contratto richiede indipendenza.

Un Documento con font, margini, lingua e stile già configurati mediante numerose operazioni è un buon candidato a prototipo: copiarne lo stato evita di ripetere la stessa preparazione, poi si modificano i dati specifici del nuovo documento. Nel wiring di agenda e note, il Main può creare il `Calendario` e il `Notebook` iniziali e iniettarli nel client, che lavorerà con i loro contratti di copia. La prima istanza deve comunque essere costruita prima di poterla clonare.

Nel caso ciclico Dipendente–Manager, il dipendente riferisce il proprio manager e il manager conserva la lista dei dipendenti. Una copia ricorsiva ingenua passerebbe dal dipendente al manager e di nuovo allo stesso dipendente senza terminare. Si crea prima il clone vuoto del dipendente, lo si registra nella mappa originali–cloni, poi si copia il manager. Quando la copia della lista incontra il dipendente originario, recupera il clone già registrato: il manager clonato punta così allo stesso dipendente clonato, senza creare un duplicato. Per oggetti distinti ma uguali per contenuto la mappa deve distinguere l'identità degli originali, per esempio con `IdentityHashMap`.

Capitolo 8

Design pattern strutturali

8.1 Comporre strutture e localizzare le dipendenze

I *design pattern strutturali* descrivono come comporre classi e oggetti per ottenere strutture più flessibili, riusabili ed estendibili. Mentre i pattern creazionali controllano come ottenere gli oggetti, quelli strutturali controllano come collegarli: possono rendere compatibili due interfacce, nascondere un sottosistema, rappresentare una gerarchia ricorsiva, aggiungere responsabilità oppure separare due dimensioni indipendenti della progettazione.

In Adapter, Facade, Composite, Decorator e Bridge il client, cioè il codice che richiede un servizio, deve dipendere dal contratto necessario e non dai dettagli che possono cambiare. Il ricorso a questi pattern non è automatico: ogni astrazione aggiunge classi, riferimenti e livelli di delega; è giustificato quando esiste il problema strutturale che essa risolve.

I cinque pattern trattati agiscono su problemi distinti:

Adapter converte l'interfaccia disponibile nell'interfaccia attesa dal client; risolve quindi un'incompatibilità fra contratti.

Facade offre un'unica interfaccia di alto livello a un sottosistema formato da più classi; riduce la complessità che il client deve conoscere.

Composite rappresenta una struttura ricorsiva parte–tutto e consente di trattare allo stesso modo un elemento semplice e un contenitore di elementi.

Decorator avvolge un oggetto per aggiungergli dati o comportamento dinamicamente, mantenendo il contratto dell'oggetto avvolto.

Bridge separa un'astrazione dalla sua implementazione quando entrambe variano indipendentemente; le collega mediante composizione anziché fondere le variazioni nell'ereditarietà.

Un'*interfaccia* è il contratto di operazioni visibile a chi usa un oggetto; una *composizione*, nel senso della progettazione a oggetti, è il possesso di un riferimento a un altro oggetto per delegargli parte del lavoro. La *delega* è proprio l'inoltro di una richiesta a tale oggetto. Queste definizioni non vanno confuse con la composizione UML, che esprime invece un vincolo di ciclo di vita fra intero e parte.

8.2 Adapter: tradurre un contratto incompatibile

L'intento di *Adapter* è convertire l'interfaccia di una classe in un'altra interfaccia attesa dai client, permettendo così la collaborazione di classi altrimenti incompatibili. Il caso tipico è una classe di libreria utile ma non utilizzabile direttamente: i nomi dei metodi, il numero o tipo dei parametri, il tipo restituito oppure la sequenza di chiamate non coincidono con il contratto

già usato dall'applicazione. Non si può, o non conviene, modificare né la libreria né tutti i chiamanti.

Nella variante *Object Adapter*, basata sulla composizione, i ruoli sono:

Target l'interfaccia che il client si aspetta;

Client il chiamante che usa soltanto oggetti conformi a Target;

Adaptee l'oggetto o la classe di libreria con l'interfaccia incompatibile;

Adapter l'oggetto che implementa Target, conserva un riferimento all'Adaptee e traduce la richiesta invocandone le operazioni appropriate.

Il client invoca dunque, per esempio, `target.request()`; Adapter converte argomenti e risultato quando necessario e invoca il metodo dell'Adaptee. Il client vede Adapter come un Target, non come una libreria. Target resta un contratto autonomo: se coincidesse con Adapter, il client sarebbe accoppiato all'adattamento e una modifica locale si propagherebbe ai chiamanti.

Nella variante *Class Adapter*, basata sull'ereditarietà, Adapter estende Adaptee e implementa Target: può chiamare i metodi ereditati senza mantenere un riferimento. L'Adaptee è però fissato nella gerarchia, quindi non è selezionabile a runtime. Questa forma può essere un *adapter a due vie*: poiché espone sia Target sia l'interfaccia ereditata di Adaptee, può essere passato a codice che si aspetta l'uno o l'altro contratto. Il termine "due vie" non significa che ogni Object Adapter abbia automaticamente questa proprietà.

Adapter mantiene indipendenti client e libreria: sostituire l'Adaptee richiede di intervenire nell'adattatore, non nel client. Può inoltre aggiungere controlli di *precondizione*, cioè condizioni da soddisfare prima dell'operazione, e di *postcondizione*, cioè proprietà garantite al termine se l'operazione riesce. Un Object Adapter può istanziare pigramente l'Adaptee alla prima invocazione che ne richieda l'uso (*lazy initialisation*). Il costo è un livello di indirettezza: a ogni invocazione del client corrisponde almeno un'invocazione dell'adattatore, con possibile rallentamento e minore immediatezza del codice.

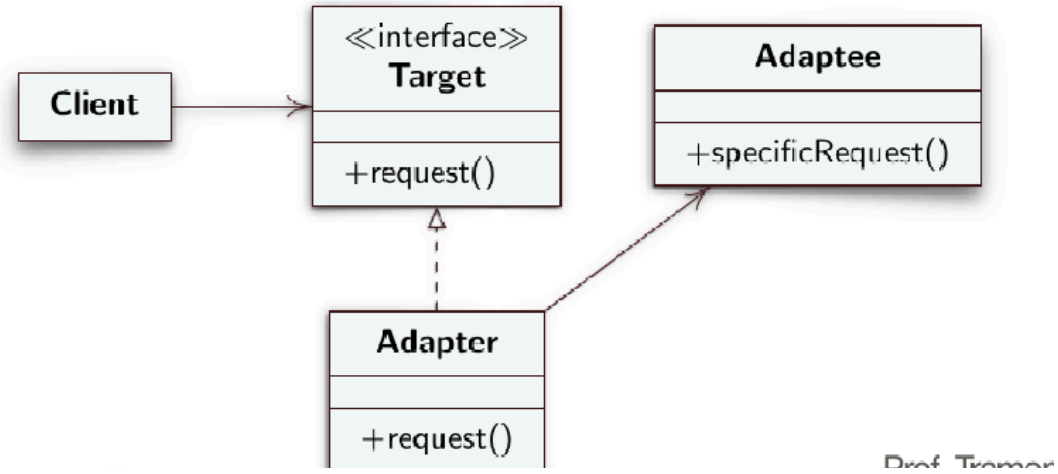


Figura 8.1: Object Adapter: Target è separato dalla libreria Adaptee.

Per supportare una nuova versione incompatibile della libreria si può mantenere l'adattatore precedente e aggiungerne uno per la versione nuova. Il client continua a usare Target, mentre il punto di configurazione seleziona l'adattatore necessario e può ripristinare quello precedente.

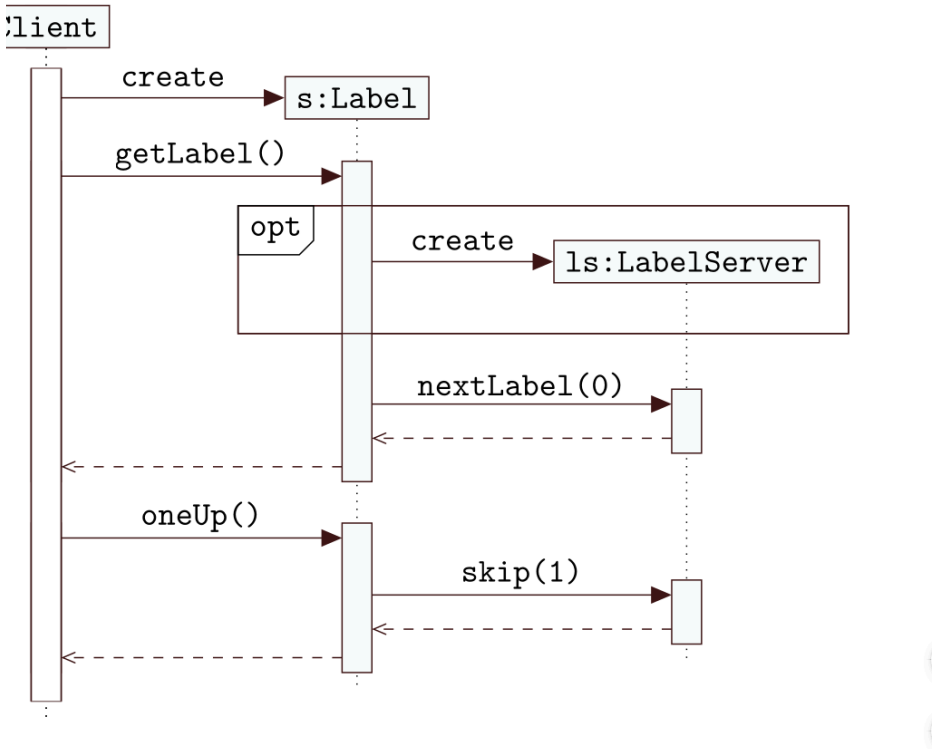


Figura 8.2: `Label` crea `LabelServer` solo quando necessario; il frame `opt` esprime il controllo opzionale.

Con la composizione, creare Adapter e creare Adaptee sono momenti distinti: il secondo può essere rimandato alla prima richiesta. Con un Class Adapter l'oggetto contiene invece anche la parte ereditata, inizializzata durante la sua stessa costruzione. Esporre entrambi i protocolli è possibile anche con composizione, se si progettano appositamente i due contratti. In Java ciò non rende un Object Adapter sottotipo della classe Adaptee: aggiungere metodi con nomi uguali non equivale a ereditarne il tipo. La compatibilità va verificata rispetto ai tipi richiesti dai due client.

8.2.1 Delega nella sequenza delle chiamate

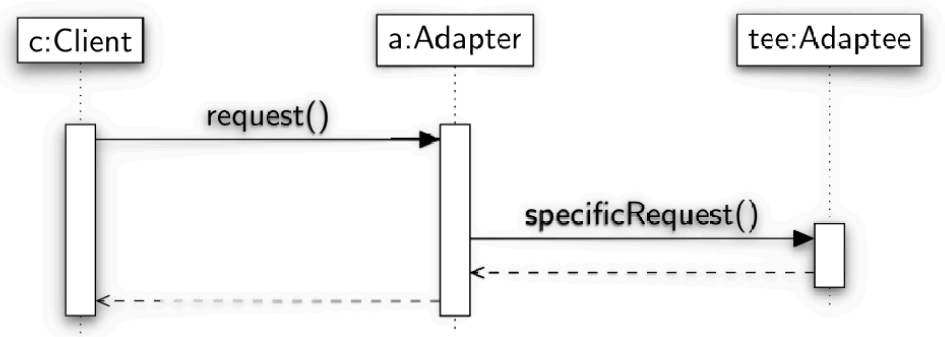


Figura 8.3: Il client invoca request sul Target; Adapter traduce la chiamata in specificRequest sull'Adaptee.

L'indirezione aggiunge una chiamata, ma mantiene separati il contratto dell'applicazione e quello della libreria. I controlli di precondizione avvengono prima della delega e quelli di postcondizione dopo il risultato; devono conservare le garanzie promesse dal Target.

8.2.2 Esempio Label e LabelC

`Label` svolge il ruolo di Object Adapter: adatta le chiamate dell'interfaccia che il client usa ai metodi di un oggetto di libreria. L'adattatore conserva l'Adaptee come campo e, nell'operazione Target, esegue la conversione necessaria prima di delegare. La versione `LabelC` è invece Class Adapter: è sottoclasse dell'Adaptee e implementa Target. L'esempio mostra il punto da verificare nella scelta: la prima versione può ricevere un oggetto adattato, la seconda lega Adapter alla sua superclasse.

```
interface ILabel {
    String getLabel(); String getPrefix(); void oneUp();
}
class LabelServer {
    private int labelNum = 1;
    private final String[] labelPrefix;
    LabelServer(String prefix) {
        labelPrefix = new String[]{prefix, "", ""};
    }
    public String nextLabel(int i) {
        if (checkRange(i)) return labelPrefix[i] + labelNum++;
        return null;
    }
    public String strip(int i) {
        return checkRange(i) ? labelPrefix[i] : null;
    }
    public void skip(int count) { labelNum += count; }
    public void setPrefix(String prefix, int i) {
        if (checkRange(i)) labelPrefix[i] = prefix;
```

```
    }
    private boolean checkRange(int i) {
        return i >= 0 && i < labelPrefix.length;
    }
}
class Label implements ILabel {
    private LabelServer ls;
    private void init() {
        if (ls == null) ls = new LabelServer("MyLabel");
    }
    public String getLabel() { init(); return ls.nextLabel(0); }
    public String getPrefix() { init(); return ls.strip(0); }
    public void oneUp() { init(); ls.skip(1); }
}
class LabelC extends LabelServer implements ILabel {
    LabelC() { super("MyLabel"); }
    public String getLabel() { return nextLabel(0); }
    public String getPrefix() { return strip(0); }
    public void oneUp() { skip(1); }
}
```

`Label` traduce nomi e parametri: `getLabel` diventa `nextLabel(0)`, `getPrefix` diventa `strip(0)`, `oneUp` diventa `skip(1)`. `init` crea il server soltanto al primo uso; nel diagramma di sequenza questa creazione appartiene al frammento opzionale `opt [ls==null]`. Con `ILabel x=new Label()`, la prima `x.getLabel()` restituisce `MyLabel1`; `x.oneUp()` salta il numero 2 e la successiva etichetta è `MyLabel3`. Il prefisso è `MyLabel`. Il server condivide il contatore fra i tre indici di prefisso e rifiuta indici fuori da $0, 1, 2$.

`LabelC` realizza lo stesso Target ereditando il server: è Class Adapter, senza riferimento delegato né inizializzazione lazy. Può essere usato sia come `ILabel` sia come `LabelServer`; questa è la variante a due vie. L'Object Adapter espone soltanto il contratto che decide di pubblicare e può scegliere o sostituire l'istanza adattata.

8.3 Facade: accesso di alto livello a un sottosistema

L'intento di *Facade* è fornire un'interfaccia unificata a un insieme di interfacce di un sottosistema, definendo un punto di accesso di alto livello che ne renda l'uso più semplice. Un *sottosistema* è qui un gruppo di classi che realizza funzioni correlate. Quando il client deve conoscere molte classi e coordinare molte chiamate, anche individuare l'interfaccia essenziale può diventare difficile.

Facade è una classe che espone poche operazioni orientate all'uso del client e, al loro interno, crea o usa gli oggetti del sottosistema e ne coordina le invocazioni. Il client interagisce con Facade; Facade non trasforma un'interfaccia incompatibile, come Adapter, ma semplifica un insieme di interfacce già esistenti. La complessità non scompare: viene centralizzata e incapsulata nella facciata, mentre il sottosistema può restare invariato.

Facade riduce l'accoppiamento fra client e sottosistema: finché il contratto esterno rimane stabile, una modifica interna può essere assorbita dalla facciata. La riduzione delle ricompilazioni dovute alle dipendenze interne dipende dal linguaggio e dalla build; in C++, per esempio, gli header interni possono essere inclusi dalla sola implementazione della facciata. L'accesso diretto al sottosistema può restare disponibile ai client avanzati. Se deve essere vietato, occorrono regole di visibilità effettive, come classi annidate private o accessi controllati tramite package e moduli: né l'annidamento da solo né la separazione in cartelle costituiscono un vincolo sufficiente.

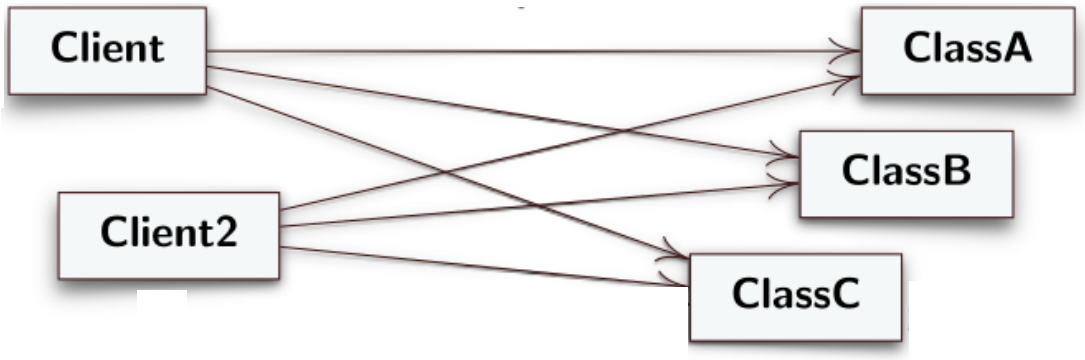


Figura 8.4: Prima della facciata ciascun client conosce più classi del sottosistema.

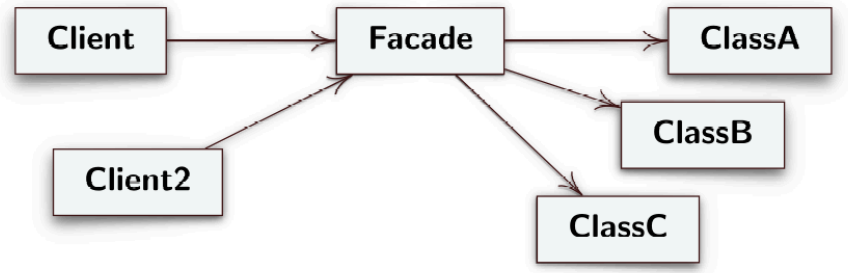


Figura 8.5: La facciata concentra l'accesso al sottosistema.

Se tre classi espongono rispettivamente tre, cinque e sei metodi, una facciata che si limiti a esporre tutte le quattordici operazioni non offre necessariamente la semplificazione cercata. Deve proporre operazioni di livello utile al client, coordinando internamente i dettagli.

8.3.1 Esempi Translator e Registro

Translator: coordinare due dizionari

La Facade espone `addEnglish(String)` e `multiPrinting()`; `getText()` appartiene invece alla classe interna `English`. I dizionari conservano quattro espressioni corrispondenti nella medesima posizione:

```
import java.util.List;

class English {
    private String text = " ";
    private final List<String> d =
        List.of("Alright", "Hello", "Understood", "Yes");
    boolean test(String s) { return d.contains(s); }
    void add(String s) { text = text + " " + s; }
    String getText() { return text; }
    int getIndex(String s) { return d.indexOf(s); }
    void printText() { System.out.println(text); }
}
class Italian {
    private String text = " ";
```

```
    private final List<String> d =
        List.of("Va bene", "Ciao", "Capito", "Si");
    void add(int i) { text = text + " " + d.get(i); }
    void printText() { System.out.println(text); }
}
class Translator {
    private final English e = new English();
    private final Italian i = new Italian();
    void addEnglish(String s) {
        if (e.test(s)) {
            e.add(s);
            i.add(e.getIndex(s));
        }
    }
    void multiPrinting() {
        System.out.print("Italiano: "); i.printText();
        System.out.print("English: "); e.printText();
    }
}
```

Il client crea `Translator t`, chiama `t.addEnglish("Hello")` e `t.multiPrinting()`: ottiene Ciao nella riga italiana e Hello in quella inglese. La sequenza interna è controllo con `contains`, aggiunta inglese, ricerca dell'indice con `indexOf`, aggiunta italiana allo stesso indice. Una parola non ammessa non viene aggiunta a nessuno dei due testi. La corrispondenza dipende dall'ordine parallelo dei dizionari; non è una traduzione generale di frasi arbitrarie.

Registro: una richiesta di prestito, più responsabilità

Un prestito richiede un socio abilitato, un libro disponibile e il rispetto del limite dei prestiti contemporanei; cinque libri è un possibile limite applicativo, non una proprietà della Facade. `Registro` riceve `ListaLibri` nel costruttore ed espone `prestaLib(socio,lib): boolean` e `trovaSocio(lib): String`. Le responsabilità del sottosistema sono distinte: `Soci.verifica` controlla l'iscrizione; `BlockList.accettato` controlla l'abilitazione rispetto alla politica dei prestiti; `ListaLibri.libero` controlla la disponibilità e `rimuovi` aggiorna l'elenco; `Prestiti.registra` collega socio e libro e `trovaPersona` ricerca il titolare del prestito.

Il protocollo di `prestaLib` verifica dapprima `socio!=null` e `lib!=null`, restituendo falso in caso contrario. Quindi esegue, nell'ordine, `verifica`, `accettato`, `libero`, `rimuovi` e `registra`. I primi controlli falliti interrompono il flusso; se la rimozione non riesce non si registra un prestito riuscito. I ritorni anticipati rendono leggibile il percorso principale senza annidare tutti gli `if`. `trovaSocio` delega alla ricerca nei prestiti. Il client conosce l'operazione applicativa, mentre Registro conosce l'ordine necessario delle collaborazioni. Gestione di accessi concorrenti e atomicità fra rimozione e registrazione richiedono un contratto ulteriore e non sono garantite dalla sola presenza della Facade.

8.4 Composite: elementi semplici e composizioni ricorsive

L'intento di *Composite* è comporre oggetti in strutture ad albero per rappresentare gerarchie di parti e tutto, facendo trattare al client in modo uniforme gli oggetti singoli e le loro composizioni. Una struttura ad albero è una struttura ricorsiva: un nodo contenitore può avere figli e ciascun figlio può essere a sua volta un contenitore. Senza Composite, il client di un file system dovrebbe distinguere con `if/else` file e cartelle e ripetere tale distinzione a ogni livello; analogo problema si presenta in un editor con figure semplici e gruppi di figure.

I partecipanti sono:

Component il contratto comune degli elementi semplici e composti; dichiara le operazioni che il client può richiedere a entrambi e può realizzare una sola volta quelle comuni alle sottoclassi;

Leaf un elemento semplice, sottoclasse o implementazione di Component, che esegue direttamente l'operazione;

Composite un elemento contenitore che mantiene una collezione di Component, implementa la gestione dei figli e delega ricorsivamente l'operazione a essi;

Client il codice che interagisce attraverso Component.

Per esempio, quando il client invoca `stampaContenuto()` su una Leaf, l'elemento stampa il proprio contenuto; se la riceve un Composite, questo la inoltra a ciascun figlio, eventualmente compiendo attività prima o dopo. La lista deve avere tipo Component, non soltanto Leaf o un particolare Composite, altrimenti non potrebbe rappresentare una profondità arbitraria.

Il contenimento di figli non impone da solo la composizione UML. Se le parti possono vivere indipendentemente, come un elemento trasferibile fra cartelle, la relazione può essere di aggregazione; se una parte appartiene esclusivamente al tutto e ne condivide il ciclo di vita, si usa la composizione. Per esempio, uno stato conservato come parte esclusiva di uno specifico ordine può essere modellato con rombo pieno. La scelta dipende dalla proprietà e dal ciclo di vita effettivi, non dal semplice uso di una lista.

Le operazioni di gestione dei figli, quali `add()`, `remove()` e `getFigli()`, pongono un compromesso. Se sono dichiarate in Component, il client può usarle senza distinguere Leaf e Composite (*trasparenza*), ma su una Leaf non hanno significato e possono fallire o essere ignorate. Se sono dichiarate solo in Composite, il compilatore impedisce di chiamarle su una Leaf (*sicurezza*), ma il client che le deve usare perde uniformità. Un metodo quale `getComposite()`, che restituisce `null` per Leaf e l'oggetto stesso per Composite, non elimina il problema: sostituisce `instanceof` con un controllo su `null`.

Composite permette di aggiungere nuovi tipi di Leaf o Composite senza modificare i client che usano Component, purché le nuove classi ne rispettino il contratto. Questo realizza l'Open/Closed Principle: estendere i tipi disponibili può lasciare stabile il codice che li utilizza.

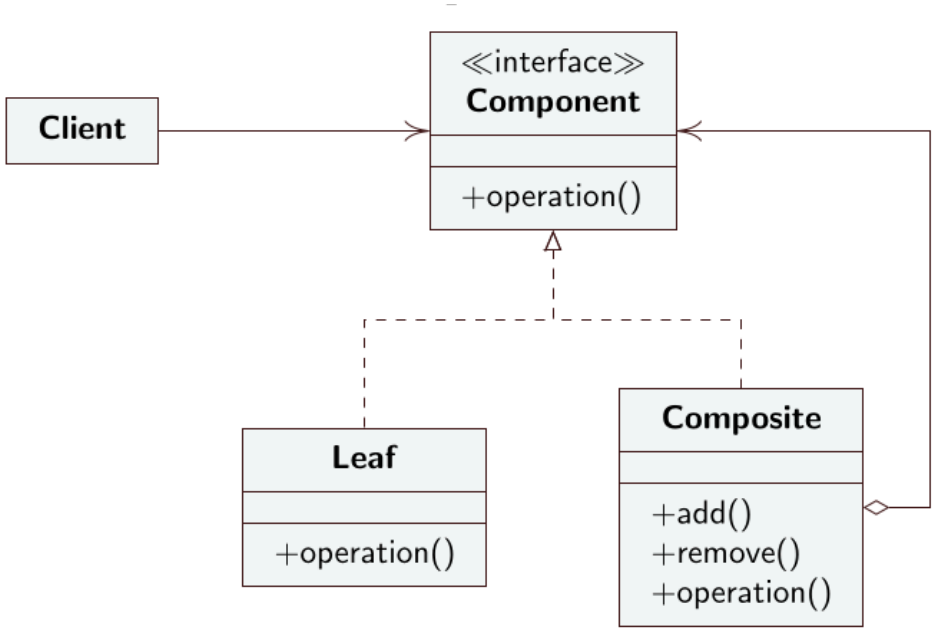


Figura 8.6: Component, Leaf e Composite e molteplicità dei figli.

La lista dei figli appartiene al Composite: inserirla in Component sprecherebbe spazio nelle foglie che non la usano. Il riferimento al padre può appartenere invece al contratto o a una base comune se tutti i nodi devono navigare verso l'alto. Quando si aggiunge, rimuove o sposta un nodo, relazione padre–figlio e cache degli antenati vanno mantenute coerenti. Una *cache* conserva un risultato già calcolato per riutilizzarlo. Con una cache della dimensione, modificare una foglia richiede di invalidare il suo contenitore, poi il padre del contenitore, fino alla radice, per esempio mediante un'operazione `invalidaCache()` richiamata sul padre finché il riferimento non è nullo.

L'ordine dei figli può determinare ordine di disegno, stampa o esecuzione; non va lasciato implicito quando modifica il risultato. Nel modello generico una collezione di Component ammette sia foglie sia altri compositi: una cartella che accetti soltanto file richiede un vincolo aggiuntivo. La limitazione è del modello scelto; non è un divieto generale di progettare varianti con contratti o tipi più restrittivi.

Una nuova Leaf `Shortcut` può essere usata dagli stessi client di File e Cartella se implementa le operazioni di Component. Il suo eventuale riferimento a un altro nodo richiede una politica di visita: seguire i collegamenti può introdurre cicli che non esistono nell'albero di contenimento.

Il rombo UML non dice che il metodo del contenitore debba soltanto inoltrare chiamate: l'operazione del Composite può aggiungere lavoro prima o dopo la visita. Aggregazione e composizione descrivono la relazione fra gli oggetti e il loro ciclo di vita, non il numero di istruzioni del metodo. Se una Leaf è realizzata tramite record, il nome di un componente non soddisfa automaticamente un metodo diverso dell'interfaccia: se il componente si chiama `nome` e Component richiede `descrizione()`, va implementato esplicitamente tale metodo.

8.4.1 Esempi: file, cartelle e figure

Un file system può distinguere `File` e `Cartella`, ma entrambi sono Component. Una cartella conserva Component e può quindi contenere file e ulteriori cartelle; invocare `stampaContenuto()` sulla radice percorre la struttura ricorsivamente. L'editor di figure è equivalente: una `Linea` o un `Box` è una Leaf, mentre un gruppo è Composite; `disegna()` o `sposta()` vale uniformemente per entrambi. Gli esempi chiariscono che "uniformemente" riguarda l'interfaccia del client, non implica che Leaf e Composite eseguano internamente lo stesso algoritmo.

Esercizio: completare un Composite con una Leaf record

Dato il contratto `descrizione()`, si deve realizzare una foglia `Auto` e un contenitore di elementi semplici o composti. Il record può implementare il metodo richiesto con un componente omonimo; se il componente ha un nome diverso, serve un'implementazione esplicita:

```
import java.util.*;

interface Elemento { String descrizione(); }
record Auto(String nome) implements Elemento {
    public String descrizione() { return nome; }
}
class Gruppo implements Elemento {
    private final List<Elemento> figli = new ArrayList<>();
    public void add(Elemento e) { figli.add(e); }
    public void remove(Elemento e) { figli.remove(e); }
    public String descrizione() {
        StringJoiner testo = new StringJoiner(", ", "[", "]");
        for (Elemento e : figli) testo.add(e.descrizione());
        return testo.toString();
```

```
    }
}
```

La scelta di parentesi e separatore rende osservabile la visita, senza cambiare i ruoli del pattern. Una foglia con nome `auto` restituisce quel testo; un gruppo che contiene tale foglia e un altro gruppo mostra il risultato annidato. La lista di `Elemento` permette profondità arbitraria. Le operazioni sui figli sono qui soltanto in `Gruppo`: il client dell'operazione `descrizione` è uniforme, quello che modifica la struttura deve conoscere il contenitore. Per mantenere un albero non si deve aggiungere un gruppo a sé stesso o a un proprio discendente.

8.5 Decorator: responsabilità aggiunte a un oggetto

L'intento di *Decorator* è aggiungere responsabilità a un singolo oggetto dinamicamente, offrendo un'alternativa flessibile alla creazione di sottoclassi. Una *responsabilità* è qui un dato o comportamento aggiuntivo. L'ereditarietà assegna una responsabilità alla classe a design-time: con più estensioni e combinazioni genera molte sottoclassi rigide. Decorator avvolge invece l'oggetto (*wrap*) in un altro oggetto conforme alla stessa interfaccia; le decorazioni possono essere aggiunte, sottratte o annidate a runtime.

I ruoli sono:

Component l'interfaccia comune a oggetti decorabili e decoratori;

ConcreteComponent l'oggetto di base, che implementa Component e non conosce chi lo avvolgerà;

Decorator una classe che implementa Component e conserva un riferimento a un Component; in genere inoltra a esso le richieste;

ConcreteDecorator una sottoclasse di Decorator che aggiunge stato o logica prima e/o dopo la delega.

La doppia relazione di Decorator con Component è essenziale: lo implementa, quindi il client può usare un decorator ovunque si aspetti Component; ne contiene uno, quindi può avvolgere sia un ConcreteComponent sia un altro ConcreteDecorator. Le richieste attraversano quindi gli strati di decorazione fino al componente di base; ciascuno strato può aggiungere il proprio comportamento prima o dopo la delega.

La composizione di decorazioni è ricorsiva e anche la stessa responsabilità può essere aggiunta più volte creando più istanze del medesimo ConcreteDecorator. Ciò evita classi di base gonfie e classi per ogni combinazione, favorendo il principio di Singola Responsabilità: ogni decoratore può avere un compito circoscritto. I limiti sono importanti: l'oggetto risultante non ha né lo stesso tipo concreto né lo stesso riferimento dell'oggetto avvolto, dunque confronti basati su tipo o identità possono non funzionare come prima. Inoltre una lunga catena crea molti piccoli oggetti e deleghe; il debug e la comprensione richiedono di seguire l'ordine degli avvolgimenti.

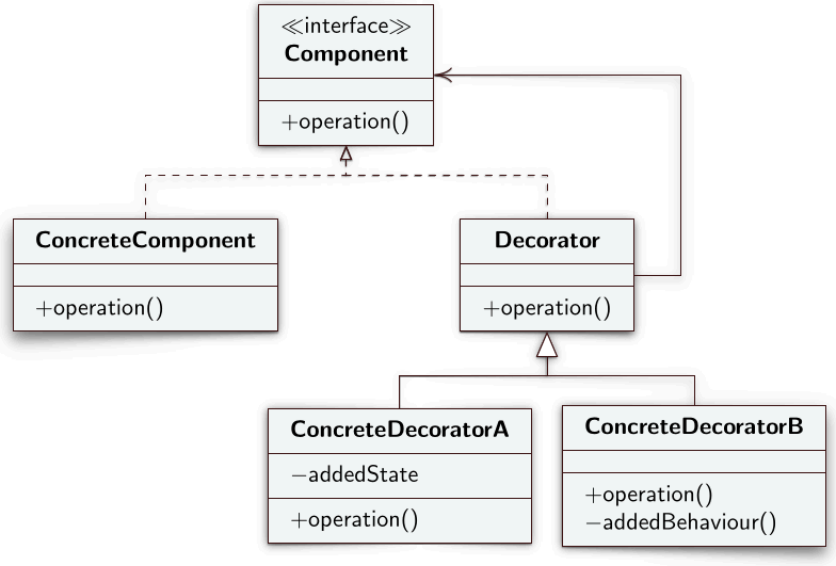


Figura 8.7: Il decoratore è un Component e contiene un Component.

L'aggiunta a runtime avviene componendo istanze di classi il cui codice esiste già: il pattern non genera nuovo codice dentro l'oggetto avvolto. Il ConcreteDecorator conserva il nome e il contratto dell'operazione di Component; può richiamare la delega della superclasse e aggiungere stato o comportamento. Decorator può essere astratto anche quando tutti i suoi metodi hanno un corpo: `abstract` impedisce l'istanziazione diretta e non obbliga a riscrivere i metodi concreti ereditati.

L'ordine di avvolgimento è significativo: applicare uno sconto prima di una maggiorazione può differire dall'applicarlo dopo. Per rimuovere uno strato si deve rendere nuovamente raggiungibile il componente interno; eliminare uno strato interno può richiedere di ricostruire la catena esterna. Ripetere lo stesso decoratore è possibile, ma il contratto deve chiarire se l'effetto ripetuto abbia senso.

Un riferimento già conservato al componente base non viene aggiornato automaticamente quando un altro riferimento riceve il wrapper:

```
Component base = new Testo();
Component prima = base;
Component decorato = new Bordo(base);
// prima e' ancora il componente base.
// Le operazioni aggiuntive si ottengono usando decorato.
```

Per vedere la decorazione il client deve ricevere il riferimento al decoratore o un'indirezione stabile che sia stata aggiornata. Trasparenza di interfaccia non significa identità di oggetto. Il decoratore dovrebbe dipendere dal contratto e dai risultati del componente, senza richiedere di conoscerne l'algoritmo interno; una cooperazione troppo specifica segnala che il contratto potrebbe essere insufficiente.

8.5.1 Esempio: bordo e catena di decorazioni

Per aggiungere un bordo a `Testo`, la soluzione rigida sarebbe introdurre una sottoclasse che possieda sempre il bordo. `DecoratorBordo` contiene invece un Component Testo, inoltra la

richiesta e aggiunge il comportamento del bordo. Poiché Decorator implementa Component, la costruzione concettuale

```
Component c = new ConcreteDecoratorA(
                  new ConcreteDecoratorB(
                      new ConcreteComponent()));
```

è valida: il decoratore più esterno è ancora un Component. L'ordine è osservabile se le responsabilità eseguono attività prima o dopo la delega; una catena va quindi costruita deliberatamente e non confrontata con l'identità del componente interno.

Implementazione della delega e applicazione ai biglietti

```
interface Component { void operation(); }
class ConcreteComponent implements Component {
    public void operation() { System.out.print("componente"); }
}
abstract class Decorator implements Component {
    private final Component innerC;
    Decorator(Component c) { innerC = c; }
    public void operation() { innerC.operation(); }
}
class ConcreteDecorator extends Decorator {
    ConcreteDecorator(Component c) { super(c); }
    public void operation() {
        super.operation();
        addedBehavior();
    }
    private void addedBehavior() { System.out.print(" + aggiunta"); }
}
```

Il campo finale conserva il collaboratore; `super.operation()` raggiunge la delega già implementata nella base, che arriva al componente avvolto. La stampa scelta rende visibile l'ordine: prima il componente, poi l'aggiunta. Annidando due decoratori si esegue prima il componente e poi ciascuna aggiunta dall'interno verso l'esterno.

Per i biglietti il contratto `Prodotto` offre `printNome()` e `getCosto()`. `Biglietto` è il componente; `Decor` conserva un Prodotto e delega entrambe le operazioni. `Formatta` modifica la presentazione di `printNome`, mentre `Sconto` modifica il risultato di `getCosto`. `Configura.getBiglietto` costruisce la catena base, `getBiglScontato` quella con lo strato di sconto. Una configurazione `new Formatta(new Sconto(biglietto))` porta la richiesta di costo da Formatta a Sconto e poi a Biglietto, applicando lo sconto al ritorno. Il formato della stampa e la percentuale non sono fissati dalla struttura: devono essere parametri o regole del dominio. Questo esempio separa due responsabilità combinabili senza creare una sottoclasse per ciascuna combinazione.

8.6 Bridge: due gerarchie che variano indipendentemente

L'intento di *Bridge* è disaccoppiare un'astrazione dalla sua implementazione affinché le due possano variare indipendentemente. In questo pattern *astrazione* non significa necessariamente una classe astratta Java: è l'interfaccia di alto livello che dice *che cosa* il client vuole fare. *Implementazione* non significa soltanto una classe concreta: è l'insieme delle primitive di basso livello che dice *come* eseguirlo. Le due dimensioni sono *ortogonali* quando cambiano per ragioni indipendenti.

Se forme (rettangoli e cerchi) e piattaforme di disegno (P1 e P2) fossero fuse mediante sola ereditarietà, ogni combinazione richiederebbe una classe, quale `V1Rettang`. La scelta della piattaforma sarebbe permanente e non cambiabile a runtime. Bridge separa invece le due gerarchie: Forma conserva un riferimento a `DrawingAPI`, la cui interfaccia è implementata dalle piattaforme adattate. La composizione permette a un oggetto Forma di delegare le primitive alla piattaforma ricevuta e, se opportuno, sostituirla.

I partecipanti sono:

Abstraction definisce le operazioni di alto livello per il client e conserva un riferimento a Implementor; inoltra a esso il lavoro di basso livello;

RefinedAbstraction estende Abstraction con operazioni specifiche, continuando a usare l'Implementor ereditato;

Implementor definisce le operazioni primitive richieste dall'astrazione; la sua interfaccia non deve coincidere con quella di Abstraction;

ConcreteImplementor realizza le primitive per una particolare piattaforma.

Per il rettangolo, per esempio, RefinedAbstraction può chiamare più volte una primitiva di linea dell'Implementor per disegnare i quattro lati. La delega generica è `operation()` che richiama `impl.operationImpl()`: la prima appartiene all'astrazione, la seconda al contratto dell'implementazione. Il client deve conoscere Abstraction, non i ConcreteImplementor: ciò realizza *information hiding*, cioè nasconde i dettagli che non gli servono.

Bridge permette di aggiungere una forma senza modificare le piattaforme e una piattaforma senza modificare le forme, purché il contratto delle primitive sia sufficiente. Consente inoltre di configurare o sostituire l'implementazione a runtime. Quando piattaforme preesistenti offrono operazioni incompatibili, i ConcreteImplementor possono adattarle al contratto comune: Bridge organizza le dimensioni indipendenti, Adapter traduce le interfacce concrete.

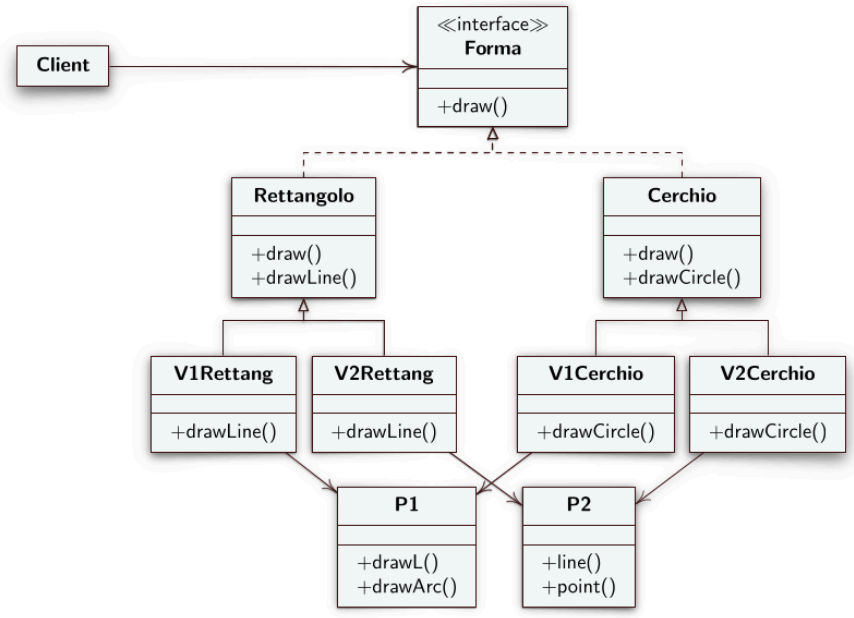


Figura 8.8: La struttura iniziale fonde forma e piattaforma in una classe per ogni combinazione.

Siano m il numero di varianti dell'astrazione, per esempio le forme, e n il numero di varianti dell'implementazione, per esempio le piattaforme. Fondere le due dimensioni in una gerarchia

richiede una combinazione concreta per ogni coppia:

$$N_{\text{ereditarieta}} = mn. \tag{8.1}$$

Con Bridge le due gerarchie restano separate e il numero concettuale di classi varianti è:

$$N_{\text{bridge}} = m + n. \tag{8.2}$$

I valori $N_{\text{ereditarieta}}$ e N_{bridge} confrontano soltanto le varianti dovute alle due dimensioni, escludendo le interfacce e le classi base comuni. Con due forme e tre piattaforme, la prima soluzione richiede sei combinazioni; la seconda, cinque varianti distribuite sulle due gerarchie, oltre alle strutture comuni e agli eventuali adattatori. Il vantaggio principale resta l'indipendenza delle estensioni.

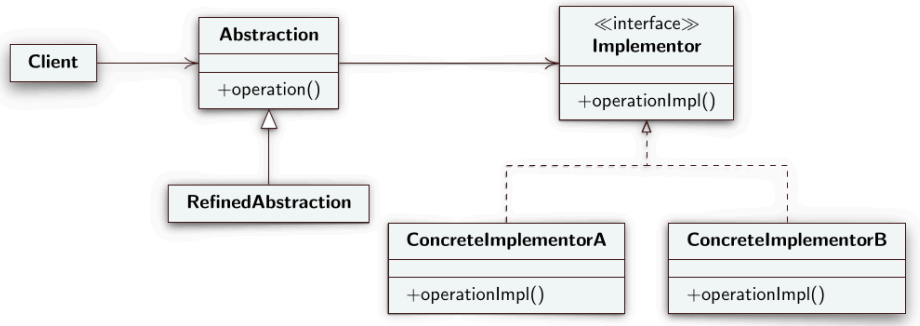


Figura 8.9: Abstraction mantiene un Implementor; le due gerarchie possono essere estese separatamente.

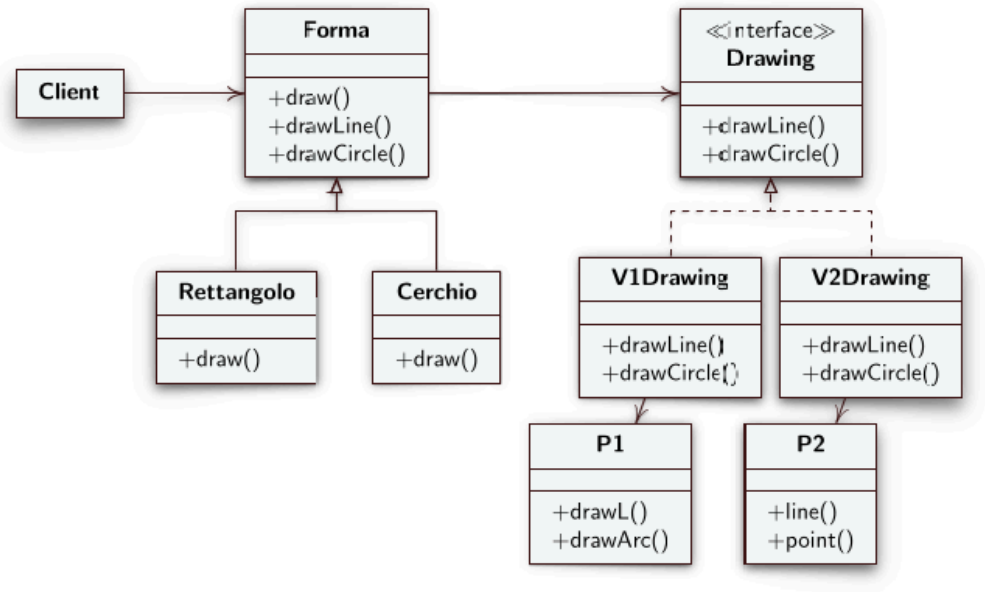


Figura 8.10: Le gerarchie Forma e Drawing sono collegate; V1Drawing e V2Drawing adattano le piattaforme P1 e P2.

`Drawing` espone `drawLine()` e `drawCircle()`; P1 dispone di `drawL()` e `drawArc()`, P2 di `line()` e `point()`. I ConcreteImplementor traducono queste primitive, svolgendo anche Adapter. Dal punto di vista dell'Adapter, Forma è Client, Drawing è Target, V1Drawing è Adapter e P1 è Adaptee. Il Bridge gestisce invece la variazione indipendente delle due gerarchie.

Le RefinedAbstraction possono avere operazioni specifiche; rimangono indipendenti dalle piattaforme finché le primitive di Implementor sono sufficienti. Se una nuova forma richiede una primitiva non esprimibile con il contratto esistente, anche Implementor e le implementazioni devono evolvere: l'indipendenza non elimina la necessità di un contratto adeguato.

Lo stesso problema si presenta se Studente, Registro e Professore devono usare piattaforme AWT o WEB. Legare ogni astrazione alla piattaforma porta a sei varianti, che diventano nove con una terza piattaforma. Il Bridge rende espliciti i due motivi di variazione: entità del dominio e servizi della piattaforma.

8.6.1 Quattro lati espressi mediante una primitiva comune

```
interface Drawing {
    void drawLine(int x1, int y1, int x2, int y2);
}
class Forma {
    private Drawing impl;
    public void setImplementor(Drawing imp) { impl = imp; }
    protected void drawLine(int x1, int y1, int x2, int y2) {
        impl.drawLine(x1, y1, x2, y2);
    }
}
class Rettangolo extends Forma {
    private final int a, b, c, d;
    Rettangolo(int xi, int yi, int xf, int yf) {
        a = xi; b = yi; c = xf; d = yf;
```

```
    }
    public void draw() {
        drawLine(a,b,c,b); drawLine(a,b,a,d);
        drawLine(c,b,c,d); drawLine(a,d,c,d);
    }
}
```

Il client deve chiamare `setImplementor` con un Drawing valido prima di `draw`. `Rettangolo` conosce gli estremi geometrici e le quattro primitive; la concreta di Drawing traduce ciascuna primitiva nell'API grafica. Cambiare backend mantenendo l'interfaccia non richiede di modificare il codice dell'astrazione né dei client che dipendono solo da essa. In Java ciò limita anche le dipendenze di compilazione: una modifica interna a una concreta non impone di riscrivere o ricompilare concettualmente tutte le forme; un cambiamento al contratto comune può invece propagarsi.

Un'estensione distingue anche i punti e i vertici: `Punto` espone `getX`, `getY` e `move`; `Rettangolo` e `Triangolo` conservano vertici accessibili con `getVertex` e realizzano lo spostamento agendo su di essi. `Forma` coordina `draw`, `drawLinea`, `drawPunto` e `setLook`; Drawing espone ora sia `drawLine` sia `drawPoint`. `V1Drawing` traduce verso `StdDraw`, `V2Drawing` verso la piattaforma alternativa. L'estensione geometrica e quella del backend restano due assi distinti; aggiungere una primitiva comune richiede invece di adeguare il contratto e i suoi implementatori.

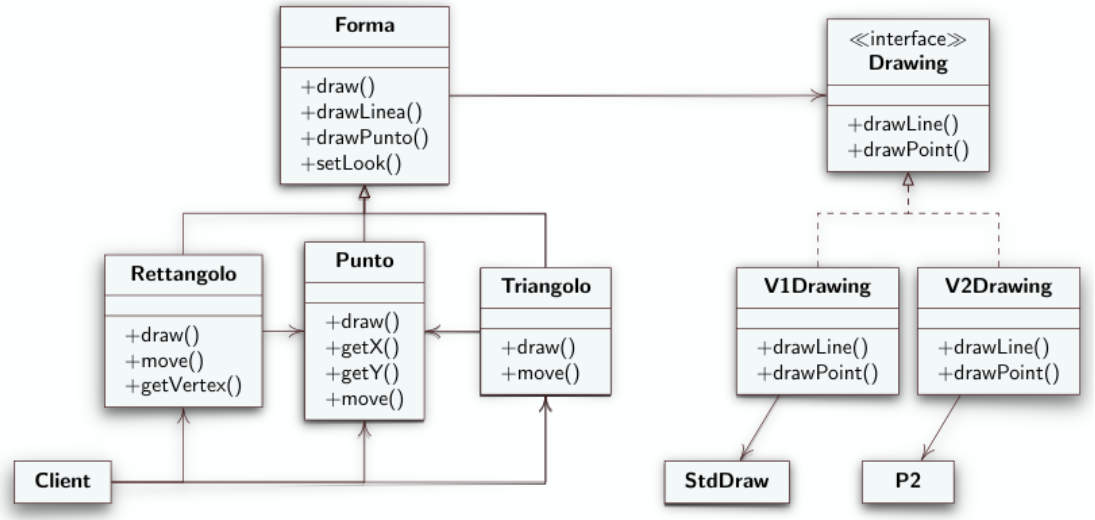


Figura 8.11: Vertici, primitive di linea e punto e due implementazioni grafiche indipendenti dalle forme.

Capitolo 9

Design pattern comportamentali e MVC

9.1 Algoritmi, comunicazione e distribuzione delle responsabilità

I *design pattern comportamentali* organizzano algoritmi, comunicazione e distribuzione delle responsabilità fra oggetti. Specificano chi esegue un comportamento, quando lo esegue e come gli oggetti cooperano attraverso contratti, evitando che ogni partecipante conosca i dettagli di tutti gli altri.

MVC (*Model–View–Controller*) affronta un problema di ampiezza diversa: è un pattern architetturale che organizza un'applicazione interattiva in dati e logica, presentazione e gestione dell'input. Può usare collaborazioni comportamentali, in particolare Observer, all'interno di questa separazione di responsabilità.

Un *algoritmo* è una sequenza ordinata di passi per ottenere un risultato; *incapsularlo* significa rappresentarlo dietro un'operazione o un oggetto, così che chi lo usa non dipenda dai dettagli. I pattern del capitolo risolvono famiglie differenti di problemi:

Template Method fissa lo scheletro di un algoritmo nella superclasse e rinvia passi variabili alle sottoclassi.

Strategy separa algoritmi alternativi intercambiabili dal contesto che li usa.

State sostituisce i rami condizionali dipendenti dallo stato con oggetti che rappresentano gli stati.

Observer realizza una dipendenza uno-a-molti: un cambiamento di un soggetto notifica automaticamente gli osservatori registrati.

Mediator concentra in un mediatore la coordinazione fra molti oggetti collaboranti.

Chain of Responsibility fa percorrere una richiesta a una catena di possibili gestori, disaccoppiando mandante e ricevente.

Command rende una richiesta un oggetto, quindi la rende memorizzabile, accodabile, componibile e, se progettato a questo fine, annullabile.

Iterator separa il percorso di una collezione dalla sua rappresentazione interna.

Null Object sostituisce un riferimento assente con un oggetto conforme al contratto che esegue un comportamento neutro.

La scelta non dipende dalla somiglianza del diagramma UML. In particolare State e Strategy hanno entrambi un Context che delega a un'interfaccia, ma il primo rappresenta l'evoluzione

interna di una macchina a stati; il secondo sceglie un algoritmo alternativo. Occorre perciò partire dall'intento e dalla causa del cambiamento.

9.2 Template Method: scheletro fisso e passi specializzati

L'intento di *Template Method* è definire in una operazione lo scheletro di un algoritmo e rimandare alle sottoclassi la definizione di alcuni passi, senza cambiare l'ordine complessivo. Si applica quando più classi condividono i passi necessari ma differiscono nei dettagli: duplicare l'intero algoritmo in ogni classe renderebbe incoerenti le modifiche comuni.

`AbstractClass` dichiara le *operazioni primitive*, cioè i passi indispensabili che le sottoclassi devono implementare, e definisce il template method. `ConcreteClass` implementa tali primitive. Un *hook* è invece un'operazione di aggancio con implementazione predefinita, spesso vuota: la sottoclasse può ridefinirla per personalizzare un punto dell'algoritmo ma non è obbligata. Il template può anche invocare operazioni concrete della superclasse, operazioni sovrascritte della sottoclasse, operazioni di classi client e metodi factory. Un *metodo factory* è qui un passo che crea un oggetto il cui tipo concreto è deciso dalla sottoclasse.

Il vantaggio è il riuso della parte invariabile, particolarmente utile nelle librerie e nei framework. Un *framework* è un'infrastruttura che chiama il codice applicativo nei punti previsti: qui vale il principio di Hollywood, "non chiamarci, noi ti chiameremo", perché la superclasse controlla il flusso e invoca i passi della sottoclasse. Il prezzo è il legame di ereditarietà: la variazione è fissata dalla classe concreta dell'oggetto e non può essere sostituita semplicemente cambiando un riferimento durante l'esecuzione.

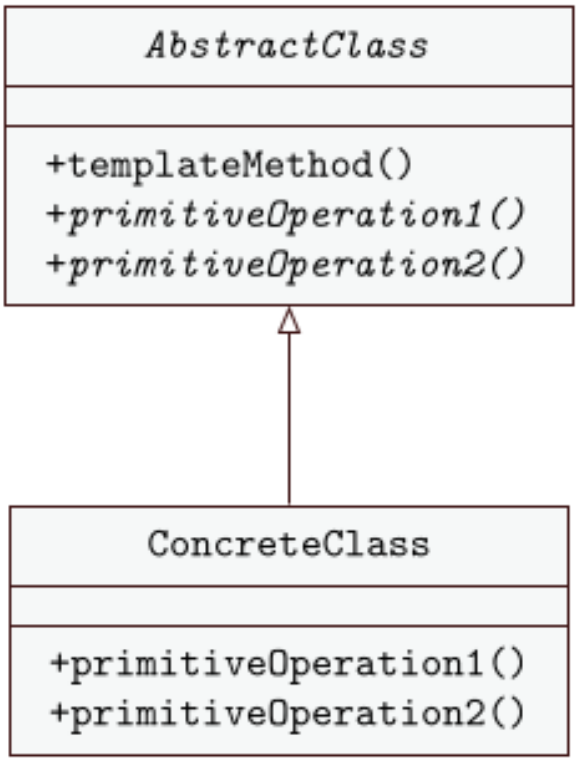


Figura 9.1: AbstractClass definisce il template e le primitive; ConcreteClass implementa i passi.

Un factory method usato come passo del template può essere una primitiva astratta o avere un prodotto predefinito quando esiste una scelta sensata. Il tipo di ritorno Product non

garantisce che sia possibile istanziarlo direttamente: può essere astratto o un'interfaccia.

Rendere il template `final` è una possibile decisione per proteggere la sequenza dai cambiamenti delle sottoclassi; la keyword non è necessaria in ogni applicazione del pattern.

9.2.1 Esempio: apertura di un documento

Nell'esempio Template Method, `Application.openDocument()` determina la sequenza per aprire e leggere un documento; `MyApplication` decide come creare e verificare il documento, mentre `MyDocument` realizza i dettagli di lettura.

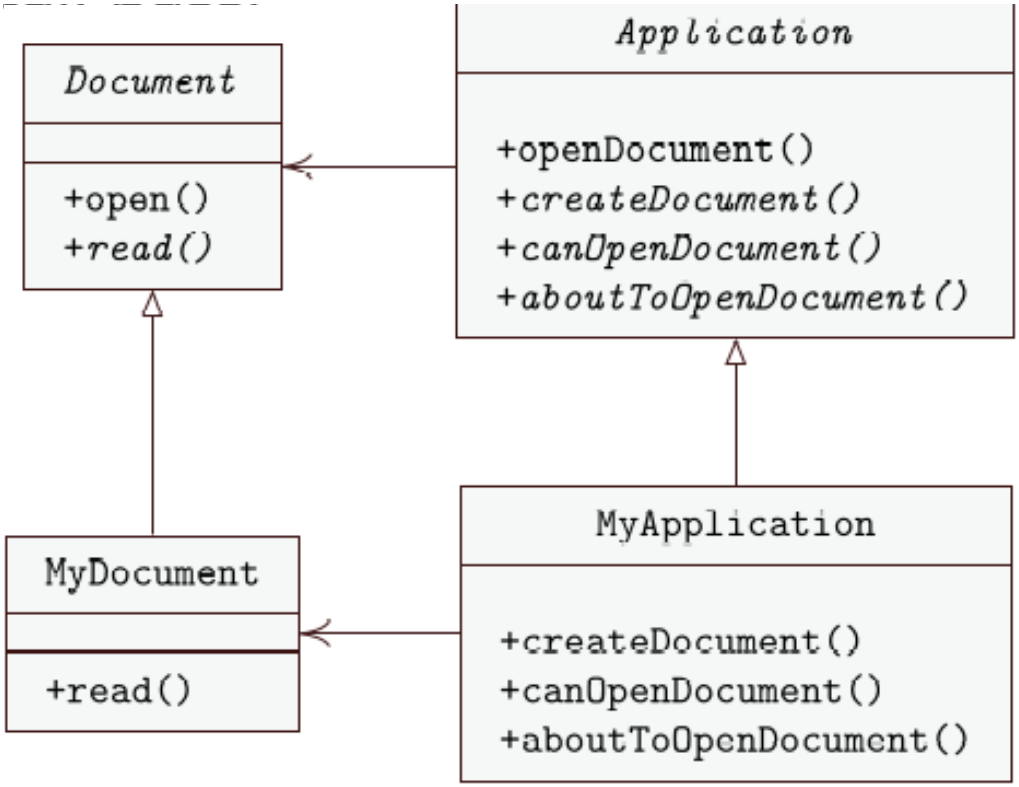


Figura 9.2: Application e Document mantengono la struttura comune; MyApplication e MyDocument specializzano i passi.

```java
public abstract class AbstractClass {
    public void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
    }
    protected abstract void primitiveOperation1();
    protected abstract void primitiveOperation2();
}
```

```java
public class ConcreteClass extends AbstractClass {
    @Override
    protected void primitiveOperation1() {
        // codice per l'operazione
    }
    @Override
    protected void primitiveOperation2() {
    }
}
```

Figura 9.3: Implementazione del template che invoca primitiveOperation1 e primitiveOperation2 ridefinite nella sottoclasse.

9.3 State: comportamento dipendente dallo stato corrente

State permette a un oggetto di alterare il comportamento quando muta il suo stato interno, facendo apparire al client che abbia cambiato classe. Il problema tipico è un *type code*, cioè una variabile enumerativa o numerica che rappresenta lo stato, su cui molti metodi ripetono

`if/else` o `switch`. Questa duplicazione viola DRY (*Don't Repeat Yourself*, non ripetere la stessa conoscenza) e rende costoso aggiungere stati.

Il `Context` è l'interfaccia visibile al client e mantiene il riferimento persistente allo `State` corrente; delega una richiesta, per esempio `request()`, a `currentState.handle(...)`. `State` dichiara le operazioni comuni e ogni `ConcreteState` incapsula il comportamento di uno stato. Il Context può passare sé stesso a `handle` affinché lo stato legga dati o esegua `setState`. Le transizioni possono essere centralizzate nel Context, più facilmente leggibili come insieme, oppure distribuite nei ConcreteState, che conoscono il proprio successore. In entrambi i casi la transizione sostituisce un riferimento a State, non confronta un type code.

State localizza comportamenti e transizioni e consente nuove classi di stato, al costo di più classi. Il refactoring associato è *Replace Type Code with State*, caso di *Replace Conditional with Polymorphism*: ogni ramo dello `switch` diventa un'implementazione polimorfica. Nell'esempio del tornello, `LockedState` sa che l'inserimento di un gettone porta a `Unlocked`; affidargli la transizione evita che Context ricostruisca un'altra catena di condizioni per riconoscere lo stato corrente.

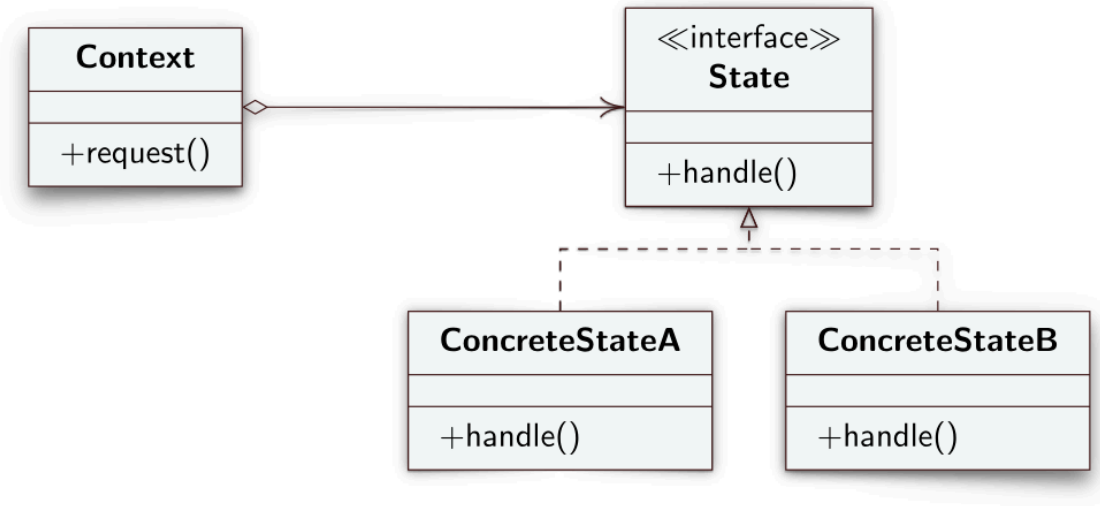


Figura 9.4: Context delega allo State corrente e i ConcreteState forniscono i comportamenti.

Un ConcreteState può cambiare il contesto ricevuto come parametro oppure restituire il nuovo State affinché il Context lo assegni. Il seguente schema rende esplicita la seconda possibilità:

```
interface Stato {
    Stato evento();
}
final class Contesto {
    private Stato corrente;
    Contesto(Stato iniziale) {
        corrente = java.util.Objects.requireNonNull(iniziale);
    }
    void evento() {
        corrente = corrente.evento();
    }
}
```

Le implementazioni devono restituire uno stato valido, eventualmente `this` quando non cambia. La logica delle transizioni può stare nel Context o nei ConcreteState; non è contemporaneamente obbligatoria in entrambi. L'eventuale condivisione di istanze di stato richiede che non contengano dati mutabili specifici di un singolo contesto.

9.3.1 Transizioni e concatenamento delle chiamate

Un'operazione di stato può restituire il successore da installare nel contesto. Se i tipi restituiti espongono le operazioni successive, le chiamate possono essere concatenate in una forma *fluent*. Si consideri uno stato concreto `Bloccato`, la cui operazione `inTesta(s)` riceve l'argomento `s` e restituisce uno stato. L'espressione

```
return new Bloccato().inTesta(s);
```

equivale ai passi:

```
var b = new Bloccato();
var successivo = b.inTesta(s);
return successivo;
```

Il risultato è quello di `inTesta(s)`, non necessariamente l'istanza appena costruita. Un ulteriore `.paga()` è possibile soltanto se il tipo restituito espone tale operazione: il risultato della catena è quello dell'ultima chiamata. La notazione fluente non impone quindi che tutti i metodi restituiscano `this`.

9.3.2 Esempio: modalità di scrittura di un libro

`Libro` conserva un collaboratore `Display` che realizza `scrivi`; `Colonna` e `SingolaParola` ne sono implementazioni alternative. I metodi `modeColonna()` e `modeParola()` sostituiscono Display, evitando di ripetere `switch(mode)` nelle operazioni del libro. La struttura realizza State quando Display rappresenta lo stato del contesto e le sue evoluzioni; se rappresenta soltanto un algoritmo di formattazione scelto dal client, l'intento è invece quello di Strategy. Il criterio decisivo è il significato del collaboratore, non il solo diagramma delle classi.

Dal selettore numerico agli oggetti Display

La versione condizionale `LibroPrimaDiState` conserva `mode=2`; `mostra` esegue uno `switch`, con caso 1 per la singola parola e caso 2 per la colonna, mentre `setMode(int)` sostituisce il codice. Il refactoring trasferisce i due corpi in classi separate:

```
import java.util.*;

interface Display { void scrivi(List<String> testo); }
class Libro {
    private final String testo =
        "Darwin's _Origin of Species_ persuaded the world that the " +
        "difference between different species of animals and plants " +
        "is not the fixed immutable difference that it appears to be.";
    private final List<String> lista =
        Arrays.asList(testo.split("[\\s+]+"));
    private Display mode = new Colonna();
    public void mostra() { mode.scrivi(lista); }
    public void modeColonna() { mode = new Colonna(); }
    public void modeParola() { mode = new SingolaParola(); }
}
class Colonna implements Display {
    private final int numCar = 38, numRighe = 12;
    public void scrivi(List<String> testo) {
        int riga = 0, col = 0;
        for (String p : testo) {
            if (col + p.length() > numCar) {
                System.out.println(); riga++; col = 0;
```

```
            }
            if (riga == numRighe) break;
            System.out.print(p + " ");
            col += p.length() + 1;
        }
    }
}
class SingolaParola implements Display {
    public void scrivi(List<String> testo) {
        int maxLung = 0;
        for (String p : testo) maxLung = Math.max(maxLung, p.length());
        System.out.println();
        for (String p : testo) {
            int sinistra = (maxLung - p.length()) / 2;
            int destra = maxLung - p.length() - sinistra;
            System.out.print(" ".repeat(sinistra) + p + " ".repeat(destra));
            try { Thread.sleep(300); }
            catch (InterruptedException e) {
                Thread.currentThread().interrupt(); break;
            }
            System.out.print("\b".repeat(maxLung));
        }
        System.out.println();
    }
}
```

Il client crea Libro, chiama `mostra`, poi `modeParola` e di nuovo `mostra`. Il contesto conserva lo stesso testo ma delega al nuovo collaboratore. La regex divide su sequenze di spazi bianchi o del carattere `+`; per dividere soltanto sugli spazi bianchi basta `"\\s+"`.

`Colonna` va a capo quando la parola supererebbe 38 caratteri e interrompe dopo 12 righe; il semplice algoritmo non spezza parole più lunghe della larghezza e conta anche lo spazio stampato dopo ciascuna parola. `SingolaParola` calcola la larghezza massima, centra ciascuna parola, attende 300 millisecondi e tenta di riportare indietro il cursore. La divisione intera assegna alla destra l'eventuale spazio dispari: incrementare lo spazio destro soltanto per parole di lunghezza dispari sarebbe sufficiente solo con larghezza massima pari. Il carattere backspace non garantisce cancellazione visiva in ogni terminale; l'effetto dipende dal dispositivo di uscita. Una modalità a due colonne sarebbe un'ulteriore implementazione di Display, con un proprio algoritmo di impaginazione.

Nella variante di State in cui il Context decide la transizione, un'operazione dello stato può restituire un booleano che segnala esito o condizione raggiunta; il Context lo interpreta e aggiorna lo stato. È un'alternativa alla restituzione diretta dello State successivo, non una regola secondo cui ogni operazione debba cambiare stato.

9.4 Strategy: algoritmi alternativi intercambiabili

Strategy incapsula invece una famiglia di algoritmi alternativi. `Strategy` è il contratto dell'algoritmo, `ConcreteStrategy` ne è una variante e `Context` conserva e delega a una Strategy. Si applica quando classi correlate differiscono soprattutto per un comportamento, quando il Context deve essere configurabile con una fra più varianti, oppure quando varianti dello stesso algoritmo offrono compromessi diversi nell'uso delle risorse. A differenza di Template Method, la variazione usa composizione e può essere sostituita a runtime. A differenza di State, normalmente il client o il Context sceglie dall'esterno la strategia in base a configurazione o necessità; le strategie non rappresentano una sequenza di stati e non si passano il controllo l'una all'altra. La struttura può coincidere, ma l'intento resta il criterio decisivo. Il vantaggio

è aggiungere algoritmi senza modificare Context ed evitare condizionali; il costo è che il client deve conoscere differenze rilevanti fra strategie e che un contratto troppo ampio può trasferire parametri inutili alle varianti più semplici.

State e Strategy possono entrambi cambiare collaboratore a runtime. La distinzione è l'intento: evoluzione dello stato del contesto rispetto a scelta di un algoritmo. ConcreteStrategy normalmente non conosce le strategie alternative; ConcreteState può conoscere gli stati successivi e guidare la transizione. È una convenzione di responsabilità, non un divieto sintattico del linguaggio.

Una selezione concreta di Strategy riguarda l'ordinamento: secondo la dimensione dei dati e i compromessi richiesti, il contesto può scegliere un algoritmo diverso e sostituirlo nuovamente durante l'esecuzione. Gli algoritmi devono rispettare lo stesso contratto di ordinamento; il cambio non autorizza a cambiare il risultato richiesto.

9.5 Observer: notificare i dipendenti senza conoscerne i tipi

L'intento di *Observer*, detto anche *Publisher–Subscribe*, è definire una dipendenza uno-a-molti: quando un oggetto cambia stato, tutti i suoi dipendenti sono notificati e aggiornati automaticamente. Senza il pattern, un oggetto `Dati` dovrebbe chiamare esplicitamente `aggiorna()` su tabella, grafico e monitor; conoscerebbe ogni vista concreta e richiederebbe una modifica per aggiungerne o rimuoverne una.

`Subject` mantiene una collezione di `Observer` e offre `attach`, `detach` e `notify`. `Observer` dichiara `update`; `ConcreteSubject` conserva lo stato di interesse e lo modifica, per esempio mediante `setState`; `ConcreteObserver` mantiene il riferimento al subject e lo stato locale da rendere consistente. Quando il subject è coerente dopo la modifica, `notify` itera sugli iscritti e invoca `update`. L'asterisco nel diagramma UML indica molteplicità: a un Subject sono associati zero o più Observer.

Nel modello *pull*, la notifica comunica soltanto che è avvenuto un cambiamento e l'Observer interroga il Subject con, per esempio, `getState`. È flessibile perché ogni osservatore legge solo i dati necessari, ma richiede il riferimento al Subject. Nel modello *push*, `update` riceve direttamente i dati modificati: l'Observer non interroga il Subject, ma il Subject deve conoscere quali dati servano agli osservatori. Il modello pull lascia maggiore libertà a osservatori con esigenze diverse; la scelta fra i due modelli dipende anche dal costo delle letture e dal contenuto della notifica.

Il disaccoppiamento permette di aggiungere osservatori senza modificare Subject, ma non elimina i costi: la notifica è un broadcast, può provocare aggiornamenti inattesi e molte operazioni.

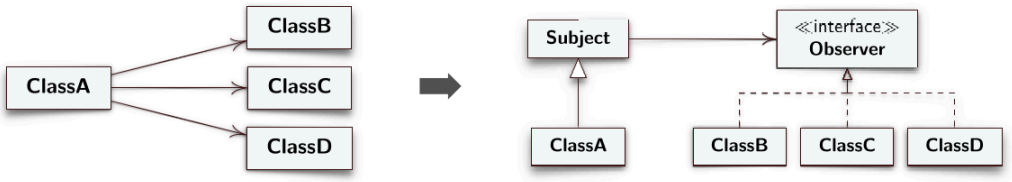


Figura 9.5: Le dipendenze dirette da ciascun osservatore vengono sostituite dal contratto Observer.

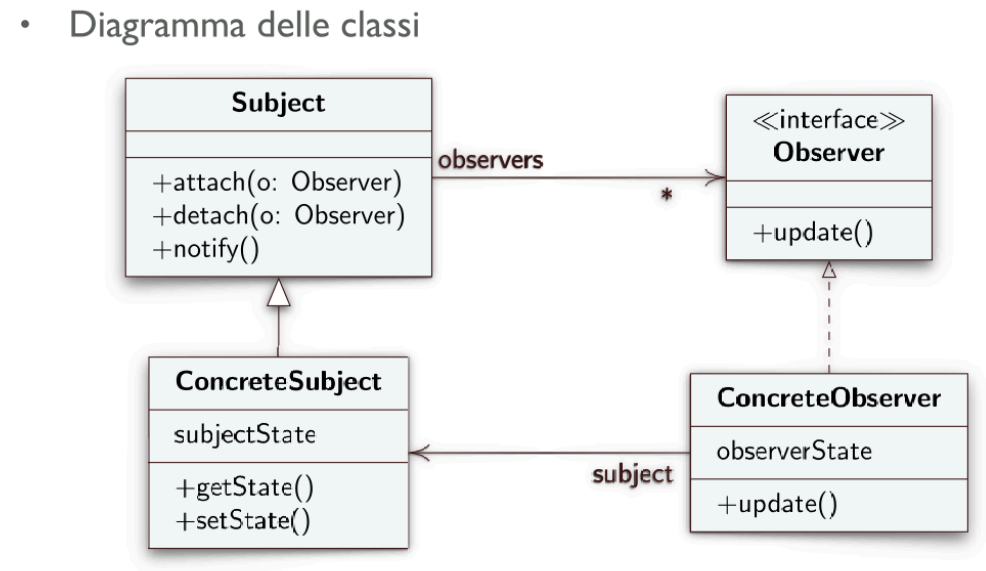


Figura 9.6: Il Subject conserva gli Observer e il ConcreteObserver legge lo stato di interesse.

Nella variante pull la notifica può indicare il completamento di un'operazione senza specificare quali campi siano cambiati. Per limitare lavoro inutile gli iscritti possono dichiarare gli eventi di interesse. Un osservatore di più subject deve ricevere o poter ricavare l'identità del mittente, per esempio con un parametro Subject di `update`. Con molti subject e pochi observer, una tabella associativa condivisa può sostituire alcune liste distribuite: è un'opzione di implementazione da valutare sul caso concreto.

Le notifiche automatiche rendono difficile dimenticare l'aggiornamento ma possono ripetersi inutilmente durante una modifica composta. Le notifiche esplicite permettono di raggrupparlo, trasferendo al chiamante la responsabilità di inviarle. In entrambi i casi lo stato pubblicato deve essere consistente. Se un Observer modifica di nuovo il Subject, possono nascere ulteriori notifiche e cicli: il protocollo deve evitare aggiornamenti ricorsivi indesiderati. Rimuovere le registrazioni quando non servono evita aggiornamenti superflui e riferimenti che prolungano la vita degli oggetti. In Java il problema non è normalmente un puntatore a memoria già liberata, ma la permanenza di riferimenti e iscrizioni non più utili.

9.5.1 Protocollo di aggiornamento coerente

Per un aggiornamento Observer coerente:

1. modificare completamente lo stato del ConcreteSubject;
2. decidere se notificare subito o raggruppare le modifiche;
3. chiamare `notify` solo quando lo stato è consistente;
4. per ogni Observer registrato, invocare `update`;
5. nel modello pull, fare leggere al ConcreteObserver i dati necessari dal Subject.

9.5.2 Rubrica osservabile e salvataggio su file

La rubrica è ConcreteSubject, il salvataggio è ConcreteObserver e `Dialog` configura il collegamento. Il contratto usa il mittente e un dato generico:

```
import java.util.*;
import java.io.*;

interface Observer { void update(Subject s, Object stato); }
class Subject {
    private final List<Observer> obs = new ArrayList<>();
    private boolean changed;
    public void attach(Observer o) { obs.add(o); }
    public void detach(Observer o) { obs.remove(o); }
    protected void setChanged() { changed = true; }
    public void notify(Object stato) {
        if (!changed) return;
        for (Observer o : obs) o.update(this, stato);
        changed = false;
    }
}
record Persona(String nome, String cognome, String telefono, String citta) { }
class AddrBook extends Subject {
    private final List<Persona> nomi = new ArrayList<>();
    public void insert(Persona p) {
        if (nomi.contains(p)) return;
        nomi.add(p);
        setChanged();
        notify(nomi);
    }
    public Persona find(String cognome) {
        for (Persona p : nomi)
            if (p.cognome().equals(cognome)) return p;
        System.out.println("AddrBook.find: NOT found");
        return null;
    }
}
class Store implements Observer {
    @SuppressWarnings("unchecked")
    public void update(Subject s, Object stato) {
        List<Persona> persone = (List<Persona>) stato;
        try (FileWriter f = new FileWriter("nomi.txt")) {
            for (Persona p : persone)
                f.write(p.nome() + "\t" + p.cognome()
                    + "\t" + p.telefono() + "\n");
```

```
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}
```

Il tipo Persona è qui un record di contatto, separato dagli esempi anagrafici precedenti; gli accessori sono quelli del record. Il client esegue `book.attach(store)` e inserisce `new Persona("Oliver","Stone","012345","NY")`. L'inserimento cambia prima la lista, imposta il flag, notifica e fa scrivere nel file nome, cognome e telefono separati da tabulazioni. Un duplicato secondo `equals` non genera notifica. `find("Stone")` restituisce il contatto; una ricerca assente stampa il messaggio e restituisce `null`.

`notify(Object)` è un metodo del Subject didattico, con parametro, distinto da `Object.notify()` senza parametri. Ogni osservatore viene chiamato sincronicamente e un errore di scrittura deve essere reso visibile al livello che può gestirlo. Il cast dal dato generico richiede l'accordo sul contenuto della notifica; un contratto parametrico può evitare questo controllo a runtime. Il flag torna falso dopo il completamento della notifica; un'eccezione prima della fine interrompe il protocollo e richiede una politica esplicita. Non si deve presumere che la notifica sia una transazione.

9.5.3 Observable storico e Reactive Streams

Nell'API storica `Observable` è una classe concreta estendibile, non un'interfaccia; estenderla occupa l'unica superclasse disponibile. Il metodo `notify()` di `Object` ha un significato di sincronizzazione tra thread e non va confuso con la notifica del pattern: l'API degli osservatori usa `notifyObservers()`. Occorre prima impostare il flag con `setChanged()`; dopo la notifica il flag è cancellato. L'Observer riceve `update(Observable,Object)`, con il mittente e un eventuale dato aggiuntivo. L'API è deprecata da Java 9 ed è mantenuta per retrocompatibilità.

La consegna storica è sincrona e bloccante: la chiamata attende il ritorno di `update` degli osservatori. Un consumatore lento rallenta così il produttore; separare le esecuzioni richiede anche di controllare quanto lavoro il consumatore sia pronto a ricevere.

I Reactive Streams introducono il controllo della domanda, detto *backpressure*. Le interfacce Java sono annidate in `java.util.concurrent.Flow`:

`Flow.Publisher<T>` riceve iscrizioni con `subscribe(...)`;

`Flow.Subscriber<T>` riceve `onSubscribe`, `onNext`, `onError` e `onComplete`;

`Flow.Subscription` collega le parti: `request(n)` aumenta la domanda e `cancel()` termina l'iscrizione.

Il Subscriber può richiedere un elemento, elaborarlo in `onNext` e richiedere il successivo, oppure richiedere un lotto. La domanda riguarda il numero di elementi consegnabili, non la creazione di thread. `SubmissionPublisher<T>` offre consegna asincrona usando un esecutore; normalmente usa un pool condiviso, non un thread nuovo per ogni messaggio. Conserva un buffer per ciascun subscriber. Se manca spazio, `submit()` può bloccare il produttore; asincronia e assenza assoluta di blocchi non sono equivalenti.

L'obiettivo è controllare lo squilibrio tra produttore e consumatore e separare le rispettive esecuzioni. Questo protocollo di sottoscrizione asincrona non coincide con le pipeline di `java.util.stream`, che trasformano una sequenza e vengono attivate da un'operazione terminale.

Dalla lista notificata agli elementi pubblicati

Nella variante asincrona AddrBook contiene un `SubmissionPublisher<Persona>`; Store implementa `Subscriber<Persona>`. L'inserimento pubblica la nuova persona, invece di consegnare a ogni aggiornamento l'intera lista:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.Flow.*;

class AddrBook implements AutoCloseable {
    private final List<Persona> nomi = new ArrayList<>();
    private final SubmissionPublisher<Persona> publ =
        new SubmissionPublisher<>();
    public void attach(Subscriber<Persona> s) { publ.subscribe(s); }
    public boolean insert(Persona p) {
        if (nomi.contains(p)) return false;
        nomi.add(p); publ.submit(p);
        return true;
    }
    public void close() { publ.close(); }
}
class Store implements Subscriber<Persona> {
    private Subscription sub;
    private final CountDownLatch fine = new CountDownLatch(1);
    public void onSubscribe(Subscription s) {
        sub = s; sub.request(1);
    }
    public void onNext(Persona p) {
        System.out.println("Store onNext: " + p.nome() + "\t" + p.cognome());
        sub.request(1);
    }
    public void onError(Throwable t) {
        System.err.println("In Store: errore " + t);
        fine.countDown();
    }
    public void onComplete() {
        System.out.println("In Store: completato");
        fine.countDown();
    }
    public void attendi() throws InterruptedException { fine.await(); }
}
```

Si riusa il record Persona del contatto. Il client collega Store, inserisce i contatti, chiude il publisher e attende `store.attendi()` prima di terminare se deve osservare tutte le stampe. Il contatore di attesa serve al ciclo di vita dell'esempio: chiudere avvia il completamento, mentre attendere permette di osservarlo. `onSubscribe` richiede il primo elemento; ogni `onNext` ne richiede un altro soltanto dopo aver completato l'elaborazione. Le callback di errore e completamento sono terminali. Questa versione di Store stampa i singoli eventi; non equivale automaticamente al salvataggio dell'intera rubrica della versione sincrona.

Lo stesso passaggio si applica a un `NoteProducer`: il tipo pubblicato diventa `String`, mentre `SlowReceiver` e `Display` sono due Subscriber distinti configurati dal Main. Il consumatore lento esprime la propria domanda senza imporre al display la stessa velocità. Il cambiamento essenziale del diagramma è la sostituzione di Subject/Observer con Publisher/Subscriber, mantenendo separati produttore e consumatori; non la creazione obbligatoria di un thread per messaggio.

9.6 MVC: dati, presentazione e gestione degli input

MVC separa le responsabilità di un'applicazione interattiva in `Model`, `View` e `Controller`, affinché dati e logica, presentazione e input possano evolvere e riusarsi indipendentemente. Il *Model* incapsula dati e logica di business, cioè le operazioni significative nel dominio del problema; non deve sapere come i dati sono mostrati né quale dispositivo genera l'input. La *View* mostra i dati all'utente. Il *Controller* riceve eventi di mouse o tastiera, li traduce in richieste di servizio per il Model oppure in comandi di interfaccia per la View.

Nel MVC classico il Model è Subject e View e Controller sono ConcreteObserver: il Model registra e notifica entrambi senza conoscerne i tipi concreti. Una View inizializza il proprio Controller e gli delega l'evento fisico con `handleEvent`; il Controller conserva a sua volta un riferimento alla View per azioni puramente grafiche, quali evidenziare un campo o mostrare un dialogo, che non appartengono al Model. Nella collaborazione classica descritta, ogni View è associata al proprio Controller e viceversa.

Il Controller può osservare il Model perché una variazione dei dati può rendere leciti o illeciti alcuni input, per esempio abilitando un pulsante o attivando la sola lettura. Una View statica, come una schermata about o una pagina di errore priva di dati dinamici, può invece svolgere il proprio ruolo senza registrarsi al Model.

Nel web, una configurazione MVC distribuisce Model e Controller sul server e la presentazione nel browser. Il Controller riceve una richiesta HTTP, usa il Model e seleziona o genera una pagina HTML da mostrare. A differenza delle viste desktop che rimangono attive e ricevono notifiche, una pagina prodotta in risposta alla richiesta non richiede necessariamente la registrazione Observer. La separazione dei ruoli resta logica anche quando cambia la loro distribuzione fisica.

Un'applicazione Java realizzata con Spring Boot può adottare questa organizzazione web MVC. Il framework fornisce il contesto applicativo, mentre il progetto deve mantenere distinte logica del dominio, presentazione e gestione delle richieste.

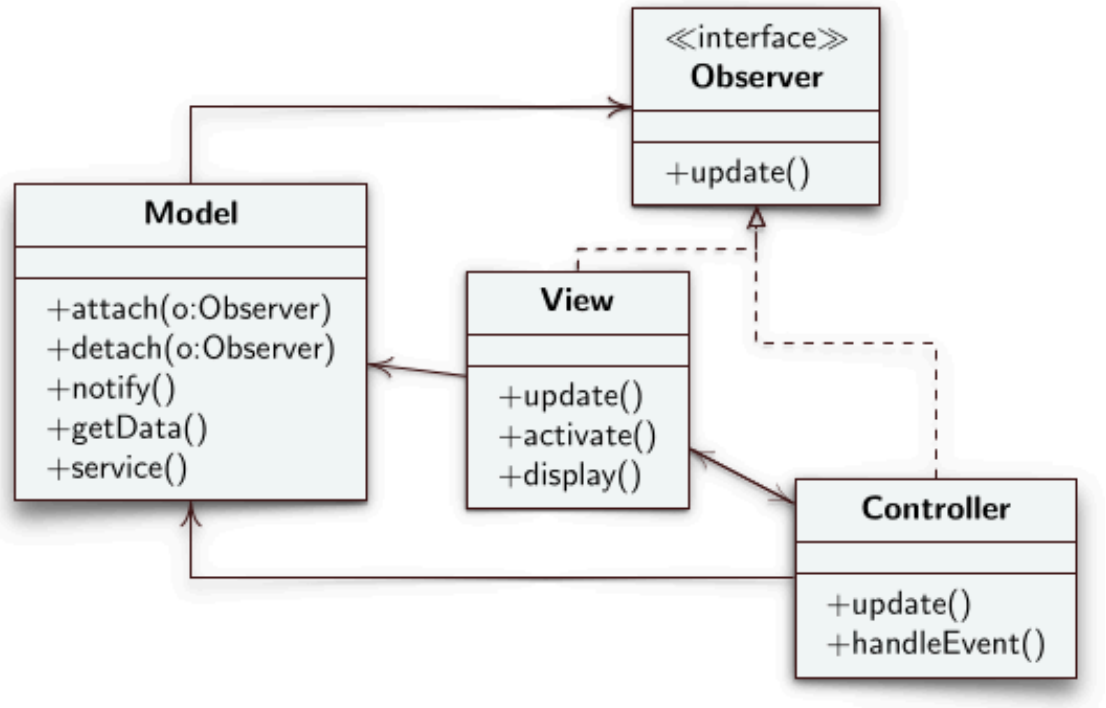


Figura 9.7: Model come Subject e View e Controller come osservatori.

Il Model può riunire Subject e ConcreteSubject, contenendo sia le registrazioni sia i dati. La separazione logica dei ruoli può essere mantenuta anche quando alcune classi sono accorpate; ciò che conta è che il Model non debba conoscere i dettagli delle interfacce utente.

La collaborazione desktop può essere ricostruita con questa sequenza:

1. la View riceve l'evento dell'utente e chiama `handleEvent()` sul Controller;
2. il Controller richiede `service()` al Model;
3. il Model aggiorna i dati e chiama la propria operazione di notifica;
4. gli Observer registrati ricevono `update()`;
5. la View legge `getData()` e chiama `display()`;
6. per un'azione puramente grafica, il Controller può chiamare `activate()` sulla View senza modificare il Model.

Questi nomi appartengono all'esempio: il criterio architetturale è separare i ruoli, non imporre nomi universali ai metodi.

9.7 Chain of Responsibility: trovare un gestore lungo la catena

Chain of Responsibility evita di accoppiare il mandante di una richiesta al suo ricevente: concatena i possibili gestori e inoltra la richiesta finché uno la tratta. `Handler` definisce l'interfaccia e il riferimento al successore; `ConcreteHandler` gestisce ciò di propria competenza oppure inoltra; il client inizia dal primo handler. Nell'esempio dell'aiuto di una GUI, un bottone cerca il messaggio specifico e, se non lo possiede, inoltra al dialogo o alla finestra che lo contiene. La catena può quindi coincidere con una gerarchia Composite. Ogni handler conosce soltanto il successore immediato; si possono inserire, rimuovere o riordinare gestori a runtime. Non è però garantito che qualcuno gestisca la richiesta e una catena lunga aumenta attraversamento e difficoltà di debugging.

Sono possibili due contratti. Con un metodo per tipo di richiesta, come `handleHelp()` e `handlePrint()`, il compilatore controlla le richieste valide, ma aggiungerne una modifica Handler e tutti i ConcreteHandler. Con un solo `handleRequest(code)`, il *codice* identifica il tipo a runtime e l'interfaccia non cambia; si perde però sicurezza a compile-time e un codice sconosciuto può arrivare silenziosamente in fondo alla catena.

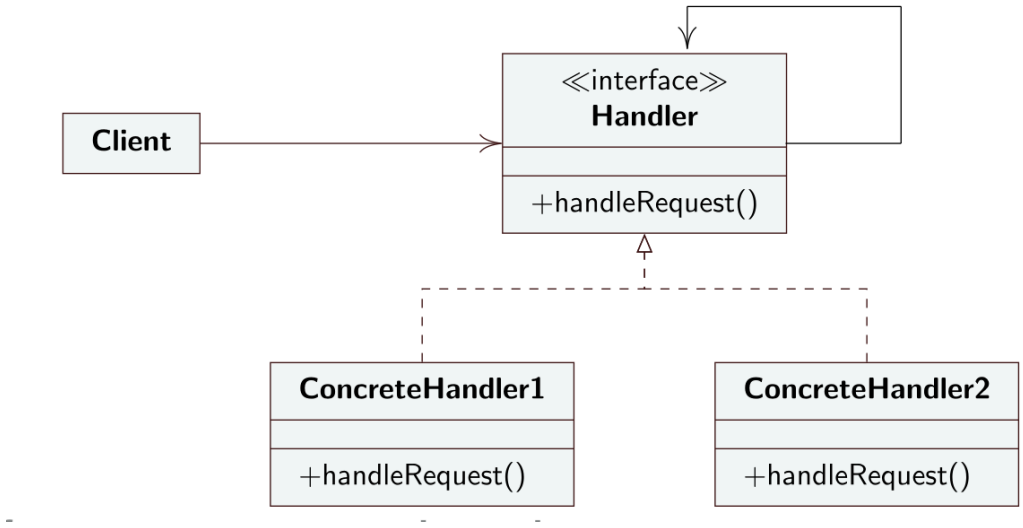


Figura 9.8: Il riferimento al successore collega i gestori senza una lista globale nel singolo handler.

Il successore può essere definito in Handler con un inoltro predefinito, oppure separatamente nei ConcreteHandler: la seconda scelta concede più controllo ma può duplicare codice. Un riferimento al padre già esistente in Composite può realizzare la risalita della richiesta di help, senza costruire una seconda rete di collegamenti.

La correttezza dei tipi non garantisce che un handler soddisfi la richiesta. In entrambe le varianti occorre stabilire l'esito quando il successore è assente e nessuno risponde. Il contratto con codici numerici richiede inoltre un accordo sulla codifica e controlli a runtime.

9.7.1 Esempio: aiuto contestuale

Nel sistema di aiuto, `Button` e `Text` sono ConcreteHandler: il bottone inoltra la richiesta fino al testo, che fornisce assistenza se non esiste un successore.

Aiuto contestuale: contratto e chiamate

Il contratto `Handler` dichiara `handleRequest()`, `activate()` e `setNext(Handler)`. L'ultimo metodo configura la catena; `activate` descrive il controllo, non inoltra automaticamente una richiesta di aiuto.

```
interface Handler {
    void handleRequest(); void activate(); void setNext(Handler next);
}
class Text implements Handler {
    private Handler next;
    Text(Handler h) { next = h; }
    public void handleRequest() {
        if (next != null) next.handleRequest();
        else System.out.println("Text: assistenza");
    }
    public void activate() {
        System.out.println("sono un frammento di testo");
    }
    public void setNext(Handler h) { next = h; }
}
```

Il client costruisce `Text t=new Text(null)` e un Button con successore `t`, attiva entrambi e invia due richieste al bottone. Un Button che non fornisce aiuto proprio inoltra entrambe a Text: il messaggio di assistenza viene quindi prodotto due volte. Il numero di richieste non cambia il collegamento. Il comportamento specifico di Button è una scelta del suo contratto; la catena resta valida anche aggiungendo un gestore intermedio.

9.8 Command: una richiesta diventa un oggetto

Command incapsula una richiesta in un oggetto. `Command` dichiara `execute`; `ConcreteCommand` collega una azione a un `Receiver` e delega a quest'ultimo; `Invoker` conserva un Command e invoca `execute` senza conoscere né il comando concreto né il Receiver; il Client crea e collega tali oggetti. La richiesta può così essere passata, salvata, inserita in una coda o in un log e composta in un `MacroCommand`, che è un'applicazione di Composite. Un comando semplice può contenere direttamente la logica, senza Receiver separato.

Per `undo`, un comando reversibile conserva lo stato necessario a tornare alla situazione precedente e il Receiver deve offrire l'operazione inversa. Per più annullamenti l'Invoker mantiene uno *stack*, cioè una pila LIFO (*Last In, First Out*): l'ultimo comando eseguito è il primo annullato. Per `redo` serve un secondo stack. Riutilizzare una stessa istanza sovrascrivendone i dati perderebbe invece la cronologia.

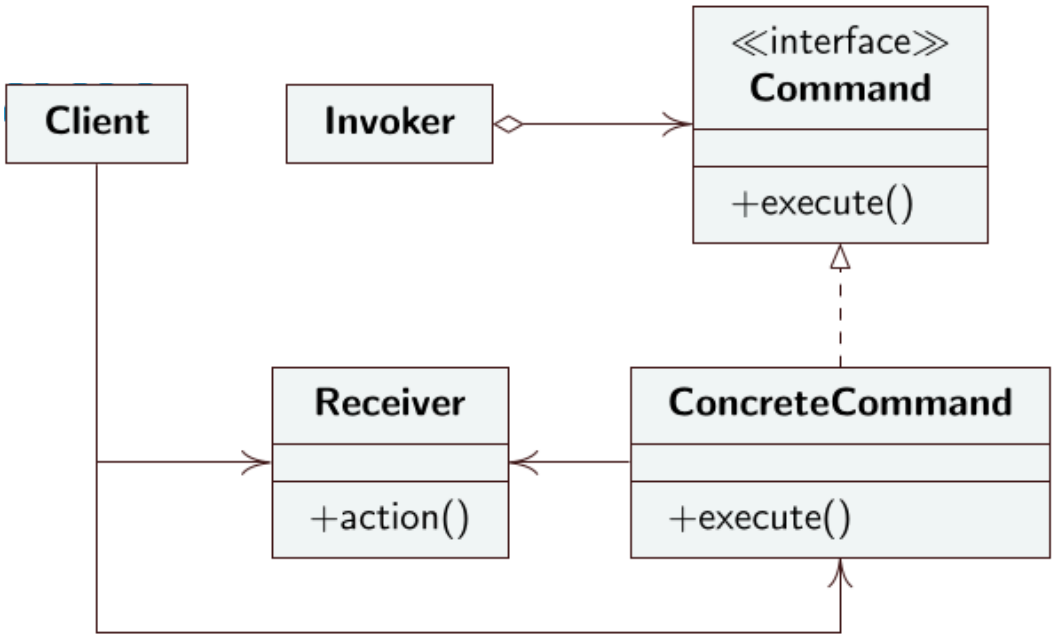


Figura 9.9: Client configura comando e ricevente; Invoker conosce soltanto Command.

I dati dell'operazione vengono forniti al ConcreteCommand quando è configurato: l'Invoker chiama poi `execute()` senza passarli nuovamente. Invoker può essere un bottone, un gestore di input o uno scheduler; non è proprietario esclusivo del comando nel modello illustrato. Nell'editor di testo, scrivere una parola, applicare il grassetto e cancellare con un `DeleteCommand` sono richieste rappresentabili con oggetti diversi. Comandi come `apriFile` e `salvaFile` possono essere riuniti in una macro.

L'annullamento può usare un'operazione inversa oppure uno stato precedente salvato. Non tutte le operazioni sono reversibili. Una MacroCommand esegue una sequenza di Command, svolgendo Composite; per annullarla si procede normalmente in ordine inverso. Un comando di aggiornamento del calendario conserva timestamps in uno stack e usa `putIfAbsent` per

evitare sovrascritture: se l'inserimento non è avvenuto perché la chiave esisteva, l'undo non deve cancellare una voce preesistente che quel comando non ha creato. Serve registrare l'effetto effettivo dell'esecuzione.

9.8.1 Protocollo undo e redo a più livelli

Per l'annullamento a più livelli con Command, sia S_u lo stack degli undo e S_r lo stack dei redo. Dopo l'esecuzione riuscita di un comando reversibile c, si inserisce c in S_u e si svuota S_r; `undo` rimuove il comando in cima a S_u, ne invoca l'inversa e lo inserisce in S_r; `redo` rimuove il comando in cima a S_r, lo riesegue e lo reinserisce in S_u. Il simbolo c indica una singola istanza di comando e "cima" indica l'ultimo elemento inserito nella pila.

9.8.2 Esempio: menu e calendario

Nel calendario, `Menu` è Invoker, `Calendario` è Receiver e `Updater` è ConcreteCommand: aggiungere o togliere note è un oggetto comando, quindi può offrire `unexecute`.

Effetti reversibili nel calendario e nell'editor

Nel calendario `aggiungi` associa una nota a un timestamp, `rimuovi` la elimina e `stampa` osserva gli appuntamenti. Menu riceve un Command e lo attiva; Updater conserva calendario, dati dell'operazione e memoria dell'esito. Per un inserimento con `putIfAbsent`, la voce va aggiunta alla cronologia soltanto se la chiave era libera. Il protocollo essenziale è:

```
// Corpo di execute: receiver e dati sono gia' configurati.
boolean inserito = calendario.aggiungi(timestamp, nota);
if (inserito) inserimenti.push(timestamp);

// Corpo di unexecute.
if (!inserimenti.isEmpty())
    calendario.rimuovi(inserimenti.pop());
```

La pila conserva l'ordine effettivo degli inserimenti riusciti. Se una chiave esisteva già, annullare il tentativo fallito non deve eliminare la nota precedente. Per annullare anche modifiche o cancellazioni occorre salvare il valore precedente, non soltanto il timestamp. La separazione dei ruoli resta la stessa: Menu non conosce la mappa, Updater interpreta la richiesta e Calendario esegue l'aggiornamento.

Nell'editor, un comando di inserimento conserva testo e ricevente. Se aggiunge una stringa `s` in fondo al documento, l'inversa può cancellare gli ultimi `s.length()` caratteri, purché nel frattempo non siano intervenute modifiche incompatibili e gli undo rispettino l'ordine inverso. Un inserimento in una posizione arbitraria richiede anche la posizione; cancellare gli ultimi caratteri non è allora un'inversa generale. Il comando di salvataggio può controllare il flag "documento modificato": salva quando è vero e non riscrive inutilmente quando è falso.

Nel diagramma delle classi di Command, l'Invoker conserva stabilmente il riferimento al Command; il Client può usarlo solo durante il wiring. L'assenza della freccia temporanea Client–Invoker in uno schema semplificato non cancella il messaggio di configurazione visibile nel diagramma di sequenza.

9.9 Mediator: incapsulare il protocollo cooperativo

Mediator incapsula il modo in cui un gruppo di oggetti interagisce. Un `Colleague` conosce il Mediatore ma non gli altri Colleague; `Mediator` è il contratto di comunicazione; `ConcreteMediator` conserva i riferimenti necessari e coordina i ConcreteColleague. In una

finestra di ricerca, i controlli `Category`, `SubCategory`, `Search`, `Ok` e `Cancel` non devono chiamarsi tutti reciprocamente: notificano il mediatore, che decide quali aggiornare. Il pattern sostituisce relazioni molti-a-molti con relazioni uno-a-molti centrate sul mediatore.

La dipendenza comportamentale non scompare: se cambia una categoria, una sottocategoria deve ancora aggiornarsi. Si elimina la dipendenza strutturale diretta fra colleghi e si concentra la logica cooperativa nel mediatore. I Colleague diventano più semplici e riusabili; il compromesso è un ConcreteMediator potenzialmente complesso e specifico dell'applicazione. Mediator non è Facade: Facade semplifica l'accesso unidirezionale a un sottosistema che non conosce la facciata; i Colleague conoscono invece il Mediator e comunicano con esso in entrambe le direzioni. Mediator e Observer possono collaborare quando i Colleague notificano il mediatore mediante Observer.

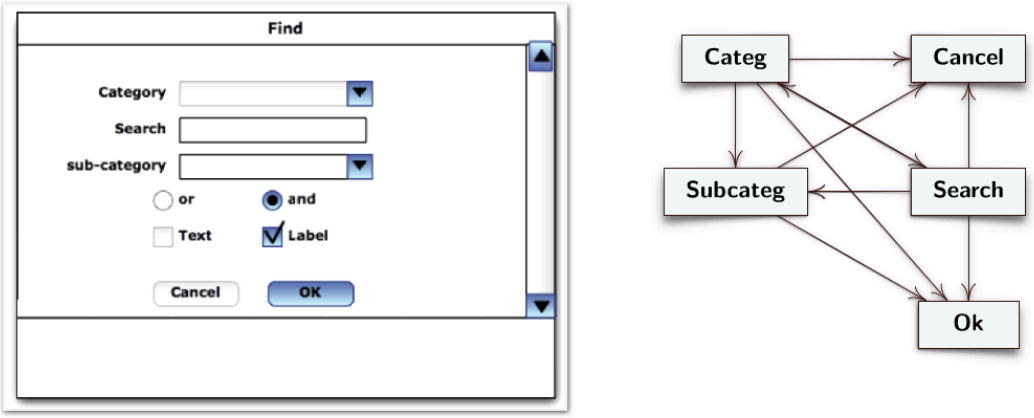


Figura 9.10: La finestra di ricerca e le dipendenze dirette fra i controlli prima del mediatore.

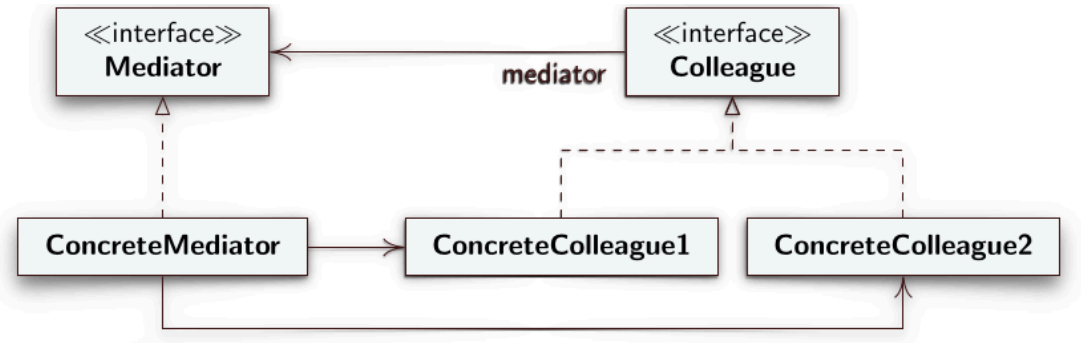


Figura 9.11: La finestra di ricerca centralizza le interazioni dei controlli in un mediatore.

Quando Category cambia, il mediatore aggiorna SubCategory; se Search non è valido può disabilitare Ok. I colleghi non scelgono direttamente quali altri controlli aggiornare. ConcreteMediator deve invece conoscere le operazioni concrete dei partecipanti: la complessità e il costo delle estensioni sono concentrati in quel punto. Il vantaggio è poter riusare i controlli e leggere la logica di cooperazione in un unico luogo.

Se Mediator e ConcreteMediator coincidono, si riducono le classi ma si perde l'interfaccia che permetteva ai colleghi di usare implementazioni alternative senza modifiche. Il mediatore è di solito specifico dell'applicazione. Può ricevere notifiche Observer dai colleghi, ma stabilisce poi il coordinamento: Observer distribuisce eventi, Mediator incapsula il protocollo cooperativo. Un ciclo di dipendenze o un disegno fitto di frecce può motivare una revisione, ma la disposizione grafica delle linee non è una metrica affidabile della bontà del progetto.

9.9.1 Coordinare lettura, numeri e nomi

Un secondo esempio impiega `Central` come Mediator e quattro ConcreteColleague: `KeyListener`, `Numeri`, `Sommatore` e `Nomi`, collegati attraverso il contratto `Elemento`. KeyListener legge un dato dalla tastiera e lo restituisce a Central; il mediatore distingue numeri e stringhe e chiama `put` sui partecipanti interessati. Numeri, Sommatore e Nomi non devono perciò conoscere reciprocamente le proprie concrete.

La sequenza inizia con Central che invoca `doWork()` su KeyListener. Ottenuto il dato, distribuisce richieste `put()` e le eventuali segnalazioni `alert()` previste dal protocollo. Se un collega riconosce la condizione di arresto, chiama `stop()` sul mediatore, che avvisa gli altri colleghi. Nel caso in cui sia Numeri a riconoscerla, la comunicazione torna quindi da Numeri a Central e da Central agli altri; Numeri non li arresta direttamente. La parola o condizione concreta di arresto e le regole esatte delle elaborazioni devono essere specificate dal dominio: il pattern determina dove coordinarle, senza fissarne il contenuto.

9.10 Iterator: attraversare senza esporre la rappresentazione

Iterator permette di attraversare una collezione senza esporne la rappresentazione. Un `Aggregate` dichiara `createIterator`; `ConcreteAggregate` contiene realmente gli elementi; `Iterator` offre operazioni quali `hasNext()` e `next()`; `ConcreteIterator` mantiene la posizione corrente, per esempio un indice, e conosce il modo corretto di percorrere la particolare struttura. Il client chiede l'iteratore all'aggregato e usa il contratto comune: non deve distinguere array, lista o albero. Il ciclo `for-each` di Java usa questo principio per percorrere un oggetto Iterable.

9.11 Null Object: rappresentare un comportamento neutro

Null Object sostituisce l'assenza di un collaboratore con un oggetto che offre un comportamento neutro; può essere introdotto anche mediante refactoring. Invece di restituire `null` e ripetere `if (x != null)`, si fornisce un `NullObject` che implementa lo stesso contratto di `RealObject` ma esegue un'azione neutra. Riduce i controlli e previene una `NullPointerException`, cioè l'errore prodotto tentando di invocare un membro su un riferimento nullo. L'eccezione importante è diagnostica: il comportamento neutro può nascondere un'assenza inattesa dell'oggetto e rendere il difetto più difficile da individuare. Va quindi usato solo quando "non fare nulla" è semanticamente corretto.

9.12 Isolare la causa del cambiamento

Un procedimento di scelta è: identificare se varia un passo di un algoritmo (Template Method), l'algoritmo intero (Strategy), lo stato interno (State), il numero di dipendenti (Observer), il destinatario di una richiesta (Chain), il momento di esecuzione della richiesta (Command), la rete di comunicazioni (Mediator) o la rappresentazione di un attraversamento (Iterator). Poi definire il contratto minimo e verificare il compromesso: classi aggiuntive, notifiche inattese, richiesta non gestita, stato necessario per undo, oppure complessità concentrata nel mediatore.

La scelta di un pattern parte dalla causa del cambiamento: quale dipendenza rende difficile estendere o modificare il sistema? Un contratto utile isola quella dipendenza e lascia stabile il codice che non deve conoscerla.

Tabella 9.1: Dipendenze da contenere e pattern adatti a isolarle.

Dipendenza o problema	Decisione di progetto	Pattern o tecniche
Istanziazione con una classe concreta	Centralizzare la scelta o clonare un prototipo attraverso un contratto.	Factory Method, Prototype.
Operazione o destinatario specifico della richiesta	Separare chi richiede da chi gestisce e da quando la richiesta viene eseguita.	Chain of Responsibility, Command.
Piattaforma hardware o software	Isolare il codice specifico e configurare l'implementazione.	Factory, Bridge.
Rappresentazione e implementazione interna	Nascondere struttura e costruzione dietro interfacce stabili.	Factory, Bridge, Memento, Proxy.
Algoritmo soggetto a sostituzione	Isolare una variante di algoritmo, un passo o il comportamento di uno stato.	Builder, Iterator, Strategy/State, Template Method, Visitor.
Accoppiamento stretto fra componenti	Riorganizzare creazione, accesso, richieste e notifiche.	Abstract Factory, Bridge, Chain, Command, Facade, Mediator, Observer.
Troppe estensioni tramite sottoclassi	Comporre oggetti attraverso contratti e limitare le gerarchie non necessarie.	Bridge, Chain, Composite, Decorator, Observer, Strategy.
Classi che non si possono modificare	Adattare il protocollo oppure avvolgere l'oggetto aggiungendo comportamento.	Adapter, Decorator, Visitor.

Il riuso per ereditarietà è detto a scatola bianca perché può richiedere conoscenza del funzionamento della superclasse e delle dipendenze fra i suoi punti di estensione. Il riuso per composizione è detto a scatola nera perché usa contratti pubblici e può cambiare i collaboratori a runtime. Nessuna delle due tecniche sostituisce il ragionamento sull'intento: la composizione è vantaggiosa quando le dimensioni devono variare indipendentemente, mentre una vera specializzazione può giustificare l'ereditarietà.

Per valutare una soluzione creazionale occorre: individuare se varia numero, tipo, famiglia, configurazione o copia degli oggetti; separare client e concreta attraverso il contratto necessario; centralizzare la costruzione nel ruolo appropriato; definire chi possiede e reimposta lo stato; infine verificare con test che oggetti e dipendenze consegnati rispettino invarianti e contratti.

Capitolo 10

Processi di sviluppo, Agile, XP e Scrum

10.1 Attività, prodotti e coerenza del processo

Un *processo di sviluppo software* organizza le attività necessarie per ottenere e mantenere un prodotto: stabilisce relazioni, responsabilità, risultati attesi e momenti di verifica. Le fasi possono essere suddivise in compiti, o *task*. Il problema è coordinare il lavoro mentre evolvono bisogni, ambiente e conoscenza del sistema.

Le fasi ricorrenti sono analisi dei requisiti, progettazione, implementazione, convalida e manutenzione. Un *artefatto* è un prodotto intermedio o finale di una fase, per esempio una specifica, un modello UML, codice o una suite di test. La consistenza fra artefatti è importante: ciascuno deve concordare con quelli da cui deriva.

10.1.1 Dalle specifiche all'implementazione

Analisi e specifica trasformano i bisogni in requisiti verificabili; la progettazione assegna responsabilità a sottosistemi, componenti e interfacce; l'implementazione traduce queste decisioni in codice eseguibile. I modelli di oggetti, interazioni, stati e flussi di dati devono restare coerenti con ciò che il programma realizza. Progettazione e implementazione possono alternarsi: un risultato sperimentale può richiedere di rivedere una decisione precedente.

Quando un test rivela un difetto, si localizza la causa, si corregge il modello se esprime una decisione errata, si aggiorna il codice e si riesegue il test. Correggere soltanto il programma lasciando il modello incoerente rende più difficile la manutenzione successiva.

10.1.2 Verifica, rilascio e cambiamento

Verifica e validazione (V&V) accompagnano lo sviluppo mediante revisioni e test a livelli diversi. Il rilascio consegna una versione del prodotto; la manutenzione continua poi con correzioni, miglioramenti e adattamenti. Una modifica può conservare il comportamento, come nel refactoring, oppure aggiornarlo per soddisfare nuovi requisiti o usare nuove piattaforme.

Se il cliente chiede un comportamento diverso dopo aver approvato requisiti già soddisfatti, si analizza la nuova informazione sul bisogno e se ne valuta l'impatto su specifiche, piano e verifiche. La conformità a un documento deve quindi essere accompagnata dalla validazione dell'utilità del prodotto.

10.2 Cascata: stabilità e documentazione delle fasi

Il modello *a cascata* (Waterfall) è un processo sequenziale: si avvia una fase solo dopo aver completato la precedente. La sequenza è definizione dei requisiti, progetto di sistema e software, implementazione e test di unità, integrazione e test di sistema, operazione e manutenzione. Requisiti e documenti delle fasi concluse sono congelati: l'intento è mantenere coerenti tutti gli artefatti.

I vantaggi sono ampia documentazione, consistenza fra artefatti e qualità del codice prodotto. Gli svantaggi sono il tempo lungo per avere il prodotto, poche interazioni con il cliente oltre la fase iniziale e l'elevato costo di introdurre cambiamenti. È indicato quando i requisiti sono stabili e chiaramente definiti, quando occorre gestire gruppi numerosi e per sistemi grandi, complessi o critici, nei quali l'affidabilità è essenziale.

La necessità di processi organizzati nasce anche dalla *crisi del software*: alla fine degli anni Sessanta, attorno al 1969, il rapido progresso dell'hardware conviveva con difficoltà nel rispettare tempi, costi e obiettivi dei programmi. Lo schema sequenziale è associato al lavoro di Royce del 1970 e richiama l'organizzazione di produzioni industriali, come quella automobilistica. Documentazione e coerenza degli artefatti rispondono all'esigenza di governare la complessità, senza garantire da sole la correttezza.

Il richiamo alla legge di Moore e a un raddoppio orientativo su una scala di circa diciotto mesi riassume il rapido progresso hardware. Questo ordine di grandezza non si traduce in un raddoppio periodico garantito della velocità di ogni programma: prestazioni e costo dipendono anche da algoritmi, implementazione e uso delle risorse.

La rappresentazione strettamente sequenziale congela le fasi; uno schema con frecce di ritorno permette invece feedback controllati verso fasi precedenti. Sono due gradi diversi di rigidità: correggere requisiti o progetto è possibile nella seconda rappresentazione, ma può obbligare a riesaminare documentazione e verifiche già approvate. Il modello non va quindi confuso con l'impossibilità materiale di tornare indietro. Una durata complessiva di dieci–dodici mesi è un esempio di pianificazione, non una proprietà necessaria della cascata. Software per aerei o centrali nucleari esemplifica contesti nei quali documentazione, tracciabilità e controllo delle modifiche hanno particolare rilievo.

10.3 Sviluppo evolutivo, incrementale e per riuso

Un *processo evolutivo* costruisce il sistema mediante trasformazioni successive e ha due varianti.

Sviluppo per esplorazione. Sviluppatori e cliente lavorano insieme: si comprende, progetta, implementa e mostra una parte, quindi il feedback guida il passo successivo. È utile quando la visione complessiva non è ancora sufficientemente chiara, ma dovrebbe partire da requisiti iniziali chiari e aggiungere le caratteristiche richieste dal cliente.

Build and Fix. Si costruisce una prima versione e la si modifica fino alla soddisfazione del cliente, con comprensione limitata del sistema e design e documentazione quasi inesistenti. La mancanza di progettazione e documentazione rende difficile controllare la qualità del codice e il costo delle modifiche.

Entrambe le varianti possono richiedere tempi lunghi e produrre sistemi difficili da comprendere, modificare o mantenere corretti, per mancanza di una visione d'insieme. Sono quindi appropriate per sistemi piccoli, singole parti di sistemi grandi, come un'interfaccia utente, o prototipi a vita breve.

Nel flusso evolutivo una descrizione iniziale orienta specifica, sviluppo e validazione, che si alimentano reciprocamente invece di costituire tre blocchi irreversibili. Le versioni iniziale, intermedia e finale rendono visibile questa convergenza progressiva.

Nello *sviluppo incrementale* si implementano per prime le funzionalità di base o prioritarie; a ogni incremento si aggiunge codice per un ulteriore gruppo di funzionalità, fino al completamento. Il processo CBSE (*Component-Based Software Engineering*) basa invece il progetto sul riuso di COTS (*Components Off The Shelf*), cioè componenti esistenti: analisi dei componenti disponibili, eventuale modifica dei requisiti, progetto tramite riuso, sviluppo e integrazione.

Il riuso può ridurre il lavoro di costruzione sia usando COTS sia recuperando componenti già sviluppati dall'azienda. Occorre però verificarne compatibilità, qualità e costo di integrazione. Il prezzo del nuovo progetto non deriva automaticamente da quello dei componenti. Analogamente, lo studio di fattibilità valuta risorse, competenze, tempi e costi, mentre durata e condizioni economiche dello studio dipendono dall'accordo organizzativo.

10.4 Spirale: obiettivi e riduzione dei rischi

Il modello a spirale di Boehm focalizza prodotti parziali, cioè sottosistemi funzionanti. Ogni ciclo della spirale è una fase e attraversa quattro settori: identificazione degli obiettivi della fase corrente; valutazione e riduzione dei rischi; produzione e convalida della parte; revisione del progetto e pianificazione della fase seguente. Un *rischio* è qualcosa di sconosciuto che può impedire il successo, inteso come soddisfacimento dei requisiti; gli attributi rilevanti sono probabilità di occorrenza e impatto, ossia la gravità del danno peggiore. Quando un ciclo realizza una parte del prodotto, il codice sviluppato viene testato e integrato nel sistema complessivo. La ripetizione dei cicli consente una prima versione in tempi ridotti e l'interazione con il cliente.

I cicli possono avere obiettivi diversi: costruire un prototipo, realizzare una parte dell'interfaccia oppure ridurre un rischio tecnico. Esempi di rischio sono la dipendenza da una sola persona competente, l'indisponibilità di personale e difetti nelle librerie. La natura iterativa emerge già confrontando due cicli successivi; una durata dell'ordine di sei mesi per ciclo è un esempio orientativo, da adattare al progetto, non una durata minima del modello.

La spirale è un *metaprocesso*: la valutazione dei rischi orienta anche la scelta del modello e delle tecniche da usare nel settore di sviluppo di ciascun ciclo. Gli obiettivi sono valutati insieme ad alternative e vincoli; simulazioni, modelli e benchmark possono confrontare opzioni prima di impegnarsi nella costruzione. Prototipi successivi, da una prima versione a una terza e poi a un prototipo operativo, riducono incertezze diverse.

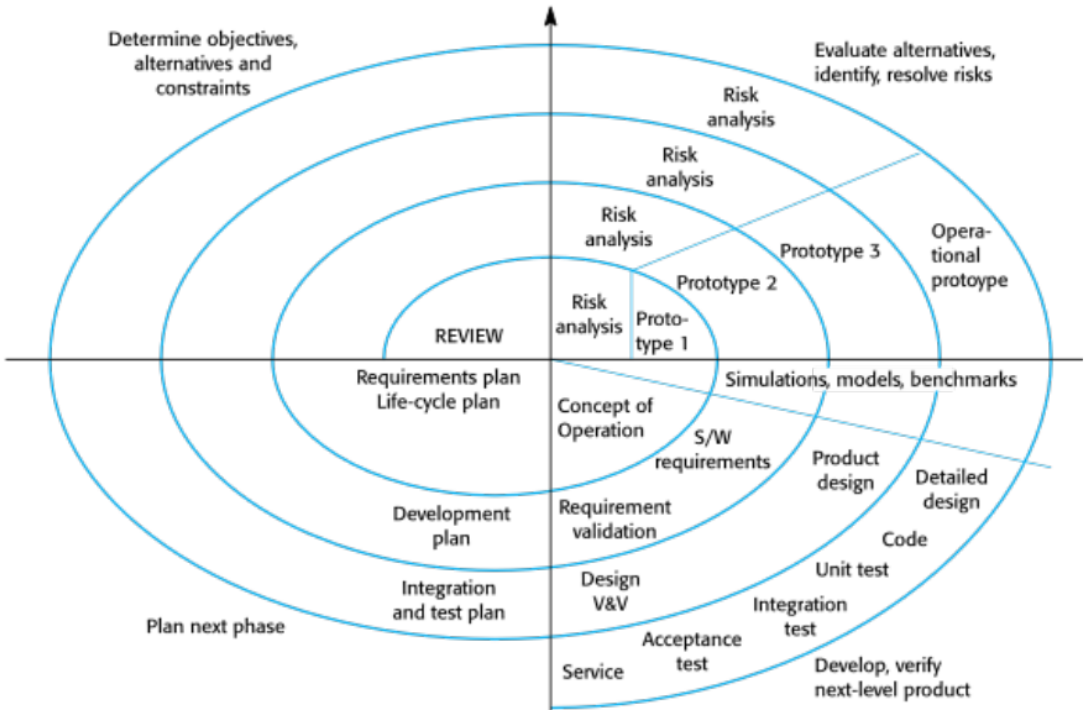


Figura 10.1: Cicli guidati dal rischio: prototipi, verifiche, piani e sviluppo progressivo.

Il dettaglio dei cicli può passare dal concetto operativo ai requisiti software, al progetto del prodotto, al progetto dettagliato, al codice e ai test di unità, integrazione e accettazione. Validazione dei requisiti e verifica e validazione del progetto accompagnano tali passi. Piani del ciclo di vita, dello sviluppo e dell'integrazione e dei test collegano gli impegni dei cicli successivi. Questa varietà spiega perché non ogni giro debba ripetere identiche attività né produrre subito codice definitivo.

Se si ipotizzano almeno due cicli di circa sei mesi, il calendario dell'esempio è dell'ordine di un anno; si tratta di una scelta di scala, non di un minimo matematico del modello. In quel contesto una revisione complessiva dei rischi non coincide con una semplice riunione settimanale o mensile. La frequenza appropriata dipende comunque dall'entità del rischio e dalla durata effettiva dei cicli.

10.5 Approccio Agile: valori, feedback e cambiamento

Il Manifesto per lo sviluppo Agile del 2001 privilegia individui rispetto a processi e strumenti, software funzionante rispetto alla documentazione, collaborazione con il cliente rispetto alla negoziazione contrattuale, e pronta risposta ai cambiamenti rispetto alla mera esecuzione di un piano. Il confronto non dichiara inutili gli elementi a destra: indica che, nelle scelte, hanno maggiore valore quelli a sinistra.

L'agilità accoglie positivamente richieste di cambiamento anche avanzate, fornisce frequentemente release funzionanti, costruisce il sistema con persone motivate e presta continua attenzione all'eccellenza tecnica. Collaborazione, comunicazione e adattabilità richiedono persone motivate, competenze adeguate e disponibilità del cliente a fornire chiarimenti e feedback frequenti.

10.6 Extreme Programming: dodici pratiche che si sostengono

Extreme Programming (XP), formalizzato nel 1999, è un metodo Agile basato sullo sviluppo e sulla consegna di piccoli incrementi di funzionalità. Usa iterazioni corte e di durata costante, piccoli gruppi, indicativamente da due a dodici sviluppatori, comunicazione diretta fra sviluppatori, clienti e manager, miglioramento costante del codice e prodotti testati fin dall'inizio. Privilegia una documentazione essenziale e aggiornata durante il lavoro, in particolare story card e CRC card. È adatto a requisiti instabili e a progetti con rischi elevati, per esempio consegne brevi o software innovativo per il team.

I suoi principi sono feedback rapido, semplicità, cambiamenti incrementali, supporto al cambiamento e produzione di lavoro di qualità.

Tabella 10.1: Le dodici pratiche di Extreme Programming.

Pratica	Funzione nel lavoro quotidiano
Gioco di pianificazione	Cliente e sviluppatori definiscono storie, stime e priorità; le storie eccessive vengono suddivise.
Pair programming	Una postazione condivisa, ruoli driver e navigator, revisione continua e rotazione dei partner.
Piccole release	Parti piccole ma funzionanti permettono feedback rapido; una cadenza tipica è di due settimane.
Metafora	Un'immagine condivisa aiuta a discutere architettura, nomi e scelte.
Design semplice	Si costruisce la soluzione necessaria ora, guidata da test, chiarezza e assenza di duplicazione.
Testing	Test di unità e funzionali documentano il comportamento e proteggono l'integrazione.
Possesso collettivo	Tutti sono responsabili del codice e possono migliorarlo, rispettando standard e test.
Integrazione continua	Le parti testate vengono integrate frequentemente e verificate insieme.
Settimana di quaranta ore	Un ritmo sostenibile evita che la stanchezza comprometta ragionamento e qualità.
Cliente in sede	Una persona disponibile chiarisce storie, decide priorità e definisce risultati attesi.
Standard di codifica	Convenzioni comuni su nomi, formattazione e commenti rendono il codice uniforme.
Refactoring	Piccole trasformazioni migliorano il progetto conservando il comportamento verificato.

Le dodici pratiche si sostengono reciprocamente: per esempio, la proprietà collettiva del codice richiede standard condivisi e test affidabili, mentre integrazione frequente e refactoring beneficiano di modifiche piccole. Omettere una pratica può indebolire i benefici delle altre.

10.6.1 Gioco di pianificazione, story card e piccole release

Nel *gioco di pianificazione*, gli utenti scrivono le *storie utente* sulle *story card*; gli sviluppatori stimano il tempo necessario a ciascuna storia. Una storia troppo complessa per essere stimata deve essere riportata al cliente e divisa. Gli utenti dividono, fondono e assegnano priorità alle

storie; gli addetti al business decidono date delle release, contesto e priorità. La pianificazione dell'intera release è grossolana e quella della nuova iterazione è ravvicinata: non si pianifica troppo in avanti. Per la release corrente, gli sviluppatori scompongono ogni storia in task, li stimano e si impegnano a realizzarli, affrontando prima i task più rischiosi.

Una story card è un caso d'uso leggero: in due o tre frasi, importante per il cliente, testabile, stimabile e associabile a una priorità. I campi indicati sono data, numero, priorità, tempo stimato, riferimenti, descrizione del requisito, lista delle attività e note. Le *piccole release* devono essere il più piccole possibile e richiedono tipicamente due settimane: consentono feedback rapido, riducono il rischio, rendono visibili risultati validi e permettono aggiustamenti ai requisiti.

Le story card possono avere il formato di schede di circa 12×7 cm: poco spazio incoraggia richieste piccole, comprensibili e verificabili. Data e numero identificano le schede; stime e priorità rendono esplicita la selezione. Una storyboard può avere colonne per prima settimana, seconda settimana e lavoro successivo. La misura fisica e le colonne sono strumenti dell'esempio, non requisiti del metodo.

Story card: un browser di immagini

Per il progetto *Gestione Immagini*, tre storie scritte da Jim, tutte con priorità 1, separano comportamenti verificabili. Jack ne stima rispettivamente tre, uno e due giorni:

1. mostrare i file JPG e PNG di una cartella in una griglia di 10×6 immagini, indipendentemente dalla risoluzione dello schermo;
2. ridimensionare le immagini rispettandone le proporzioni iniziali;
3. selezionare le immagini digitando lettere in una casella di testo: restano quelle il cui nome di file contiene il testo inserito.

La terza scheda reca la data 15 marzo 2011. Autore, stimatore, stima, sviluppatori e tempo effettivamente impiegato sono campi diversi: confrontarli permette di apprendere dall'esecuzione. Una board su tre settimane distribuisce le schede nelle aree *To do* e *Done*, rendendo visibile ciò che resta e ciò che è stato completato. Le durate sono stime di quel lavoro, non limiti universali per una storia.

Nel planning game il cliente ordina le storie soprattutto per valore; l'indicazione di non farsi guidare inizialmente dalle dipendenze riguarda questa priorità di business. Gli sviluppatori devono comunque analizzare le dipendenze quando scompongono, stimano e ordinano i task. La stima si discute collettivamente: anche due giorni possono risultare troppi per una specifica iterazione e motivare una suddivisione, senza definire una soglia generale. Una terza area per il lavoro successivo evita di fingere una pianificazione dettagliata di tutto il futuro.

10.6.2 Pair programming, metafora, design semplice e CRC

Nel *pair programming* due programmatori lavorano insieme: il *driver* usa tastiera e mouse e ragiona sull'implementazione del metodo; il *navigator* valuta l'approccio, lo revisiona, pensa ai test e cerca una soluzione più semplice. Ruoli e partner si scambiano. La pratica diffonde la conoscenza del sistema e richiede motivazione e collaborazione; un forte divario di competenze può ridurne l'efficacia se impedisce a entrambi di partecipare al ragionamento.

Nella configurazione con una sola postazione, il navigator non lavora in parallelo su un secondo PC: segue il codice e può annotare su carta i test da eseguire. La rotazione permette di condividere conoscenza e controllo del risultato, invece di dividere stabilmente chi scrive e chi osserva.

Una *metafora* guida il progetto con una frase che rappresenta l'architettura e rende più semplici le discussioni; per esempio, "l'interfaccia utente è un desktop". Deve risultare

comprensibile al cliente. Il *design semplice* è quello che passa i test, non contiene parti duplicate, esprime le intenzioni importanti per i programmatori e usa il minor numero possibile di classi e metodi. La regola è fare la cosa più semplice che può funzionare e pagare solo ciò che si usa, evitando di progettare anticipatamente caratteristiche non richieste.

Le CRC card (*Class–Responsibility–Collaboration*) descrivono il design in linguaggio naturale: il nome della classe compare in alto, seguito da due aree per responsabilità e collaborazioni. Sono uno strumento per ragionare su compiti e relazioni, anziché una lista di firme Java.

Schede, pair programming, codice leggibile, test e cliente disponibile distribuiscono la conoscenza del progetto durante il lavoro. La documentazione non richiede necessariamente una fase separata, ma deve comunque comprendere ciò che serve per usare, verificare e mantenere il prodotto.

Dalle storie alle responsabilità: tre CRC card

Le stesse storie conducono alle seguenti schede. La distinzione fra responsabilità e collaboratori impedisce di trasformare ogni requisito in una classe isolata.

Classe	Responsabilità	Collaborazioni
`ImgBrowser`	Mostrare un frame con due pannelli, uno per le immagini e uno per i controlli.	`ImgPanel`, `CtrlPanel`, `JFrame` come superclasse.
`ImgPanel`	Visualizzare le immagini, occupare la parte superiore della finestra e calcolare le proprie dimensioni.	`Cartella`, `Img`, `JPanel` come superclasse.
`Cartella`	Leggere i file immagine da una cartella, filtrarli secondo un criterio e conoscere il numero di immagini presenti e selezionate.	`Img`, `File` di `java.io`.

La superclasse è indicata come tale: non è una collaborazione fra due oggetti distinti. La griglia e le dimensioni riguardano la presentazione; lettura, selezione e conteggio dei file restano nella responsabilità della cartella. Le schede preparano così le interfacce e le collaborazioni prima dei corpi dei metodi.

10.6.3 Testing, integrazione, proprietà collettiva e ritmo sostenibile

In XP si testa continuamente tutto ciò che può fallire e si rieseguono i test più volte al giorno dopo nuovo codice. I *test funzionali*, scritti dal cliente dal punto di vista dell'utente, sono automatizzati, eseguiti almeno ogni giorno e costituiscono parte della specifica eseguibile dei requisiti. Gli *unit test*, scritti dagli sviluppatori, possono precedere il codice nel TDD (*Test Driven Development*) o seguirlo e sostengono design, codifica, refactoring e qualità.

Il *possesso collettivo del codice* significa che ciascuno può modificare ogni parte del sistema ed è responsabile dell'intero sistema; gli unit test proteggono le funzionalità. L'*integrazione continua* integra il codice testato ogni poche ore, al più entro un giorno. Tutti gli unit test devono passare; se falliscono, la coppia che ha prodotto il codice deve ripararlo oppure scartarlo e ricominciare.

Le ulteriori pratiche sono settimana di quaranta ore, per mantenere la capacità di risolvere problemi e non introdurre difetti dovuti alla stanchezza; cliente in sede, che scrive i test funzionali, stabilisce priorità, risponde alle domande e fornisce il contesto; standard di codifica condivisi, affinché il codice sia uniforme e leggibile; refactoring in piccoli passi, protetto dai test, per migliorare la struttura senza alterare il comportamento.

Con dodici sviluppatori organizzati in sei coppie, una storia stimata in quattro ore consente di integrare più volte nella giornata. La suite globale può però diventare molto grande: occorre distinguere test unitari rapidi e controlli più costosi, senza rinunciare alla verifica dell'integrazione. Gli straordinari ripetuti sono un segnale per riesaminare stime, carico, impedimenti e pianificazione frequente; non una soluzione strutturale alla scarsa produttività. Le quaranta ore esprimono il ritmo sostenibile della pratica, da leggere nel contesto organizzativo.

Gli standard rendono esplicite scelte concrete: spazi o tabulazioni per l'indentazione, posizione delle parentesi graffe, convenzioni sui nomi e uso dei commenti. L'obiettivo è leggere e modificare codice scritto da qualunque membro del gruppo, non imporre una formattazione universalmente migliore.

10.6.4 Strumenti di collaborazione e condivisione della conoscenza

Wiki, sistemi domanda–risposta e strumenti di condivisione della sessione di lavoro possono aiutare a conservare e distribuire conoscenza. La tecnologia wiki, associata al lavoro di Ward Cunningham, permette la modifica collaborativa delle pagine; Stack Overflow, attivo dal 2008, organizza domande e risposte tecniche. Questi strumenti affiancano il lavoro del gruppo senza sostituire test e comprensione del sistema.

Il *mob programming*, diffusosi come pratica riconoscibile dal 2014, estende la collaborazione dalla coppia a un gruppo: per esempio, circa cinque partecipanti possono comprendere sviluppatori, cliente e architetto. Le sessioni condivise da remoto, divenute particolarmente rilevanti dal 2020, possono usare strumenti come Live Share. Numeri e configurazioni sono esempi organizzativi: il criterio è permettere ai partecipanti di contribuire e condividere il ragionamento.

Dal 2023 l'uso di ChatGPT come supporto alla programmazione aggiunge una possibile interazione con strumenti generativi alle forme storiche di collaborazione. Il programmatore resta responsabile di comprendere, integrare e verificare il codice: un suggerimento automatico non svolge da solo le responsabilità del navigator né sostituisce la conoscenza condivisa del team.

10.7 Scrum: empirismo, lean e responsabilità del team

Scrum, descritto da una guida pubblicata inizialmente nel 2010, è una struttura di base per generare soluzioni a problemi complessi. Un *Product Owner* elenca il lavoro nel *Product Backlog*; lo *Scrum Team*, composto da Developers, Product Owner e Scrum Master, trasforma una parte del lavoro in un *incremento* durante uno *Sprint*, un'iterazione di durata fissata, al massimo un mese. Il Product Backlog è l'elenco ordinato del lavoro relativo al prodotto. Team e *stakeholder*, ossia le persone interessate al prodotto, valutano il risultato e apportano gli aggiustamenti per lo Sprint successivo.

Scrum è volutamente semplice e incompleto: altri processi e tecniche, inclusa XP, possono essere usati al suo interno. Si fonda su *empirismo*, per cui conoscenza e decisioni derivano dall'esperienza osservata, e su *lean*, cioè riduzione dello spreco e concentrazione sull'essenziale. L'approccio iterativo e incrementale mira a prevedibilità e controllo del rischio.

Il Product Owner risponde del valore del prodotto e della gestione del backlog; i Developers pianificano il lavoro e costruiscono l'incremento; lo Scrum Master sostiene l'efficacia di Scrum. Queste responsabilità chiariscono chi cura obiettivi, realizzazione e funzionamento del processo senza imporre le pratiche tecniche specifiche di XP.

10.7.1 Trasparenza, ispezione e adattamento

I tre pilastri sono:

Trasparenza. Processo e lavoro sono visibili a chi li svolge e a chi riceve i risultati. Le decisioni importanti si basano su Product Backlog, Sprint Backlog e incremento.

Ispezione. Artefatti e progresso verso l'obiettivo sono esaminati frequentemente e diligentemente per individuare problemi; gli eventi Scrum ne fissano la cadenza.

Adattamento. Se processo o risultato diventano inaccettabili, devono essere corretti immediatamente per evitare ulteriori divergenze.

10.7.2 Product Goal, Sprint Goal, backlog e Definition of Done

Il backlog può comprendere lavoro funzionale e non funzionale e continua a evolvere con ciò che il gruppo apprende. Ogni artefatto ha un impegno che ne rende esplicito lo scopo: Product Goal per il Product Backlog, Sprint Goal per lo Sprint Backlog e Definition of Done per l'incremento.

Il *Product Goal* descrive lo stato futuro del prodotto ed è associato al Product Backlog. Lo *Sprint Goal* è l'obiettivo dello Sprint. Lo *Sprint Backlog* comprende lo Sprint Goal, gli elementi del Product Backlog scelti per lo Sprint e il piano per realizzarli; un *incremento* è un risultato che avvicina al Product Goal. La *Definition of Done* precisa lo stato dell'incremento quando soddisfa le misure di qualità richieste; chiarisce quindi che cosa significhi davvero "finito", anche per obiettivi intermedi.

Done deve indicare un risultato osservabile: per esempio codice integrato, test superati e qualità richiesta verificata. Scrivere l'ultima riga di codice non basta se la Definition of Done richiede anche altre attività. I criteri di accettazione descrivono invece le condizioni specifiche della funzionalità.

Empirismo e lean si traducono così in decisioni fondate su risultati osservabili e attenzione al lavoro necessario. L'esempio Toyota rende concreta la riduzione degli sprechi: rifornire i pezzi secondo le necessità evita l'accumulo di grandi quantità in magazzino.

10.7.3 Eventi e cadenze

Gli eventi sono lo Sprint, tipicamente di un mese; lo *Sprint Planning*, che pianifica il lavoro e dura al massimo otto ore per uno Sprint mensile; il *Daily Scrum*, incontro giornaliero di quindici minuti per ispezionare il progresso e adattare lo Sprint Backlog; la *Sprint Review*, che ispeziona i risultati e decide gli adattamenti; e la *Sprint Retrospective*, in cui il team migliora qualità ed efficacia esaminando persone, interazioni, processi, strumenti e Definition of Done.

Per uno Sprint mensile la Sprint Review dura al massimo quattro ore ed è il penultimo evento: segue la Retrospective, che conclude lo Sprint. Per Sprint più brevi le durate sono normalmente ridotte. Il Daily si svolge alla stessa ora e nello stesso luogo per ridurre la complessità organizzativa; resta un incontro di quindici minuti per adattare il lavoro verso lo Sprint Goal, non una revisione finale del prodotto.

10.8 Esempio: gestione degli ordini con XP e Scrum

Si consideri una funzionalità di gestione degli ordini. Nel processo XP il cliente può scrivere una story card: "un utente inserisce un ordine e ottiene la conferma", assegnandole priorità e criteri osservabili per un test funzionale. Gli sviluppatori stimano la storia; se troppo ampia, per esempio perché include pagamento, controllo disponibilità e spedizione, chiedono di dividerla. Nella release corrente una coppia realizza prima il task più rischioso, scrive ed esegue unit test, integra il codice solo quando i test passano e mostra la piccola release al cliente. Se il feedback richiede una variazione, il requisito viene ri-prioritizzato nella pianificazione successiva.

Lo stesso lavoro in Scrum può essere registrato nel Product Backlog. Durante lo Sprint Planning il team seleziona l'elemento, formula lo Sprint Goal e lo pone nello Sprint Backlog. L'incremento è accettabile soltanto quando soddisfa la Definition of Done concordata, per esempio implementazione, test e integrazione. Daily Scrum, Review e Retrospective consentono rispettivamente di adattare il lavoro corrente, valutare il risultato con gli stakeholder e migliorare il modo di lavorare. Questo esempio combina i due metodi senza confonderli: Scrum organizza il ciclo, XP suggerisce come programmare e verificare.

10.9 Dominio, specifiche e strumenti di IA

Nel *DDD*, sviluppo guidato dal dominio, la comprensione del contesto applicativo orienta il modello a oggetti. Per individuare entità, significato dei dati e responsabilità non basta conoscere il linguaggio di programmazione: occorre comprendere il problema che il software deve rappresentare. Le scelte e i pattern legati al dominio dipendono quindi dalle caratteristiche di quel contesto.

Lo *SDD* (*Specification Driven Development*) pone al centro le specifiche, anche per guidare strumenti di intelligenza artificiale che producono parti dell'implementazione. La precisione della richiesta è essenziale, e il codice generato deve essere confrontato con le specifiche mediante verifiche e test. L'applicabilità dipende dal sistema: nei progetti grandi, risultati inventati dai modelli linguistici di grandi dimensioni (LLM, *Large Language Models*), detti *allucinazioni*, possono compromettere la coerenza del lavoro. Disporre di un algoritmo generato non risolve da solo i problemi di architettura, integrazione e verifica.

Dominio, specifiche e test possono contribuire allo stesso sviluppo: chiariscono rispettivamente il contesto del problema, il comportamento richiesto e le evidenze della sua realizzazione. Un'eventuale evoluzione di Scrum verso una maggiore integrazione dell'IA resta una prospettiva distinta dall'adozione di strumenti nel lavoro quotidiano: da sola non definisce nuovi ruoli, eventi o artefatti.

Capitolo 11

Evoluzione, manutenzione, metriche e difetti

11.1 Il cambiamento dopo la consegna

Un prodotto software consegnato non è necessariamente un prodotto concluso. Il software è intrinsecamente flessibile e, quando mutano requisiti o ambiente di uso — per esempio business o hardware — deve cambiare per continuare a essere utile. La *manutenzione* è il processo di introduzione di modifiche a un prodotto software dopo la consegna al cliente. Il termine *evoluzione* è spesso usato come sinonimo di manutenzione; può anche indicare un singolo intervento di cambiamento entro un ciclo che comprende rilascio di patch ed eventuale dismissione del sistema.

Una *metrica* rende osservabile un attributo del software mediante un criterio di misurazione. Può aiutare a riconoscere dove l'evoluzione sta aumentando complessità e dipendenze, ma il suo valore va interpretato insieme a requisiti, responsabilità e comportamento del sistema.

11.2 Manutenzione ed evoluzione

11.2.1 Tipi e costo delle modifiche

Le modifiche successive alla consegna sono classificate in quattro categorie:

Correttive. Rimuovono errori o *bug*, cioè comportamenti non conformi a quanto previsto.

Adattative. Adeguano il sistema a un nuovo ambiente, per esempio a una piattaforma, a un sistema operativo o a hardware cambiato.

Perfettive. Migliorano il prodotto o aggiungono funzionalità.

Preventive. Modificano internamente il sistema per prevenire problemi futuri; il refactoring ne è un esempio.

La manutenzione può assorbire una parte molto rilevante del costo complessivo: il 67–80% è un ordine di grandezza orientativo. Una ripartizione statistica esemplificativa assegna il 17% agli interventi correttivi, il 18% agli adattativi, il 60% ai perfettivi e il 5% ai preventivi. Queste quote descrivono un possibile profilo, non prevedono la distribuzione di ogni progetto. Il peso delle estensioni motiva la preparazione al cambiamento già nel design: la parametrizzazione rende variabili valori o scelte, mentre l'incapsulamento nasconde decisioni interne dietro contratti stabili.

11.2.2 Costo e modelli di manutenzione

La partecipazione del team originario può ridurre il costo della manutenzione grazie alla conoscenza del sistema. Se però chi sviluppa non ha responsabilità contrattuali sulle modifiche successive, può avere incentivi insufficienti a predisporre il cambiamento. Qualunque sia il team incaricato, interventi non governati possono degradare progressivamente la struttura.

Si distinguono tre modalità di intervento:

Quick-fix. Applica cambiamenti direttamente al codice. È il modello più immediato, ma non impone un riesame della struttura.

Miglioramento iterativo. Esegue le modifiche dopo un'analisi del sistema esistente, controllandone la complessità e conservandone il design.

Riuso. Stabilisce i requisiti del nuovo sistema e riusa quanto più possibile del sistema esistente.

La distinzione chiarisce perché correggere un problema non equivale sempre a mantenere bene il software: la modifica deve anche preservare comprensibilità e possibilità di ulteriori cambiamenti.

11.2.3 Ristrutturazione e ricostruzione

Il *refactoring*, o *restructuring*, è un cambiamento che migliora la struttura interna senza alterare il comportamento del codice. Non aggiunge quindi una funzionalità e non è, da solo, la correzione di un difetto osservabile. Il *reverse engineering* analizza un sistema per ricavare informazioni sul suo comportamento o sulla sua struttura; il *re-engineering* altera invece il sistema per ricostituirlo in una forma diversa. Queste definizioni separano rispettivamente miglioramento interno, conoscenza dell'esistente e ricostruzione.

11.3 Dinamiche e leggi di Lehman

Le *dinamiche di evoluzione* sono i processi di cambiamento di un sistema. Le leggi di Lehman derivano da studi empirici condotti da Lehman e Belady dal 1968, e poi confermati da studi successivi, su grandi sistemi realizzati in grandi aziende. Non sono dunque regole dimostrate per qualsiasi programma.

Cambiamento continuo (1974). Un sistema deve essere adattato continuamente; altrimenti diventa progressivamente meno soddisfacente.

Aumento della complessità (1974). Durante l'evoluzione la struttura tende a diventare più complessa, salvo che si svolga lavoro esplicito per preservarla o semplificarla.

Auto-regolazione (1974). Attributi quali dimensione, intervallo fra release e numero di errori rilevati in ciascuna release sono approssimativamente invarianti.

Stabilità organizzativa (1978). Nel ciclo di vita di un sistema il tasso di sviluppo è circa costante e indipendente dalle risorse impiegate.

Conservazione di familiarità (1978). In media l'incremento di crescita del sistema tende a restare costante oppure a diminuire.

Continua crescita (1978). Per mantenere la soddisfazione dell'utente occorre incrementare continuamente il contenuto funzionale del sistema.

Diminuzione della qualità (1994). La qualità diminuisce se non è gestita e adattata rigorosamente durante i cambiamenti.

Il campo di applicazione riguarda soprattutto sistemi grandi prodotti da grandi organizzazioni. L'estensione delle stesse regolarità a piccoli prodotti, piccole organizzazioni o sistemi con molti COTS non è automaticamente giustificata: cambiano dimensione, controllo sul codice e modalità di evoluzione.

11.4 Metriche del software

11.4.1 Scopo e limiti della misurazione

Una metrica definisce un insieme di misure per un sistema software. Gli obiettivi sono monitorare il prodotto mentre è costruito, identificare il livello raggiunto da ciascuna metrica e intervenire quando il livello non è soddisfacente. Gli *attributi interni*, quali dimensione e struttura del codice, sono misurabili direttamente; gli *attributi esterni*, quali qualità percepita o manutenibilità, sono ricavati indirettamente. Perciò un valore non è una prova automatica di qualità o di difetto, ma un'indicazione da interpretare nel contesto del progetto.

11.4.2 Dimensione e commenti

LOC (*Lines Of Code*) misura la dimensione in linee di codice. NCNB (*Non Comment Non Blank*) esclude invece commenti e righe vuote. La convenzione concreta di conteggio deve essere dichiarata: nel caso di `CalcolaImporti` considerato qui, LOC conta le linee che terminano con `;`. Il confronto fra versioni e progetti è significativo soltanto se mantiene la medesima convenzione.

La *Comment Percentage* esprime la percentuale di commenti rispetto alle LOC. Un riferimento del 30% può orientare una valutazione, ma non è una soglia universale: conta soprattutto che i commenti chiariscano intenzioni e decisioni. Si considerino `CalcolaImporti` 0.1, con un metodo e 18 LOC, e la versione 0.2, con un metodo e 23 LOC. `Pagamenti` 1.1 ha invece otto metodi e 27 LOC: la media è $27/8 = 3{,}375$, circa 3,4 LOC per metodo. Sono misure di quantità; da sole non descrivono la difficoltà del comportamento.

Un confronto quantitativo esemplificativo considera anche due basi di codice:

Sistema	LOC	Classi	Metodi	Attributi	LOC/metodo
JUnit	circa 22000	231	1200	265	circa 18
JHotDraw	circa 28000	600	4814	1151	circa 6

Sono fotografie di specifiche basi di codice e convenzioni di conteggio, non misure delle versioni correnti delle librerie. La diversa media non dimostra da sola che uno dei due progetti sia migliore: occorre considerare responsabilità e struttura.

Un metodo breve può avere molte dipendenze o una condizione difficile da testare; un metodo più lungo può contenere una sequenza semplice. Per scegliere il refactoring si combinano quindi dimensione, coesione, accoppiamento e responsabilità.

Preferire metodi sotto le 30 righe è un criterio orientativo per riesaminare la leggibilità, non un limite di correttezza. Anche un valore medio di circa 18 LOC per metodo, associato a JUnit come esempio comparativo, richiede di precisare versione e convenzione di conteggio prima di usarlo per un confronto. Un numero di righe va sempre letto insieme alle responsabilità del metodo.

La complessità ciclomatica completa le misure di dimensione osservando i percorsi indipendenti del flusso di controllo. Il suo rapporto con i test rende possibile distinguere una classe corta ma ricca di decisioni da una classe più lunga con flusso semplice. Le metriche delle classi aggiungono a questa analisi le dipendenze e l'organizzazione a oggetti.

11.5 Metriche CK per sistemi a oggetti

La suite di Chidamber e Kemerer, o *metriche CK*, misura aspetti delle classi di un sistema orientato agli oggetti. Le metriche non sono giudizi isolati: rendono visibili, da prospettive diverse, numero e complessità dei metodi, profondità e ampiezza dell'ereditarietà, dipendenze, risposta ai messaggi e coesione.

11.5.1 Metodi ed ereditarietà

WMC (Weighted Methods per Class). È la somma dei pesi di complessità dei metodi: $WMC = \sum_{j=1}^{m} c_j$, con m metodi e peso c_j del metodo j. Se ogni peso è unitario, WMC coincide con il numero dei metodi; con un peso comune c vale invece cm. Un WMC alto può segnalare maggiore lavoro di manutenzione e maggiore specificità della classe, con minori possibilità di riuso.

DIT (Depth of Inheritance Tree). È il massimo numero di livelli dalla classe alla radice della gerarchia; la radice ha livello zero. Un DIT alto può rendere più difficile comprendere il comportamento, perché aumentano i metodi ereditati da considerare; può però indicare maggiore riuso.

NOC (Number of Children of a Class). È il numero di sottoclassi dirette di una classe. Un NOC maggiore può indicare più riuso e rende più ampia l'influenza della classe sul sistema: modificare una classe con molte discendenti può incidere su più comportamenti.

11.5.2 Accoppiamento, risposta e coesione

CBO (Coupling Between Object Classes). È il numero di classi con cui una classe interagisce. Un CBO alto esprime più dipendenze da altre classi e può rendere più difficile riusare, comprendere, modificare e correggere la classe.

RFC (Response for a Class). È il numero di metodi potenzialmente eseguibili in risposta a un messaggio: comprende i metodi della classe e i metodi invocati da ciascuno di essi. Un RFC alto segnala un insieme più ampio di comportamenti da comprendere e verificare.

LCOM (Lack of Cohesion of Methods). Il nome indica la mancanza di coesione dei metodi. Per interpretare un valore occorre precisare la formula adottata. Si adotta qui la definizione normalizzata:

$$LCOM = 1 - \frac{\sum_{i=1}^{a} mA_i}{a \cdot m}. \tag{11.1}$$

Qui a è il numero di campi della classe, m il numero di metodi e mA_i il numero di metodi che usano il campo A_i, dove i varia da 1 ad a. Il numeratore conta le coppie campo–metodo collegate; il denominatore $a \cdot m$ è il numero massimo di tali coppie. La formula richiede $a > 0$ e $m > 0$. Indicando con r il rapporto sottratto, r misura la densità d'uso dei campi e $LCOM = 1 - r$ ne è il complemento.

Con tre campi e tre metodi si ottengono quattro casi istruttivi:

Uso dei campi	Rapporto r	$LCOM = 1 - r$
Ogni metodo usa tutti e tre i campi	$9/9 = 1$	0
Ogni metodo usa due campi	$6/9 = 2/3$	$1/3$
Ogni metodo usa un campo	$3/9 = 1/3$	$2/3$
Un solo metodo usa tutti i campi; gli altri nessuno	$3/9 = 1/3$	$2/3$

I valori 1, 2/3, 1/3, 1/3 appartengono al rapporto d'uso, mentre l'indice di mancanza di coesione assume rispettivamente 0, 1/3, 2/3, 2/3. Non si devono scambiare le due quantità: per questa definizione un LCOM basso indica maggiore condivisione dei campi. Gli ultimi due casi mostrano anche un limite: strutture diverse possono produrre lo stesso valore. La metrica orienta quindi l'esame delle responsabilità, senza sostituirlo.

11.6 Difetti, metriche e interventi

Un *difetto* è riconoscibile confrontando il comportamento del programma con una *specifica*, cioè con la descrizione di ciò che esso deve fare: un comportamento anomalo segnala che il programma non soddisfa tale riferimento. Si distinguono difetti di specifica, quando le specifiche sono ambigue o imprecise; difetti di progettazione, quando le componenti sono progettate non correttamente; e difetti di codice, derivanti dall'implementazione.

Metriche e test hanno ruoli complementari. LOC, *cc* e CK osservano proprietà interne e aiutano a localizzare classi o metodi che meritano analisi; i test esercitano il comportamento rispetto ai risultati attesi. La correzione di un difetto richiede di: localizzare l'errore, correggere prima il modello e poi il codice, infine rieseguire il test. Se non vi è un difetto di comportamento ma la struttura ostacola le modifiche, il refactoring è l'intervento appropriato, protetto dai test di regressione.

11.7 Esempio integrato

Si consideri una classe `Pagamenti` con otto metodi e 27 LOC, secondo il conteggio appena considerato. Il suo valore medio di 3,4 LOC per metodo non basta a stabilire se la classe sia facile da mantenere. Si procede allora con misure complementari: si calcola WMC sommando le complessità degli otto metodi; si osserva CBO per sapere quante altre classi coinvolge; si determina RFC contando i metodi attivati da un messaggio; si esamina la relazione fra campi e metodi, esplicitando la convenzione di coesione prima di attribuirle un valore LCOM.

Se uno dei metodi contiene una condizione, il suo grafo di controllo può avere due percorsi indipendenti e $cc = 2$; si preparano pertanto un test per ciascun esito. Se il test rivela un risultato diverso dalla specifica, si tratta di un candidato difetto: si corregge modello e codice e si ripete il test. Se invece i test restano corretti ma CBO o WMC richiamano l'attenzione su una struttura molto dipendente o complessa, il team può analizzare l'opportunità di una ristrutturazione. Valori alti o bassi richiedono interpretazione nel contesto: non esistono soglie universali ricavabili dal solo nome di una metrica.

Glossario dei termini tecnici

accoppiamento	Grado di dipendenza fra componenti software; dipendenze non necessarie rendono i cambiamenti più costosi.
artefatto	Prodotto intermedio o finale di un'attività di sviluppo, per esempio requisito, modello, codice o test.
backpressure	Controllo del flusso nel quale il consumatore comunica quanti elementi è pronto a ricevere.
classe	Modello che definisce stato e operazioni di oggetti dello stesso tipo.
coesione	Coerenza interna di dati e operazioni di un componente rispetto alla sua responsabilità.
Command–Query Separation	Separazione fra operazioni che modificano lo stato e operazioni che restituiscono informazioni senza modificarlo.
composition root	Punto dell'applicazione che costruisce e collega le istanze, fornendo le dipendenze ai componenti.
contratto	Insieme di operazioni e condizioni osservabili che un componente garantisce ai client.
DDD	Sviluppo guidato dalla comprensione del dominio applicativo, che orienta il modello e le responsabilità degli oggetti.
debito tecnico	Costo futuro dovuto a una soluzione rapida o poco curata che ostacola comprensione e modifica del software.
dependability	Capacità di offrire un servizio su cui sia possibile riporre fiducia giustificata; comprende aspetti di reliability, safety e security.
Dependency Injection	Tecnica con cui un oggetto riceve dall'esterno i propri collaboratori anziché costruirli direttamente.
design pattern	Soluzione specializzabile a un problema di progettazione ricorrente in un dato contesto.

DRY	Don't Repeat Yourself: evitare rappresentazioni duplicate della stessa conoscenza.
feature	Insieme di requisiti correlati che consente all'utente di raggiungere un obiettivo.
hook	Operazione con comportamento predefinito che una sottoclasse può ridefinire per personalizzare un punto di un algoritmo.
interfaccia	Contratto di operazioni visibile ai client; in Java è anche il costrutto linguistico che può dichiarare tale contratto.
iterazione	Ciclo di lavoro delimitato in cui si pianifica, realizza e verifica un incremento o una parte del prodotto.
KISS	Keep It Simple, Stupid: preferire soluzioni semplici da comprendere e gestire.
Law of Demeter	Principio che limita la conoscenza dei dettagli di oggetti lontani favorendo comunicazioni con collaboratori diretti.
manutenibilità	Facilità con cui il software può essere compreso, corretto, testato e modificato.
metrica	Misura di una proprietà del software usata per osservarne dimensione, struttura o andamento.
modello	Rappresentazione selettiva di aspetti del sistema, usata per comprenderlo, progettarlo o comunicarlo.
oggetto	Istanza di una classe, dotata di stato e operazioni.
oracle	Riferimento che consente a un test di stabilire se il risultato osservato è corretto.
polimorfismo	Proprietà per cui una stessa invocazione può produrre comportamenti diversi in base al tipo concreto del ricevente.
refactoring	Trasformazione che migliora la struttura interna preservando il comportamento osservabile.
registro dei prototipi	Associazione fra chiavi semantiche e oggetti già configurati da cui ottenere nuove istanze mediante copia.

requisito	Caratteristica, servizio o vincolo che il sistema deve soddisfare.
responsabilità	Compito coerente assegnato a un elemento del sistema; è anche un motivo di cambiamento.
risposta	Risultato o effetto prodotto dal sistema in seguito a una richiesta, un input o un evento.
SDD	Specification Driven Development: approccio che mette al centro le specifiche nel guidare anche l'implementazione assistita da IA.
stream	Sequenza di elementi elaborata da una pipeline; non è una collection che memorizza gli elementi.
test case	Caso di test con stato iniziale, input, oracle e risultato atteso.
test di regressione	Riesecuzione di test dopo una modifica per controllare che comportamenti già corretti restino tali.
UML	Unified Modeling Language: notazione grafica standardizzata per modellare struttura e comportamento di un sistema.
validazione	Controllo che il prodotto soddisfi bisogni e richieste dell'utente.
verifica	Controllo che gli artefatti siano costruiti correttamente rispetto a specifiche e progettazione.
YAGNI	You Ain't Gonna Need It: implementare le funzionalità richieste evitando estensioni puramente speculative.